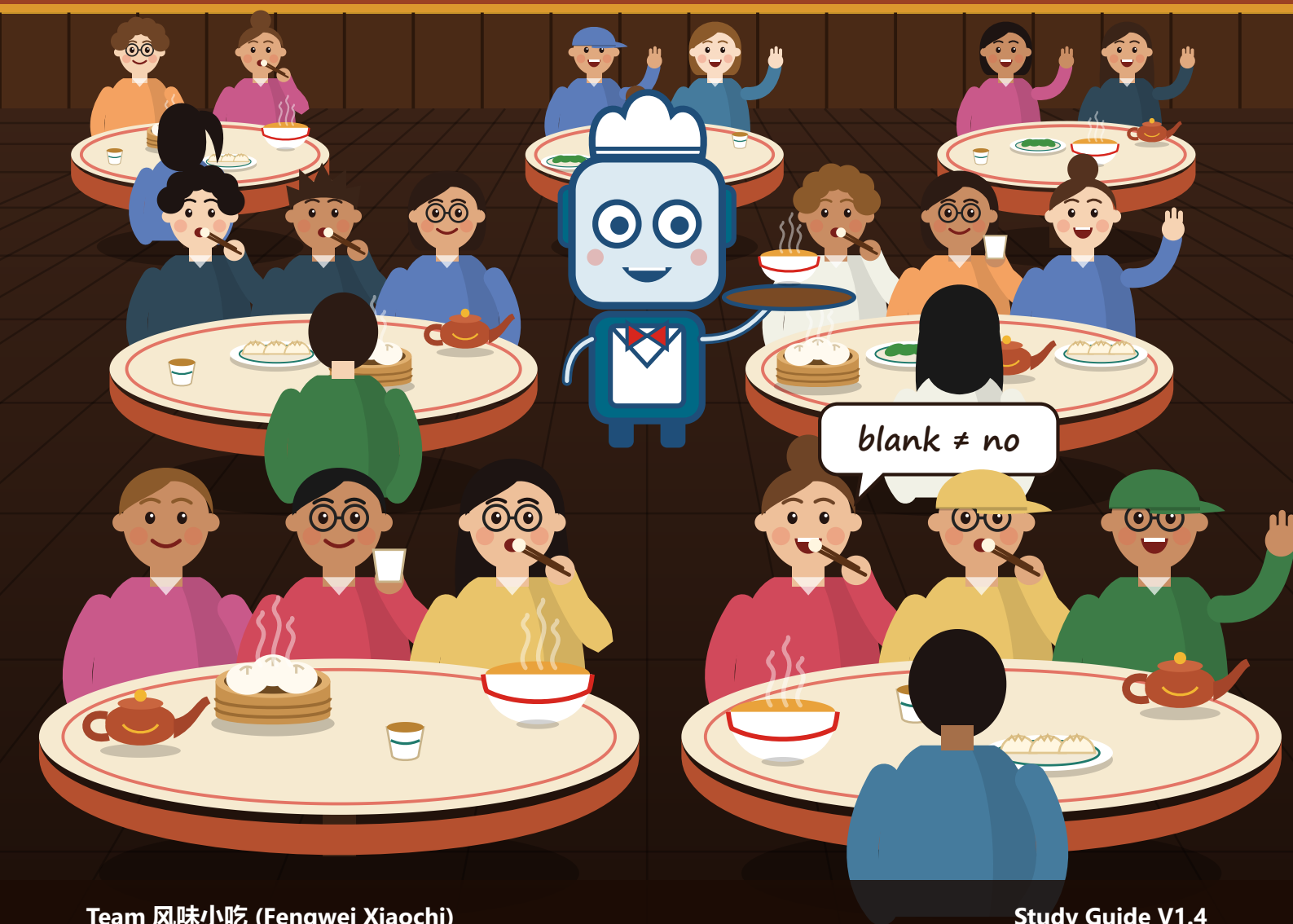
2026 · STUDY GUIDE BOOK

SOFTWARE AND SYMBOLIC AI

Help Pip, the new library robot, find out how software reaches a decision



Team 风味小吃 (Fengwei Xiaochi)

Study Guide V1.4

OUR MEMBERS

RESERVED FOR
Gunawan Nicolas Benedictus

李庆成
1820262104
Coordinator

RESERVED FOR
Ritchard Alexander

林俊宇
1820262050
Content Lead

RESERVED FOR
Eizel Mthana Conqueror

康哲
1820262106
Visual Design Lead

RESERVED FOR
Frederic Chandra

张国立
1820262083
Quality Lead

RESERVED FOR
Louis Alexander Dalim

苏路成
1820262103
Repository and Submission Lead

RESERVED FOR
Frederick Bryan The

郑明畅
1820262029
Video Lead



FENGWEI XIAOCHI

Come in. Sit down. Order a chapter.



TABLE OF CONTENTS

Tonight's menu

Learning suggestion 7

Welcome: meet Pip 8

Optional game: Rulebound 12

Part A: Software and Algorithms 14

Chapter 1: Software, AI and Symbolic AI 15

Check yourself 1.1 — Is it AI? 18

Chapter 2: Software Overview 20

Activity 2.1 — Who does what? 24

Chapter 3: Algorithms: How Programs Process Information and Make Decisions 27

Activity 3.1 — Trace and debug 30

Chapter 4: Data Structures 35

Activity 4.1 — Choose a structure and trace BFS 39

Chapter 5: Software Engineering and Databases 41

Activity 5.1 — Design and test a small program 47

Part B: Symbolic AI 50

Chapter 6: Introduction to Symbolic AI 51

Activity 6.1 — Will the rule fire? 55

Chapter 7: Logic and Reasoning 57

Activity 7.1 — Formalize and reason 61

Chapter 8: Knowledge Representation 64

Activity 8.1 — Represent the same case in different ways 68

Chapter 9: Rule-Based and Expert Systems 70

Activity 9.1 — Be the inference engine 75

Activity 9.2 — Predict, query, and change a rule 77

Chapter 10: Planning and Problem Solving 80

Activity 10.1 — Get everyone across the river 82

Activity 10.2 — Plan a route 85

Chapter 11: Symbolic Natural Language Processing 90

Activity 11.1 — Build a parse tree 93

Activity 11.2 — Be ELIZA, then meet her 96

Chapter 12: Software and Symbolic AI Together 99

Activity 12.1 — Change the library policy 102

Hands-on 12.2 — Rule-based assistant vs. machine-learning assistant 103

Pip's last day as a trainee: bringing the module together 112

Quick Reference Glossary 114

Resources, references and credits 117



ABOUT US

风味小吃 (Fengwei Xiaochi) is six students on the Software and Symbolic AI module. Our name means local-flavour snacks: the small, shared dishes you order when you want to stay a while.

That is how we want this subject to feel. Big ideas get easier when they arrive in small plates, and when there is someone across the table to argue with. So we built this book like a restaurant full of people: come hungry, try things, talk with your mouth full of questions, and stay as long as you like.

Each of us led different chapters. The team's roles and contributions are listed at the back of the book.

ABOUT THE BOOK

This book helps beginners find out how software reaches a decision. It has two parts. **Part A, Software and Algorithms,** explains how ordinary software is built and how it follows written steps. **Part B, Symbolic AI,** explains how software can reason from facts and rules that people can read, check and change.

Every chapter is a mission for **Pip**, the library's brand-new robot helper. After each library shift, Pip waits tables at our restaurant and tells the diners what went wrong today. Your job is to work out why, and to fix it. You will predict, trace, build and break things: 12 missions, 16 activities and 25 quick challenges.

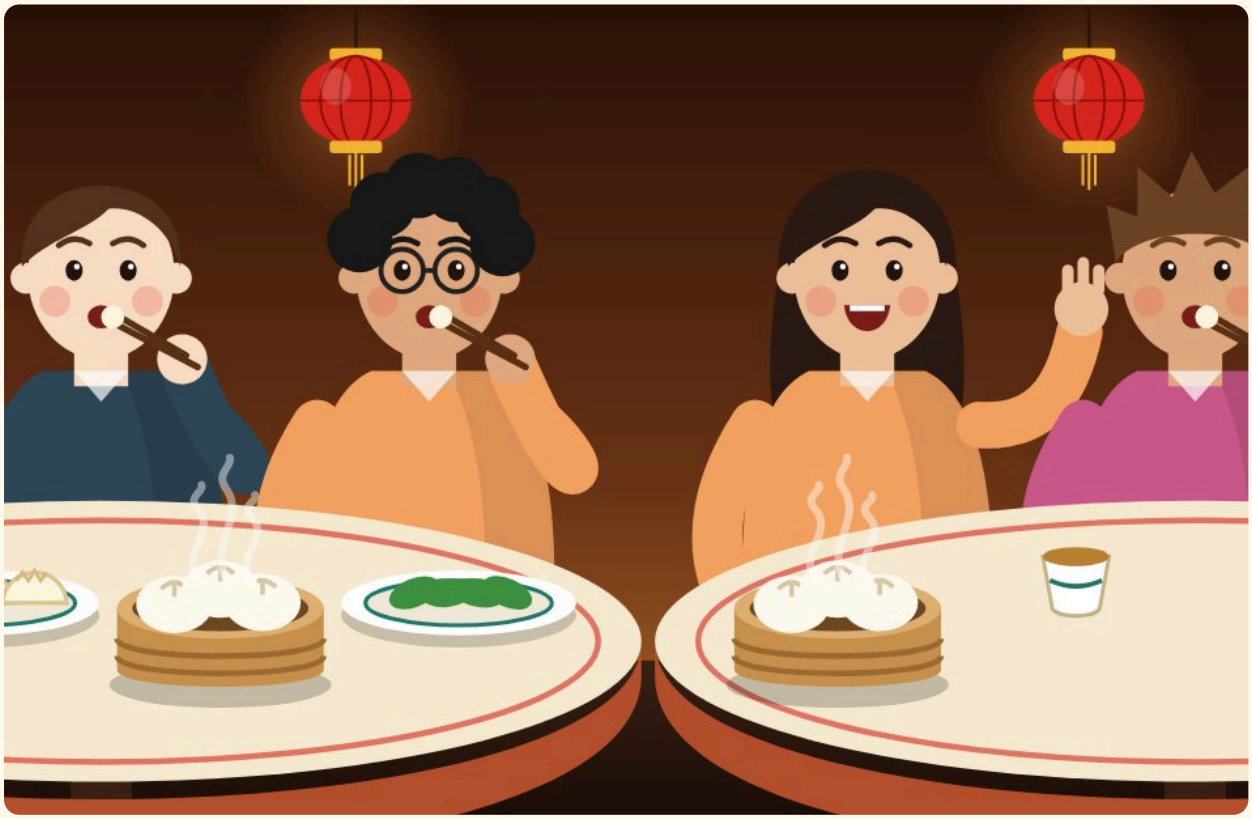
The kitchen and the dishes are only comparisons to help you remember. The technical explanation in each chapter is the one to trust, and it is the one the Test Bank asks about.

Team	风味小吃 (Fengwei Xiaochi)
Module	Software and Symbolic AI
Version	Study Guide V1.4
Based on	Module Contents V3, V2.0 (4 November 2026)
Goes with	The Test Bank, *Pip's Library Trials*, and the team's PPT slides and Video Tutorials

12
Missions
One for each chapter

16
Activities
Paper, cards and real tools

25
Quick challenges
Each with its check below



Learning Suggestion

Order one dish at a time

Work through the chapters in order. Each one uses ideas from the one before, so skipping ahead is like starting dinner with the bill.

Taste before you read the menu

When you see **Predict first**, write your answer before you read on. A wrong guess you can explain teaches more than a right guess you cannot.

Eat with friends

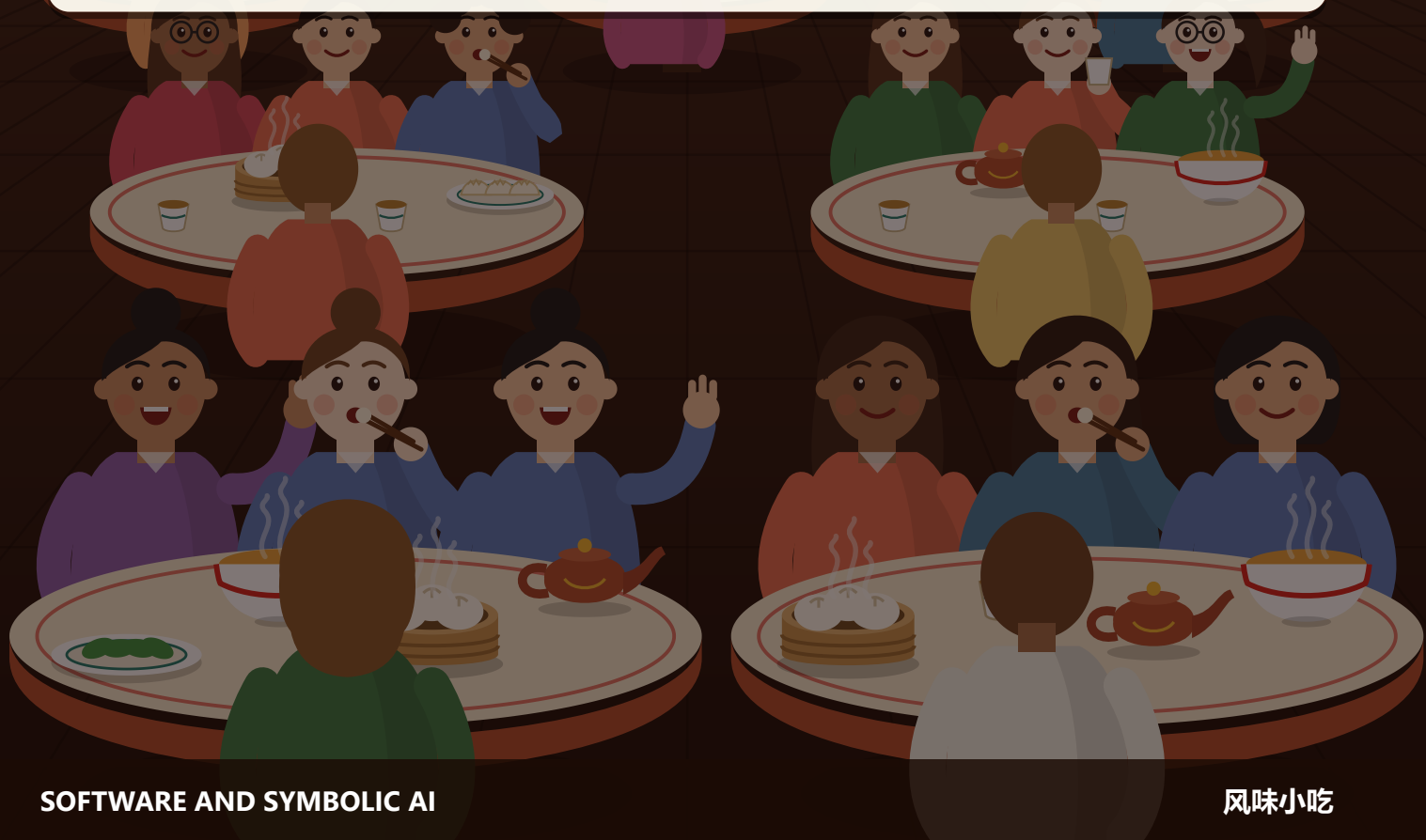
This book is meant to be talked about. Explain each idea to the person next to you. If you cannot explain it, go back one page.

Visit the kitchen

Do not only read about the tools. Use the cards, Scratch, SWISH and ELIZA, and watch the team's Video Tutorials. Each one has instructions next to its activity.

Come back for seconds

Check yourself with the Test Bank, *Pip's Library Trials*. Its Score Sheet sends you back to any chapter you need. Coming back is part of the meal.

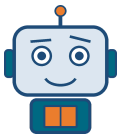


WELCOME: MEET PIP

This guide follows two cases throughout the module:

- **The umbrella decision:** Should you take an umbrella when you leave for class?
- **The library loan desk:** May a member borrow a particular book?

These cases begin with familiar decisions, then help us explore how software processes information, how symbolic AI represents facts and rules, and how different methods search for solutions.



Meet Pip

Pip is the library's brand-new robot helper. Pip is fast, polite and tireless, and takes every instruction completely literally. In each chapter, Pip makes a mistake that real software and real beginners also make. Your job is to find out why, and to fix it.

Pip is a made-up character. Real programs do not feel confused or try their best: they follow their instructions and their data. When this guide says "Pip thinks", read it as "the program does".

You do not need previous experience with programming, AI, or formal logic. You only need to be able to use a computer and read short written instructions in English. Everything else is taught before it is used.

Learning goals

By the end of this guide, you should be able to:

- Explain the difference between software, AI, machine learning, and symbolic AI.
- Describe what an operating system does and distinguish system software from application software.
- Trace an algorithm and explain how sequence, selection, and iteration control its steps.
- Choose suitable data structures and explain how breadth-first search uses a queue.
- Describe the software life cycle, design basic tests, and explain how relational databases use tables and keys.
- Read simple logical statements and explain how facts and IF–THEN rules support conclusions.
- Compare knowledge representations, rule-based reasoning, and search strategies.
- Explain the basic steps in a symbolic natural language processing system.
- Compare fixed program logic, explicit rules, and machine-learning systems, including their limitations.

How each chapter works

Every chapter follows the same route: **meet a problem → predict or try → observe → explain → apply → check**. The same labels appear everywhere, so you always know what to do next.

Label	What it means for you
Pip's problem	Starts the chapter with something going wrong, and what you will be able to do by the end.
Predict first	Write down what you think before you read on. Later in the chapter, you find out if you were right.
Try it, **Mini-challenge**	A short task with its check straight below. Cover the check until you have answered.
Activity, **Check yourself**, **Hands-on**	A bigger task: goal, preparation, task, observe, reflect, then feedback. Some also give a hint.
Common mistake	A trap that catches many beginners, and how to avoid it.
Resource	A chosen video or website: why to use it, what to do with it, and what to watch out for.
Pip says	Pip's own view, which is usually funny and sometimes wrong. Pip is a made-up character, so a technical explanation always comes first.
Mission complete	The takeaway, a checklist of what you can now do, and the Test Bank questions to try next.
Checkpoint	A short review that joins several chapters together.
Table talk	A question to discuss with the person next to you. On your own, write your answer instead.

Your route through the module

Work through the chapters in order. Each one uses ideas from the one before. Read the chapter, watch the matching video, do the activities, and then use the separate Test Bank, *Pip's Library Trials*, to check yourself. Review any question you cannot explain.



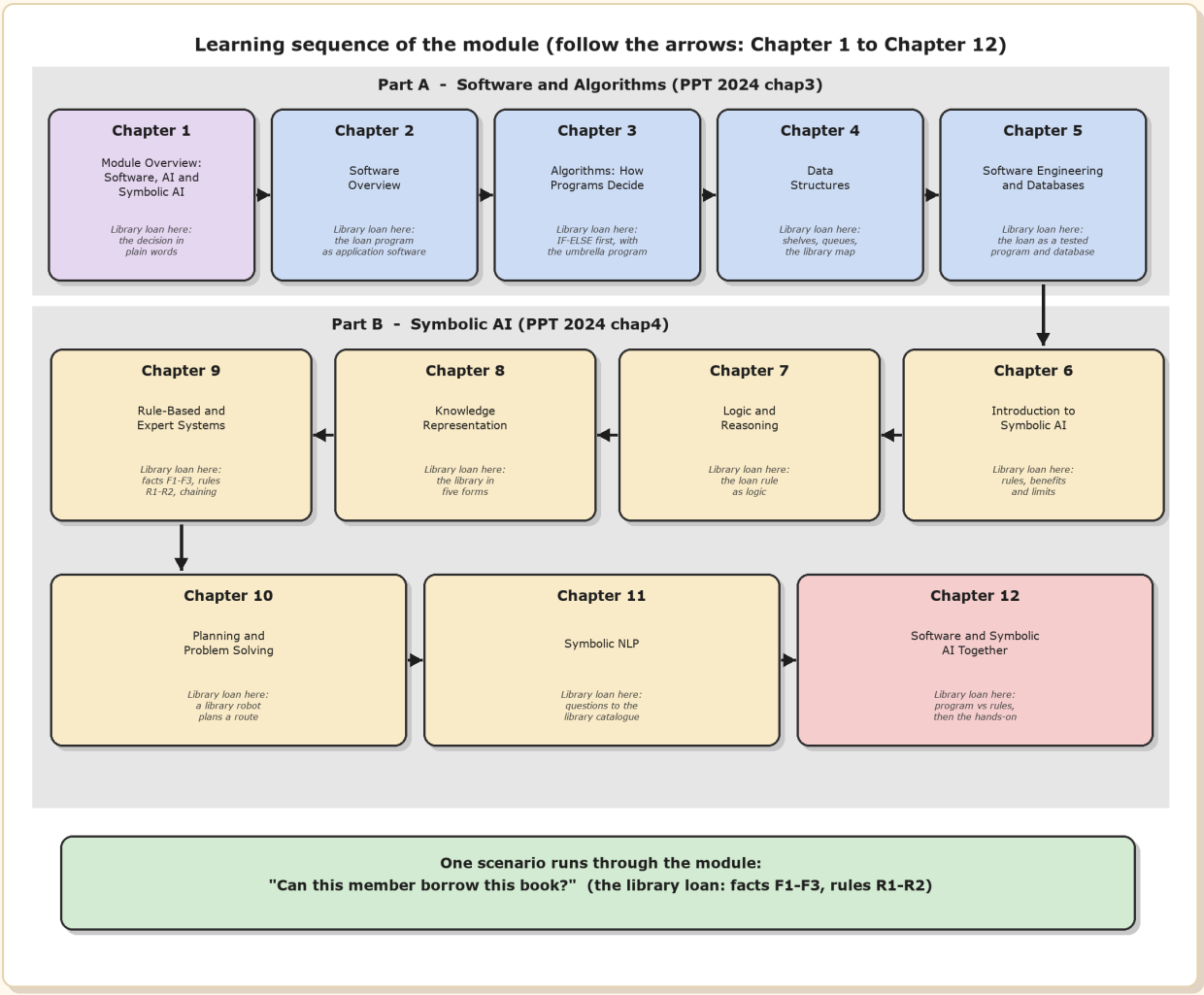
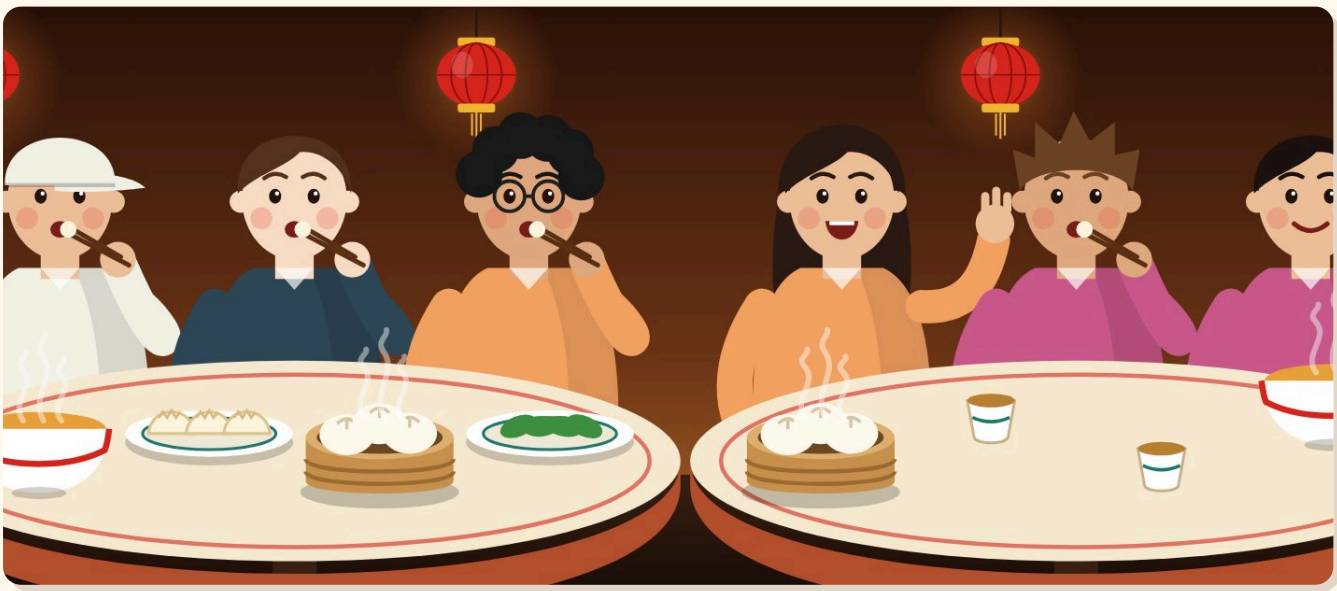


Figure 0.1. The twelve chapters in learning order, with the library-loan case that runs through them.



Ch.	Topic	Main activity	You need	Video	Test Bank
1	Software, AI and symbolic AI	Check yourself 1.1: Is it AI?	Paper	01	T01–T02, M01–M02, O01
2	Software overview	Activity 2.1: Who does what?	Paper	01	T03–T04, M03–M04, M32, O01
3	Algorithms	Activity 3.1: Trace and debug	Paper	02	T05–T06, M05–M06, O01
4	Data structures	Activity 4.1: Trace BFS	Paper	02	T07–T08, M07–M08, M37, O02
5	Software engineering and databases	Activity 5.1: Design and test	Paper	03	T09–T10, M09–M10, M33–M34, O03
6	Introduction to symbolic AI	Activity 6.1: Will the rule fire?	Paper	04	T11–T12, M11–M12, O05
7	Logic and reasoning	Activity 7.1: Formalize and reason	Paper	04	T13–T14, M13–M14, M38, O04
8	Knowledge representation	Activity 8.1: Same case, different forms	Paper	05	T15–T16, M15–M16, M31, O04
9	Rule-based and expert systems	Activity 9.1: Be the inference engine; 9.2: Prolog	Paper; browser	05	T17–T18, M17–M18, M25, M35, O05
10	Planning and problem solving	Activity 10.1: River puzzle; 10.2: Plan a route	Paper	06	T19–T20, M19–M20, M26, M29–M30, O02
11	Symbolic language processing	Activity 11.1: Parse tree; 11.2: ELIZA	Paper; browser	07	T21–T22, M21–M22, M27, O06
12	Software and symbolic AI together	Activity 12.1: Policy change; Hands-on 12.2	Paper; browser	07	T23–T24, M23–M24, M28, M36, O03, O06

Reading the codes. T = True/False, M = Multiple Choice, O = Open-ended. "Video 01" means the team's own 风味小吃 Video Tutorial with that number, from the team's Video Tutorials submission. [TEAM: add the link or folder name] Attempt a Case File (O) only after finishing every chapter it covers.



Optional game: Rulebound

Resource: Rulebound (the team's own game)

What is it? Even robots take breaks. In the staff room, Pip found an arcade cabinet: a one-on-one pixel fighting game made for this module, with five fighters, five battlegrounds and twelve stages, one for each chapter.

Why use it? Each stage opens with the chapter's main points and a false claim, and winning shows the truth behind the claim. Every ability is named after an idea from a chapter, and the game explains the link. The rival's IF-THEN rules appear on screen as it fights. With Study mode on, each ability first asks a question from the stage's chapter, and every answer is explained.

How to open it Extract the whole ZIP first, then double-click `Supporting_Files/Game_Rulebound/rulebound.html`. It runs in a browser, with no installation, sign-in or internet. On a phone, save the file and open it in Chrome or Safari, though some phones will not run a saved page. `README.md` in the same folder has the controls and explains how the rival works.

What to do After a chapter, turn on Study mode in Settings and play that chapter's stage. Write down one question you got wrong, and the correct idea in your own words. The result screen then lists the rules your rival used most: pick one and explain when it would fire.

Watch out The rival uses fixed IF-THEN rules, forward chaining and priorities, as in Chapters 6, 9 and 12. It does not search or learn, and it contains no machine learning or reinforcement learning. Ability names are memory aids, not exact descriptions. The Study mode questions are practice, and they are separate from the Test Bank.

If you cannot open it Nothing else depends on the game. Use the Test Bank and each chapter's activities instead.



Figure 0.2. A fight at Ravenmoor Square. Forward Chain sends three linked hits, and the line at the bottom shows the IF–THEN rule the rival just used.

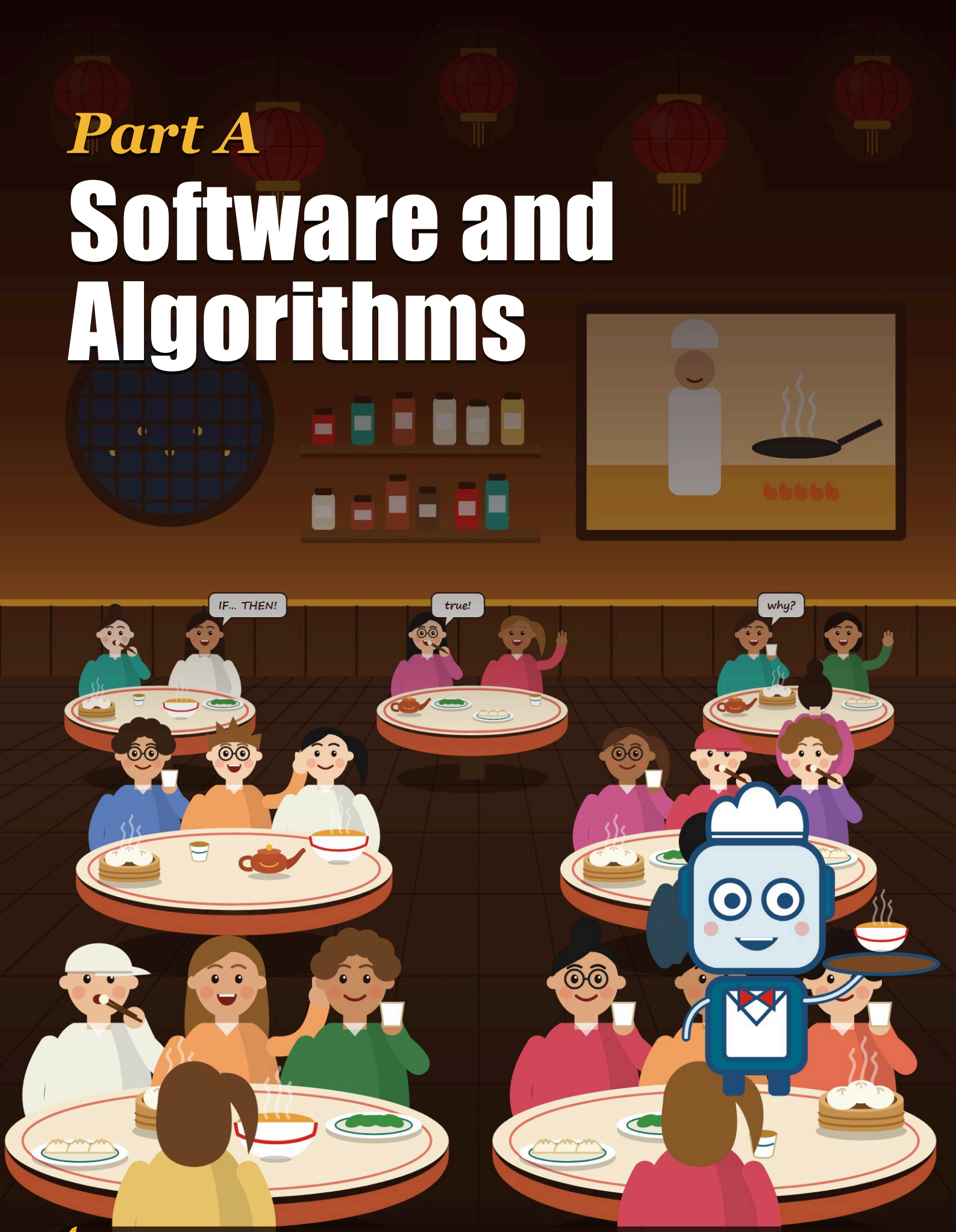


Figure 0.3. With Study mode on, an ability first asks a question from the stage's chapter. The explanation appears after every answer, right or wrong.



Part A

Software and Algorithms



Chapters 1–5 explain how ordinary software is built and how it follows written steps. By the end of Part A, Pip can run the library's loan program, and you will be able to say exactly how it works and where it goes wrong.

Chapter One

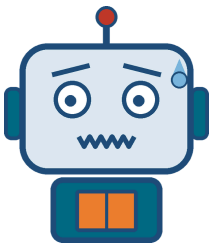
01 SOFTWARE, AI AND SYMBOLIC AI

Software, AI and Symbolic AI



On today's menu: steamed buns

A bun looks simple until you ask what is inside. So does the word "AI".



Pip's problem

On Pip's first day, a visitor asks: "Are you intelligent, or do you just follow instructions?" Pip has no idea. Worse, Pip proudly tells everyone that the library's calculator app is "AI, because it always gets the answer right."

Your mission

Help Pip sort out what software is, what AI means, and where symbolic AI fits.

You will be able to

- Explain the difference between software, AI, machine learning, and symbolic AI.
- Represent a familiar decision as facts and a rule.

You need: Nothing: this is the start.

Watch: Video 01. While watching, ask: what makes something AI rather than just useful software?

Start with a decision

Imagine that it is raining and you are going outside. **Before reading on, decide what you would do and which facts you used.** A simple system can represent the situation with two facts and a rule:

```
Facts:  raining
        going_outside
Rule:   IF raining AND going_outside THEN recommend_umbrella
```

A program can receive these facts, check the rule's conditions, and display the recommendation.

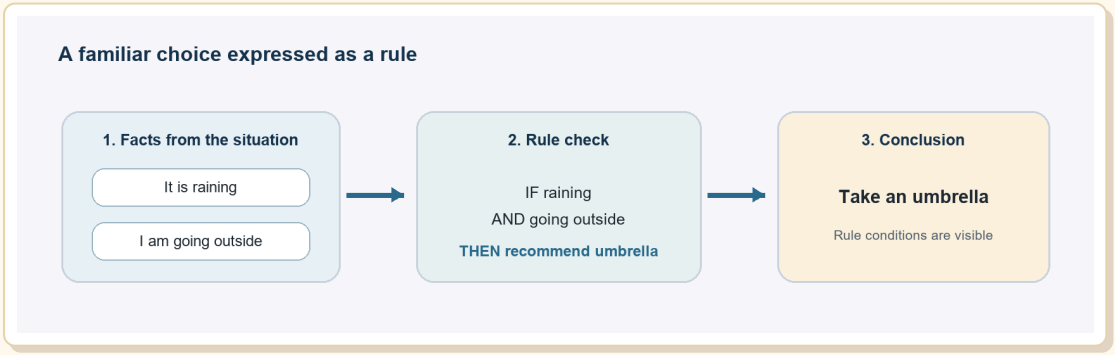


Figure 1.1. A familiar decision represented as facts, a rule, and a conclusion.

Software, AI and symbolic AI

Software is a set of programs and procedures, together with related information such as data and documentation, that tells a computer system how to perform tasks. One useful model is **input** → **processing** → **output**. A calculator, playlist, and messaging app are all software. Their usefulness does not by itself make them AI.

Artificial intelligence (AI) is a broad field. In this module, an AI system *infers* from inputs how to produce outputs such as predictions, content, recommendations, or decisions. **Machine-learning** systems commonly use patterns learned from data. **Symbolic AI** uses explicitly represented knowledge, such as facts and rules.

A **symbol** is a named representation of something in a problem. A **fact** states information about a case. A **rule** connects conditions with a conclusion or action. A rule-based system commonly has a **knowledge base** that stores facts and rules, and an **inference engine** that checks which rules match the available facts. Chapter 9 looks closer, and keeps the facts of the current case in a separate working memory.

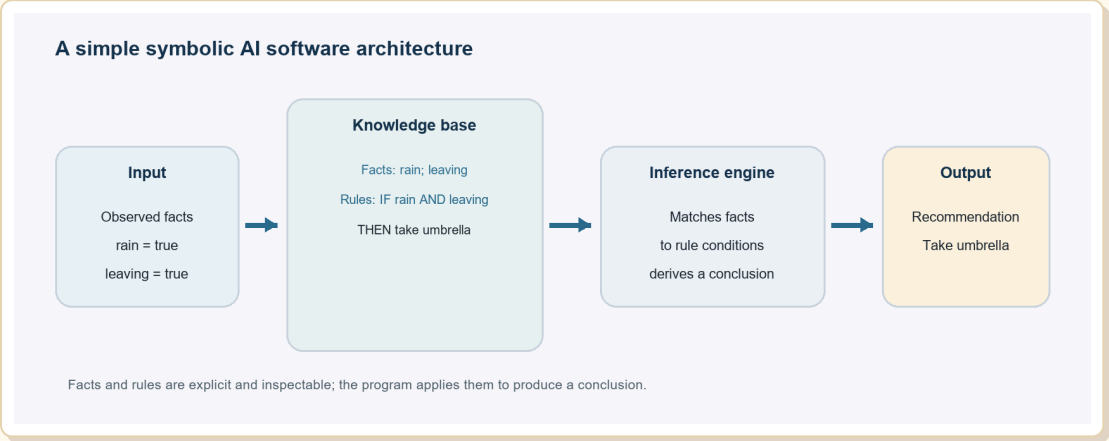
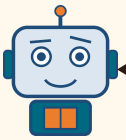


Figure 1.2. A simple architecture for symbolic AI software. Here `rain` and `leaving` are short symbols for "it is raining" and "you are going outside".



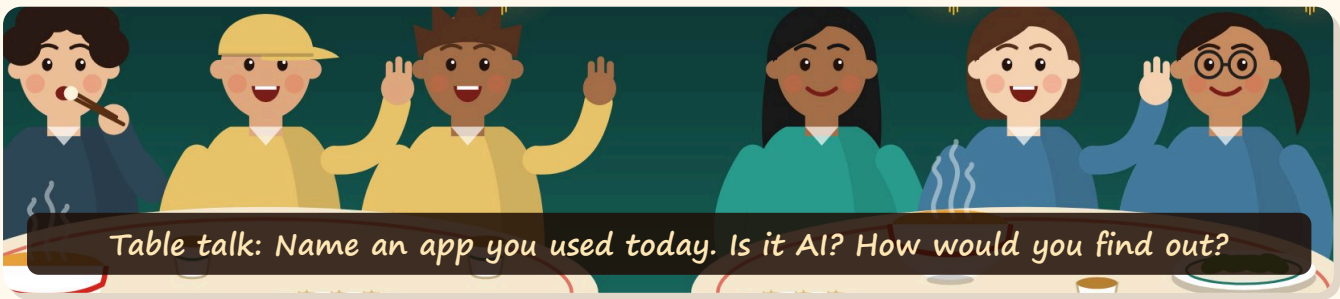
Pip says *"So the calculator is not AI, even though it is always right? I find this very unfair to the calculator."*

A first look at the library loan

The umbrella rule needed one step. The library loan desk needs two, because the conclusion of one rule feeds the next. Start with three facts: the member has a valid card, has no overdue books, and the requested book is available.

Step	Rule check	New information
1	**R1:** IF the member has a valid card AND has no overdue books THEN the member may borrow. Both conditions match.	Add the fact "member may borrow".
2	**R2:** IF the member may borrow AND the book is available THEN the loan is approved. Both conditions now match.	Add the conclusion "loan approved".

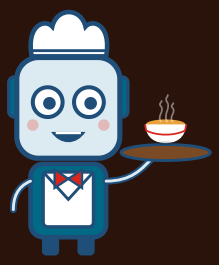
This is a small example of **forward chaining**. You will trace it properly in Chapter 9.



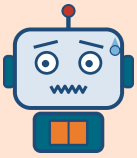
CHECK YOURSELF 1.1

Is it AI?

Goal	Decide whether a system uses AI by asking how it produces its output.
Preparation	Use the definition of AI above.
Task	1. A calculator adds 2 and 3. 2. An email filter trained on labelled messages classifies a new email. 3. A library program applies stored facts and IF–THEN rules to decide whether a member may borrow. 4. A music app plays songs in the order saved in a playlist.
Observe	For each example, write whether it uses AI under the definition, and give a one-sentence reason.
Reflect	Did you judge by how clever the system seems, or by how it produces its output?
Feedback	The calculator and playlist follow fixed procedures. The trained email filter infers a classification from patterns learned from data. The library system uses symbolic AI if it infers a conclusion by applying explicit facts and rules. Ask how the output is produced; do not judge only by whether the system seems clever.



Order up! Check your answers before the next dish.



Common mistake: "It always gets the right answer, so it must be AI."

Being useful or correct does not make a system AI. A calculator is always right and infers nothing. The question to ask is whether the system works out how to produce its output from what it receives.

Historical background

Software history includes the development of programmable machines, stored-program computers, higher-level programming languages, and software engineering. Symbolic AI grew as researchers explored whether computers could represent knowledge and reason with symbols.

Try it 1.2 — Put the milestones in order

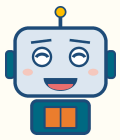
Put these selected milestones in order. A. MYCIN gives medical advice using production rules. B. The Manchester Baby runs a program stored in its memory. C. Ada Lovelace publishes an algorithm for Babbage's Analytical Engine. D. The Dartmouth proposal launches AI as a research field.

Check C (1843) → B (1948) → D (1955–1956) → A (1970s). Symbolic AI programs are software, so they require a machine that can store and change instructions and data.



Mission complete

Chapter 1: your bill



Takeaway: Software is the broader category. Symbolic AI is one way to build software that reasons from explicitly represented knowledge.

Pip's log: Pip now tells visitors: "The calculator is software. I count as AI only if I infer my answers, for example from facts and rules."

Before you move on, can you:

☐ Explain the difference between software, AI, machine learning, and symbolic AI.

☐ Represent a familiar decision as facts and a rule.

Practise next: Test Bank T01–T02, M01–M02, O01 (after Chapter 3). *Optional:* Rulebound, Stage 1.

Sources: IEEE (1990); OECD (2019); McCarthy et al. (1955); Davis et al. (1977).



Chapter Two

SOFTWARE OVERVIEW

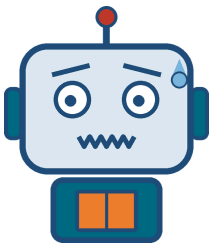
02

Software Overview



On today's menu: layered scallion pancake

The layers you never see hold up the ones you do. Software is built the same way.



Pip's problem

Pip tried to tidy the library computer by deleting "that useless printer driver nobody ever opens." Now the printer has stopped working, and Pip cannot see why.

Your mission

Help Pip find out what each kind of software does, and how the operating system keeps many programs running at once.

You will be able to

- Distinguish system software from application software by what each one does.
- Describe what an operating system manages, and trace programs taking turns on the processor.
- Tell procedural, object-oriented, functional and declarative languages apart.

You need: Chapter 1.

Watch: Video 01. While watching, ask: what would stop working if the operating system vanished?



Predict first

Pip deleted a driver, and the printer stopped. If Pip had deleted the web browser instead, would the printer still work? Write your answer and one reason.

What kind of work does the software do?

System software manages computer resources or provides services used by other programs. An **operating system (OS)** is a major example. **Application software** helps a user perform a task, such as browsing the web or checking out a library book. A **device driver** helps an operating system communicate with hardware.



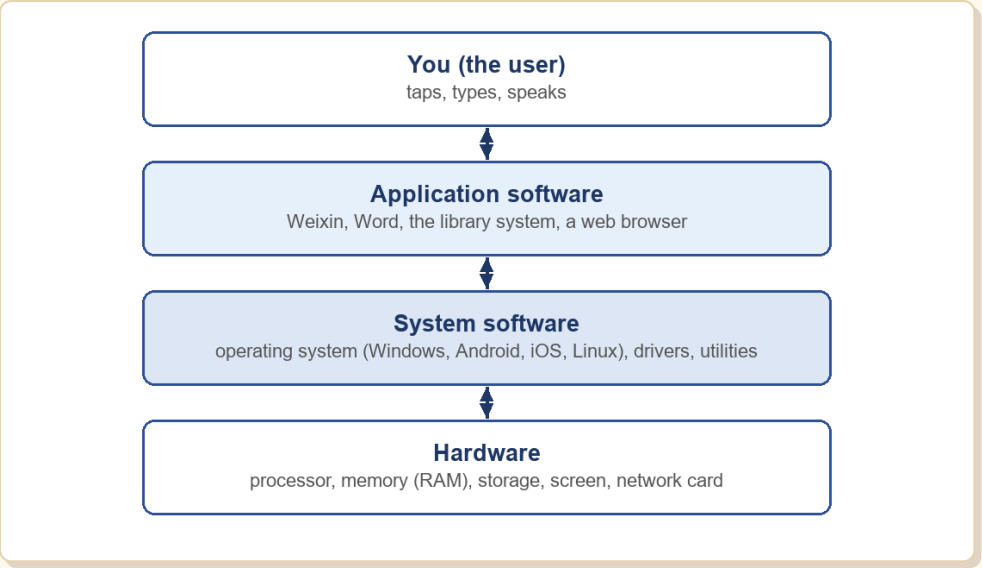


Figure 2.1. Layers of software. Each layer uses the services of the layer below it.

An operating system manages:

Responsibility	What it does	Example
Process management	Coordinates running programs and their use of the processor.	The OS shares processor time among active apps.
Memory management	Tracks and allocates working memory to programs, and helps keep them from interfering with each other.	A program requests memory while it runs.
System services and other resources	Provides common facilities used by applications, such as files and connected devices.	A messaging app can rely on OS notifications while in the background.

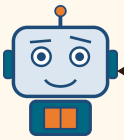
How programs take turns

A **process** is a running instance of a program. A simple processor can carry out only one process at a time, so the operating system gives each process a very short **time slice**, then lets the next one run. Because the switching is so fast, the programs seem to run together. Each process is **running** (using the processor now), **ready** (waiting for its turn), or **waiting** (for something else, such as data to arrive).

Try it 2.2 — Taking turns

Word is using the processor, Weixin is waiting for a video download, and a music player is ready for its next time slice. Name each state. Then three processes share the processor one slice at a time, in a circle (*round-robin* scheduling): Music needs 2 slices, Browser needs 3, and Weixin needs 1. Who runs in each slice, and who finishes first?

Check Word is running, Weixin is waiting, and the music player is ready. The slices go Music, Browser, Weixin, Music, Browser, Browser. Weixin finishes first, in slice 3: nobody waits too long, and the short job finishes early.



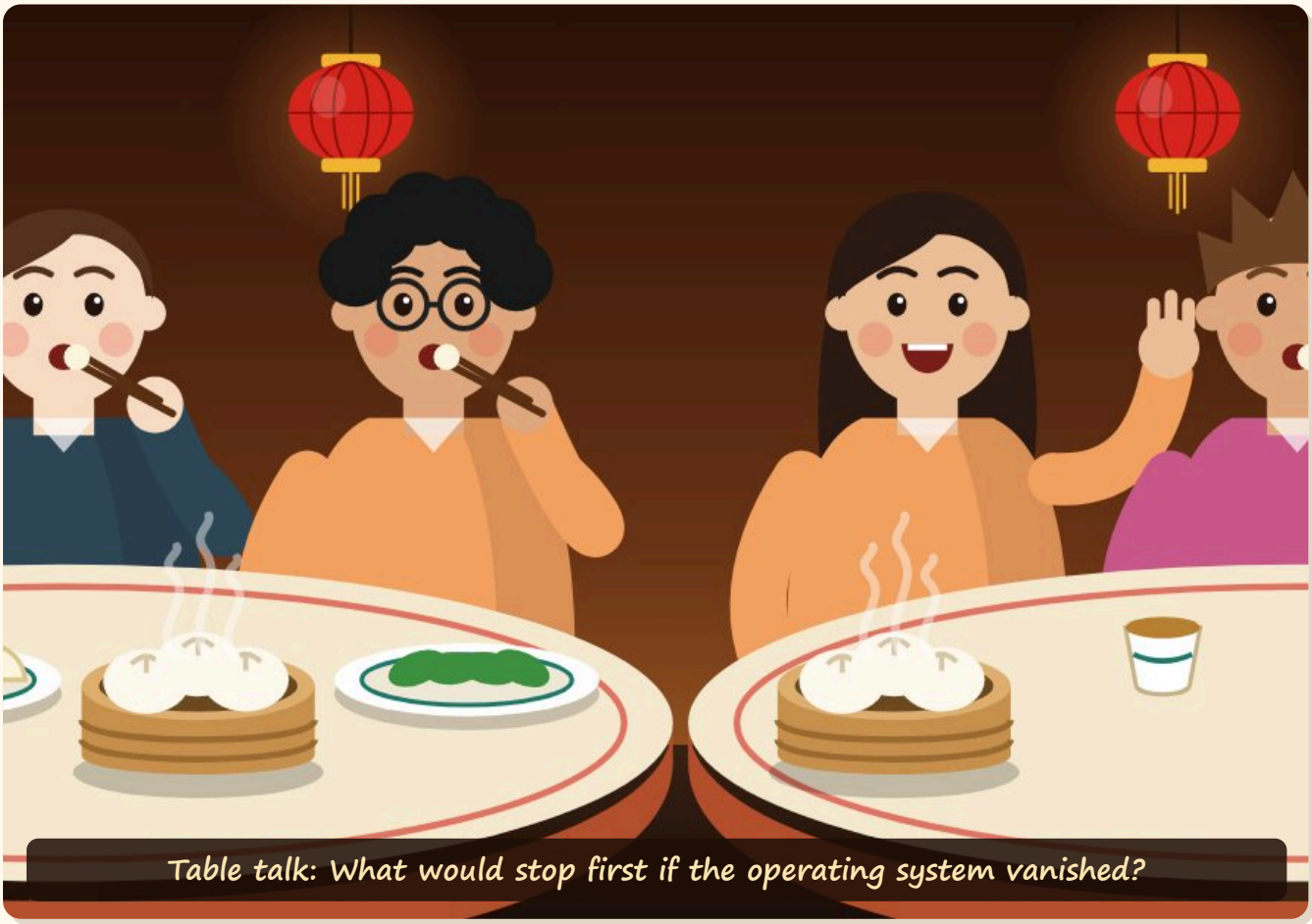
Pip says *"So when I said I was doing everything at once, I was just taking very fast turns. Please do not tell the visitors."*

Operating systems and languages change over time

Operating systems evolved as computers gained more users, programs, and hardware to manage: from **batch systems**, to **time-sharing**, to **personal computers** with windows and a mouse, to today's **networked and mobile systems**. **UNIX** is an example of an operating-system family, built in the time-sharing period. **Weixin** is an example of application software. They illustrate the difference between software that manages or supports a computer and software people use to complete tasks.

Programming languages also evolved, from instructions close to the hardware to higher-level languages that make many tasks easier to express. A high-level program must be translated before the processor can run it: a **compiler** translates the whole program first, and an **interpreter** reads and carries it out step by step. Languages differ in their features, design, and intended uses.

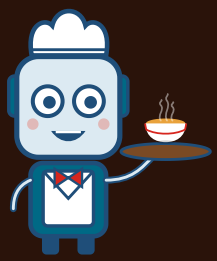
Category	Main idea	Examples from the module
Procedural or imperative	Describe the steps to carry out.	C, Pascal, FORTRAN
Object-oriented	Organize programs around objects and their behaviour.	Java, Python, C++
Functional	Express computation through functions.	Haskell, Lisp
Declarative or logic	State facts, constraints, or what result is wanted.	SQL, Prolog



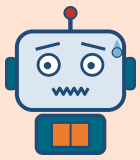
ACTIVITY 2.1

Who does what?

Goal	Distinguish system software from applications.
Preparation	Use the examples listed in the task, and the table of operating-system responsibilities above. Before you start, predict which of the five would stop other programs from working if it were deleted.
Task	1. Android 2. A library loan-check program 3. A printer driver 4. A web browser 5. A file manager supplied with an operating system
Observe	Label each one system software or application software, then explain its role in one sentence.
Reflect	Which item was hardest to classify? Would deleting it stop other programs from working? Use that to explain what Pip got wrong.
Feedback	Android and a printer driver are system software. The library program and browser are applications. A file manager supplied as part of an operating system is generally treated as system software, even though you can see it. Explain your classification by describing what the item does. **Back to your prediction:** other programs rely on the printer driver to reach the printer, so deleting it stopped printing. A browser is an application, so deleting it would not stop the printer. Pip judged the driver by whether anyone opens it, not by what it does.



Order up! Check your answers before the next dish.



Common mistake: "Nobody ever opens it, so it does nothing."

That was Pip's reasoning, and it cost the library its printer. System software mostly works out of sight, and other programs depend on it. Ask what job a program does, not whether you have ever seen it.

Try it 2.3 — Name the category

Classify each choice using the table above: (1) a C program that lists steps in order, (2) an SQL query asking for available book titles, and (3) Prolog facts and rules.

Check (1) Procedural, (2) declarative, and (3) logic/declarative. In each case, identify the way the solution is expressed, not whether the language is useful for only one task. A language can support more than one style.

Resource: Operating Systems: Crash Course Computer Science #18

What is it? Operating Systems: Crash Course Computer Science #18, from CrashCourse: youtube.com/watch?v=26QPDBe-NB8

Why use it? It shows why operating systems appeared, which makes process and memory management easier to picture.

Which part? The whole episode, about 13 minutes.

What to do Before watching, predict one problem an operating system solves. Afterwards, list two more.

Watch out It moves quickly and goes further than this chapter, for example into virtual memory. You do not need those details for the Test Bank.

If you cannot open it Team Video 01 and the table above cover what you need.



Mission complete

Chapter 2: your bill

Takeaway: Classification depends on a program's role in the system, not just its name. The operating system shares the processor and memory so that applications can run.

Pip's log: Pip has reinstalled the printer driver, and now asks "what job does this do?" before touching anything.

Before you move on, can you:

☐ Distinguish system software from application software by what each one does.

☐ Describe what an operating system manages, and trace programs taking turns on the processor.

☐ Tell procedural, object-oriented, functional and declarative languages apart.

Practise next: Test Bank T03–T04, M03–M04, M32, O01 (after Chapter 3). *Optional:* Rulebound, Stage 2.

Sources: chap3 slides 2 and 4; Forouzan (2018), Ch. 7 and 9; Brookshear and Brylow (2019), Ch. 3 and 6.



Chapter Three

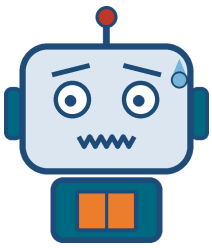
03 ALGORITHMS

Algorithms: How Programs Process Information and Make Decisions



On today's menu: hand-pulled noodles

A recipe is a clear, ordered procedure. Follow it exactly, and you get exactly what it says, even when that is not what you wanted.



Pip's problem

It is pouring outside. A visitor types Yes into Pip's umbrella reminder, and Pip replies "No umbrella needed." The visitor gets soaked, and Pip insists that it followed its instructions exactly. It did. So what went wrong?

Your mission

Learn to follow a program step by step, so that you can predict what it will do and find the error.

You will be able to

- Trace a short program and state its output for a given input.
- Check a list of steps against the five features of an algorithm.
- Trace a loop, a recursive function, linear search, binary search and a pass of bubble sort.

You need: Chapters 1 and 2. No coding experience is required.

Watch: Video 02. While watching, pause before each trace and predict the output.



Predict first

Pip's reminder checks whether the answer equals `yes`. What will it say if a visitor types `yes`? `YES`? `nope`? Write all three. The Common mistake box in this chapter has the answer.

Programs, data and algorithms

A **program** is a set of instructions a computer can execute. **Data** is information a program receives, stores, or produces. An **algorithm** is a clear, ordered procedure for solving a problem; it does not depend on a particular programming language. It may use:

- **Sequence:** instructions performed in order.
- **Selection:** a choice between paths, such as IF–ELSE.
- **Iteration:** repeating instructions with a loop.
- **Search and sorting:** finding or ordering items in a collection.

Worked example: the weather reminder

```
IF it is raining
    display "Take an umbrella"
ELSE
    display "No umbrella needed"
```

The input is whether it is raining. The processing is the IF–ELSE decision. The output is one of the two messages. Exactly one branch runs. Here is the same plan as a flowchart, and as a short Python program:

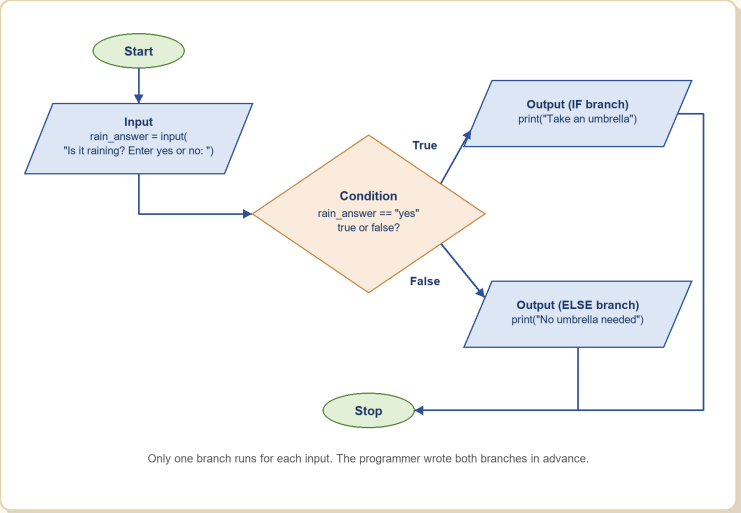
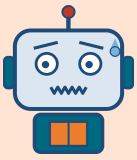


Figure 3.1. The weather reminder as a flowchart.

```
rain_answer = input("Is it raining? Enter yes or no: ")
if rain_answer == "yes":
    print("Take an umbrella")
else:
    print("No umbrella needed")
```

`input()` receives text and stores it in the variable `rain_answer`. `==` asks whether two values are equal, and `print()` displays the result. **Tracing** means following the instructions with a specific input and recording what happens: with `yes`, the condition is true and "Take an umbrella" appears; with `no`, the condition is false and "No umbrella needed" appears.



Common mistake: "Yes and yes mean the same thing."

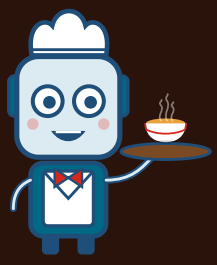
They do to you, but not to this program. It compares the answer with the exact lowercase text yes, so only `yes` gives "Take an umbrella". `Yes`, `YES` and `nope` all make the condition false, and the ELSE message appears. The program follows its written instructions. A more complete program could check the input and ask again when it is not an accepted answer.



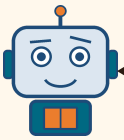
ACTIVITY 3.1

Trace and debug

Goal	Practise following an algorithm and finding a decision error.
Preparation	Read the weather reminder above. Paper is enough.
Task	1. Record the output when the answer is yes and when it is no. 2. Predict what the Python version displays for Yes with a capital Y. 3. Inspect this faulty version: IF it is raining display “No umbrella needed” ELSE display “Take an umbrella”. 4. Identify the error and correct it. 5. Draw a flowchart with one decision and two branches.
Observe	Compare the output before and after your correction.
Reflect	Which condition selects the output? Why does a computer not “know” that a message is the wrong way round?
Feedback	The Python version displays “No umbrella needed” for Yes, because the comparison is exact. The faulty version displays the opposite message in both cases: swap the messages. Tracing each input through the flowchart or program reveals the error. The IF condition selects the output. A computer does not know that a message is the wrong way round, because it follows the comparison and nothing else. The meaning of the message is in our heads, not in the program.



Order up! Check your answers before the next dish.



Pip says *"I traced it. It did exactly what it said. That turns out to be the whole problem."*

What makes a good algorithm?

A good algorithm is clear enough to trace and guaranteed to finish for the inputs it accepts. It can be described in ordinary language, pseudocode, a flowchart, or program code; the plan is separate from a particular programming language.

Feature	Beginner-friendly test
Input	What information does the procedure receive?
Output	What result must it produce?
Definiteness	Could two people interpret any step differently?
Finiteness	Is there a condition that makes the steps stop?
Effectiveness	Can each instruction actually be carried out?

Iteration and recursion

Approach	How repetition happens	What makes it stop
Iteration	A loop repeats a set number of times, or while a condition remains true.	The count runs out, or the condition becomes false.
Recursion	A function calls itself on a smaller version of the problem.	A base case gives a result without another recursive call.

```
days_late = [2, 0, 5]
total = 0
for days in days_late:
    total = total + days * 1
print("Total fine:", total, "yuan")
```

```
def countdown(n):
    if n == 0:              # base case
        print('Lift off!')
    else:                   # smaller recursive case
        print(n)
        countdown(n - 1)
```



Code	What it does
`days_late = [2, 0, 5]`	Square brackets make a **list**. This one holds three numbers, and the name `days_late` refers to all of them.
`for days in days_late:`	Takes each number in the list in turn and calls it `days`. The indented line below runs once for each number.
`def countdown(n):`	`def` defines a **function** named `countdown`. `n` holds the number it is given. Nothing runs until the function is called.
`# base case`	Text after `#` is a **comment**: a note for people, which Python ignores.
`countdown(n - 1)`	The function calls itself with a number one smaller.

Try it 3.2 — Trace a loop and a countdown

(a) A member returns three books that are 2, 0 and 5 days late, and the fine is 1 yuan per day. Trace the loop above: what is `total` after each pass, and what is printed? (b) Trace `countdown(2)`. What is printed, and what would go wrong if the recursive call used `countdown(n)` instead?

Check (a) `total` is 2, then 2, then 7, and it prints Total fine: 7 yuan. (b) It prints 2, then 1, then Lift off!. With `countdown(n)`, the input never gets smaller, so the base case is not reached and the program continues calling itself until it fails.

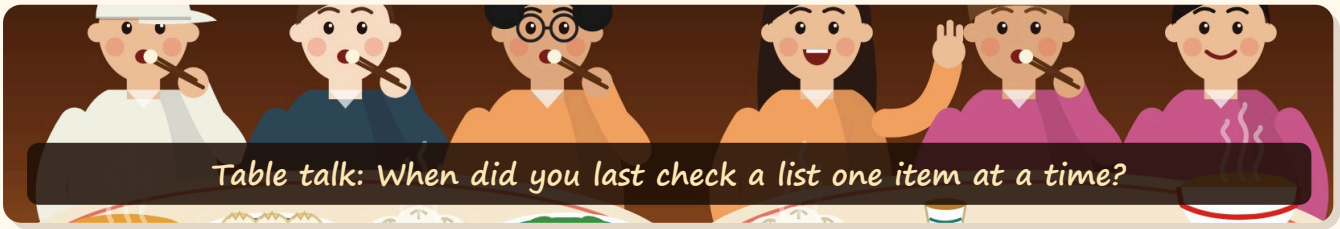
Searching and sorting

Choose a search based on how the list is organized; sorting changes the order so later searches may be easier.

Method	How it works	Requirement or limitation
Linear search	Checks items one by one until it finds the target or reaches the end.	Works even when the list is unsorted.
Binary search	Checks a middle item and discards the half that cannot contain the target.	The list must already be sorted.
Bubble sort	Compares adjacent values and swaps an out-of-order pair; repeated passes move values into order.	A pass alone may not finish the sort.

Worked example: bubble sort. Sort 4, 1, 3 into ascending order, comparing adjacent values from left to right. Compare 4 and 1: swap, giving [1, 4, 3]. Compare 4 and 3: swap, giving [1, 3, 4]. After this pass, the list is sorted.

That list was sorted after one pass, but most lists take several. **The stop rule:** keep making passes until a whole pass makes no swap, so even that short list gets one more pass to confirm it. Here is `5, 3, 8, 1`:



Pass	Comparisons and swaps	List after the pass
1	5 > 3 swap; 5 < 8 keep; 8 > 1 swap	3, 5, 1, 8
2	3 < 5 keep; 5 > 1 swap; 5 < 8 keep	3, 1, 5, 8
3	3 > 1 swap; the rest stay	1, 3, 5, 8
4	No swaps, so stop	1, 3, 5, 8: sorted

After each pass, the largest remaining number has "bubbled" to its place at the end. Bubble sort is easy to understand but slow on long lists.

Mini-challenge 3.3 — Find and order information

A library catalogue contains these titles in an unsorted list: *The Little Prince*, *A Brief History of Time*, *Python for Beginners*. First, describe how you would check each title one by one to find *Python for Beginners*. Then explain how sorting the titles alphabetically could help someone browse them.

Check Checking one by one is linear search; here it takes three checks. A search method and a sorting method solve related but different problems: one finds an item, and the other changes the order of items.

Try it 3.4 — Search and sort

(a) After one left-to-right bubble-sort pass on [3, 1, 2], what is the list? (b) Why must binary search not be used on [8, 2, 5] before sorting? (c) How many checks does binary search need to find 70 in the sorted list 10, 20, 30, 40, 50, 60, 70, 80, 90? When two items are in the middle, take the left one.

Check (a) [1, 2, 3]. (b) Binary search relies on order to know which half can be discarded, so it cannot safely decide where the target is in an unsorted list. (c) Two: the middle item is 50, so keep the right half; the next middle item is 70.

Resource: Intro to Algorithms: Crash Course Computer Science #13

What is it? Intro to Algorithms: Crash Course Computer Science #13, from CrashCourse: youtube.com/watch?v=rL8X2mlNHPM

Why use it? It shows why the choice of algorithm matters, using sorting and route-finding.

Which part? The whole episode.

What to do While watching, write down one reason why one algorithm can be better than another for the same job.

Watch out It goes further than this chapter, into other sorting methods and a route-finding algorithm. You do not need to memorize these.

If you cannot open it Team Video 02 and the worked examples above cover what you need.

Mission complete

Chapter 3: your bill

Takeaway: Tracing means following instructions with specific inputs and recording what happens.

Pip's log: Pip's reminder now asks again whenever it receives anything other than yes or no. The visitor forgives Pip, eventually.

Before you move on, can you:

☐ Trace a short program and state its output for a given input.

☐ Check a list of steps against the five features of an algorithm.

☐ Trace a loop, a recursive function, linear search, binary search and a pass of bubble sort.

Practise next: Test Bank T05–T06, M05–M06, O01. *Optional:* Rulebound, Stage 3.

Sources: chap3 slide 3; Brookshear and Brylow (2019); Knuth (1997).



Chapter Four

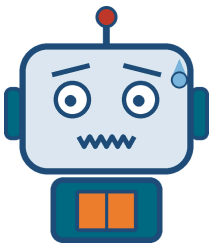
DATA STRUCTURES 04

Data Structures



On today's menu: a tower of steamer baskets

The last basket stacked is the first one lifted off. The line at the door works the other way round: first come, first served.



Pip's problem

Pip keeps members waiting at the loan desk in a stack. The last person to arrive is served first, and the people who have waited longest are getting very annoyed.

Your mission

Learn to choose a structure that fits the job, and see how a queue lets a program explore a map.

You will be able to

- Describe the main data structures, with a library example for each.
- Trace a queue, and trace breadth-first search on a map.

You need: Chapter 3.

Watch: Video 02. While watching, ask: why does this job need this structure?



Predict first

Mei, Tom and Lin arrive in that order, and Pip puts them in a stack. Who is served first? Who waits longest? Write your answer before you read on.

A **data structure** organizes information so a program can store, find, and update it. The structure should fit the task.

Structure	How it organizes data	Example use
Array	Ordered items accessed by position	A list of daily temperatures
Record	Related fields describing one item	A member's name and card status
Linked list	A chain: each item points to the next	A collection that changes size often
Stack	Last in, first out (LIFO)	Undo history or a pile of books
Queue	First in, first out (FIFO)	Members waiting at the loan desk
Tree	Parent-and-child relationships	The library's subject catalogue
Graph	Nodes joined in any pattern, possibly with cycles	A map of rooms and corridors



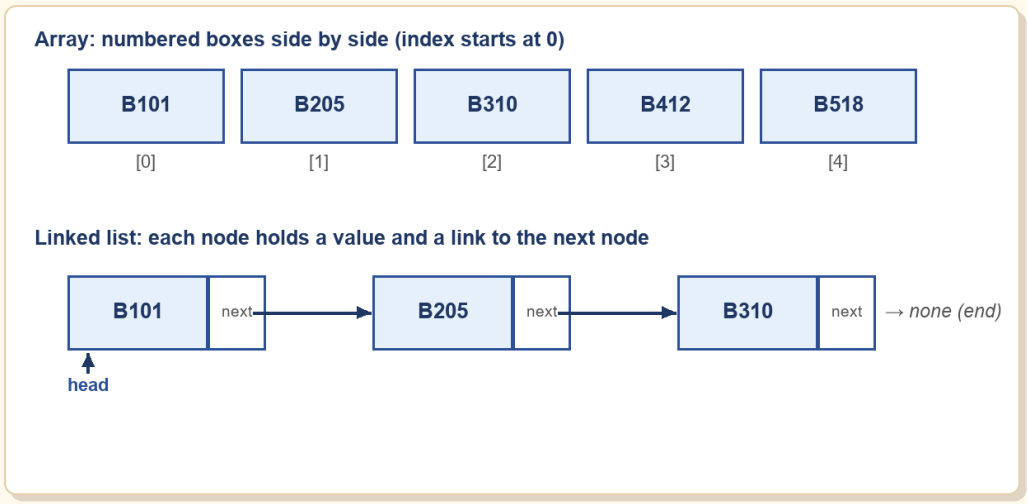


Figure 4.1. Top: an array, with numbered boxes side by side. Bottom: a linked list, where each item points to the next.

Why a linked list is good at inserting. To insert into an array, every later item must shift one box along. In a linked list, no item moves: only links change. First link the new item to the one after it, then link the one before it to the new item.

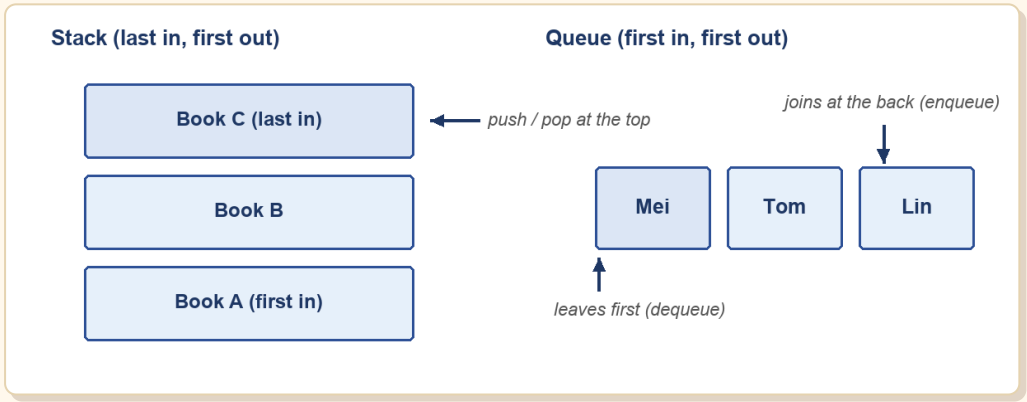


Figure 4.2. A stack adds and removes at the top. A queue adds at the back and removes from the front.

Check your prediction. In Pip's stack, Lin, the last to arrive, is served first, and Mei waits longest. In a queue, Mei would be served first.

A **binary tree** is a tree where each node has at most two children. A **graph** contains nodes and the connections between them, and it can contain cycles.

Try it 4.2 — Queue trace

Start with an empty queue. Do: enqueue Mei, enqueue Tom, dequeue, enqueue Lin, enqueue Ana, dequeue. Who is in the queue at the end, front first?

Check Lin, then Ana. Mei leaves first, then Tom: first in, first out.



Try it 4.3 — Choose the structure

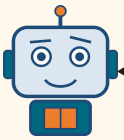
Which structure fits best, and why? (1) The order in which customers called a help line, to answer them fairly. (2) The browser's Back button. (3) The 30 seats in a classroom, numbered 0–29. (4) Which train stations are connected to which.

Check (1) Queue: first come, first served. (2) Stack: the last page you opened is the first one Back returns to. (3) Array: fixed size, numbered positions. (4) Graph: connections, possibly with cycles.

Try it 4.4 — Insert into the chain

A linked list reads B101 → B205 → B310. Insert B150 between B101 and B205. Which two links change? Do any other items move? What goes wrong if you change B101's link first?

Check B150 is linked to B205, then B101 is linked to B150, giving B101 → B150 → B205 → B310. No other item moves. If you change B101's link first, nothing remembers where B205 is, and the rest of the chain is lost. That is why the new item must be linked first.



Pip says *"I tried a Back button that worked like a queue. It took me to the first page I ever opened. I do not recommend it."*

Breadth-first search

Breadth-first search (BFS) visits a graph layer by layer: first the starting node, then its neighbours, then the neighbours' neighbours. It uses a queue to keep track of what to visit next. BFS always finds a route with the fewest edges, but it ignores any costs on those edges (Chapter 10).

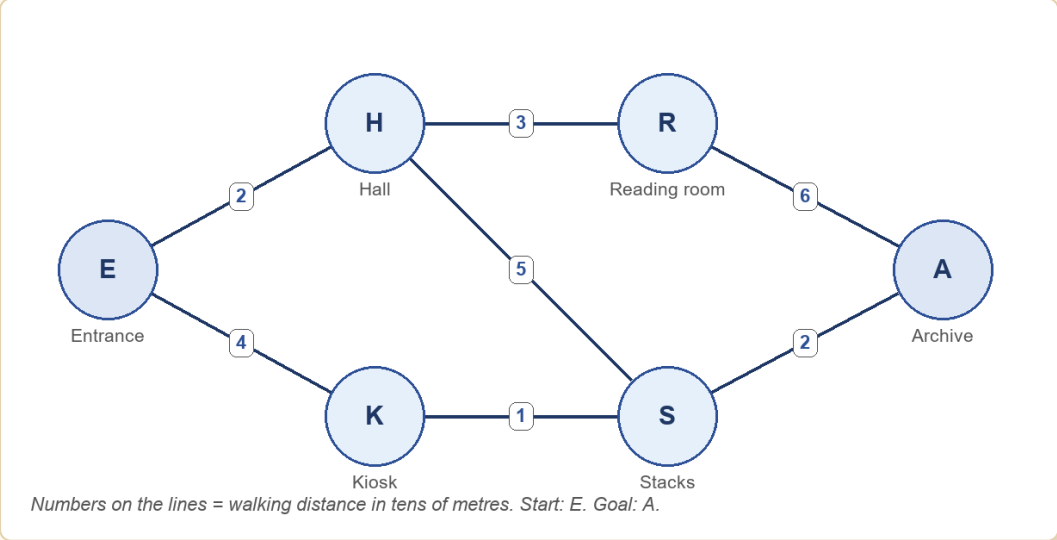
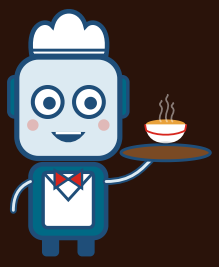


Figure 4.3. The library map. Numbers show walking distance, which Chapter 10 uses. The room called Stacks is where books are shelved; it has nothing to do with the data structure.

ACTIVITY 4.1

Choose a structure and trace BFS

Goal	Connect a data structure to a task and follow a graph search.
Preparation	Use the library map in Figure 4.3. Add neighbours in alphabetical order, and never add a room that has already been visited or is already waiting.
Task	1. Decide whether a queue or stack better represents a line at the loan desk. 2. Predict: after the Entrance (E), will BFS visit the Reading room (R) or the Kiosk (K) first? 3. Start at E and trace BFS through the map. 4. Write down the queue after each room is visited. 5. Go further: draw your own five-room map, and trace BFS on it.
Observe	Record the order in which rooms are visited.
Reflect	How does the queue make BFS visit nearby rooms before more distant ones?
Hint	Keep two lists on paper: rooms visited, and the queue. Each time you visit a room, add its new neighbours to the back of the queue.
Feedback	A queue fits a service line because the first person to join is served first. In BFS, newly found neighbours join the back of the queue, so the search advances outward layer by layer. The queue goes: H, K → K, R, S → R, S → S, A → A → empty. The visiting order is **E, H, K, R, S, A**, so K comes before R. Rooms one step away (H, K) come before rooms two steps away (R, S).



Order up! Check your answers before the next dish.

Common mistake: "A list is a list. Any structure will do."

That is how Pip ended up serving the last person first. The same three people come out in opposite orders from a stack and a queue. Ask what the job needs, then choose the structure that does it.

Resource: Data Structures: Crash Course Computer Science #14

What is it? Data Structures: Crash Course Computer Science #14, from CrashCourse: youtube.com/watch?v=DuDz6B4cqVc

Why use it? It shows how arrays, linked lists, stacks, queues, trees and graphs are organized, with animations.

Which part? The whole episode, about 10 minutes.

What to do Pause after each structure and name one library job it could do.

Watch out It shows some code that you are not expected to read in detail.

If you cannot open it Team Video 02 and the table above cover what you need.

Mission complete

Chapter 4: your bill

Takeaway: A suitable data structure makes it easier for a program to organize and process information.

Pip's log: Pip has swapped its stack for a queue. The members are much happier.

Before you move on, can you:

☐ Describe the main data structures, with a library example for each.

☐ Trace a queue, and trace breadth-first search on a map.

Practise next: Test Bank T07–T08, M07–M08, M37, O02 (after Chapter 10). *Optional:* Rulebound, Stage 4.

Sources: chap3 slides 3 and 4; Forouzan (2018), Ch. 11 and 12; Brookshear and Brylow (2019), Ch. 8.

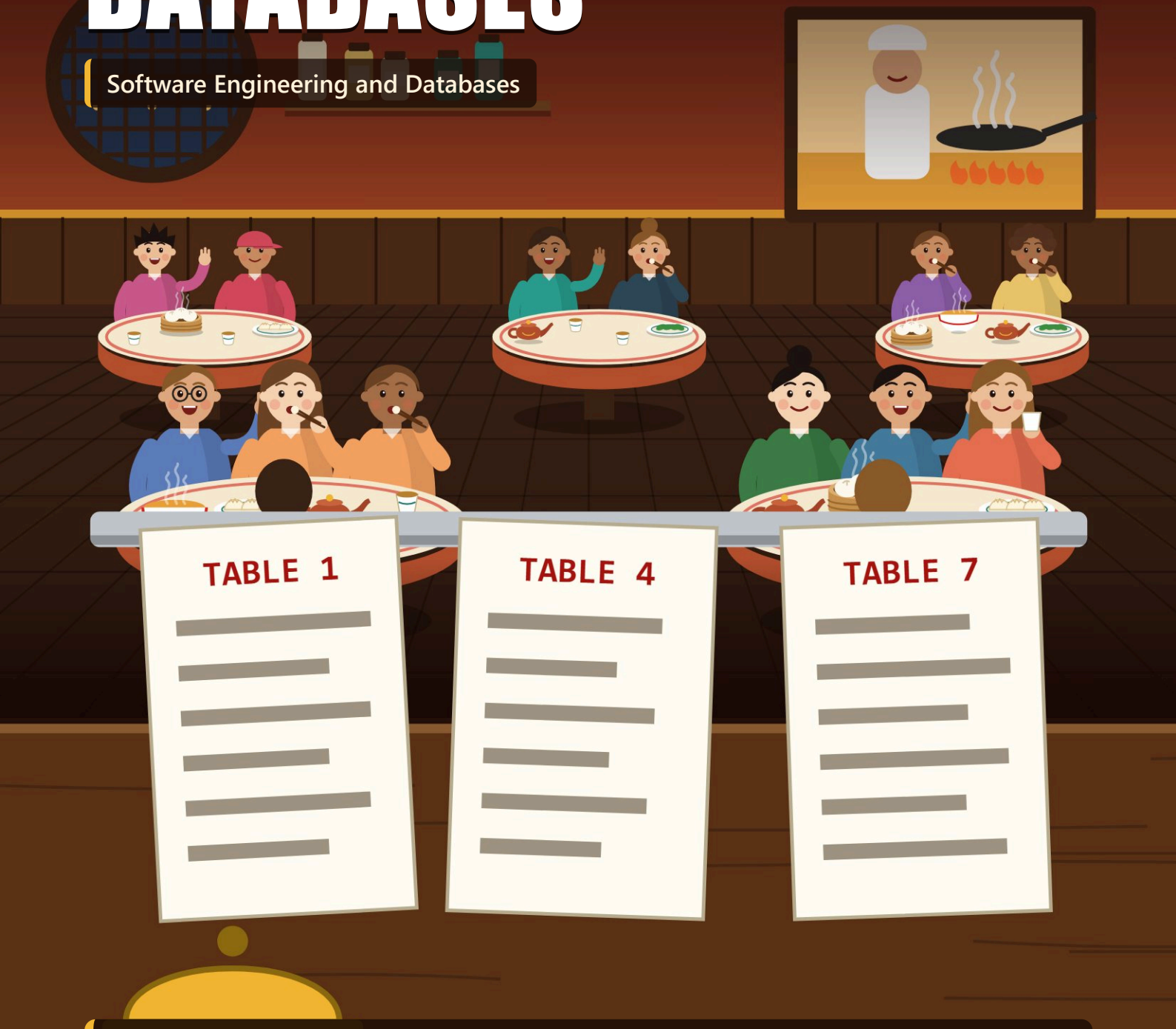


Chapter Five

05

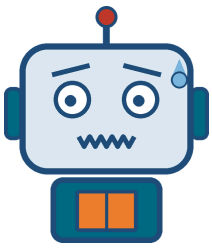
SOFTWARE ENGINEERING AND DATABASES

Software Engineering and Databases



On today's menu: kitchen tickets

One new line on the menu can change the ingredients, the method and the taste test. Software policy changes work like that too.



Pip's problem

The library adds one short policy: a member may have at most five books on loan. A member with five books asks for one more, and Pip's program happily issues a sixth. Why can one sentence require changes to data, program logic, and tests?

Your mission

Follow that change through the life of a real piece of software, from the first question to the final test.

You will be able to

- Describe the five stages of the software life cycle, and choose a process model.
- Read the loan decision as a program and write test cases for it.
- Explain tables and keys, and read simple SQL.

You need: Chapters 3 and 4. Basic IF–ELSE tracing.

Watch: Video 03. While watching, ask: what must be decided before a test is run?



Predict first

The library adds one sentence: at most five books on loan. List everything you think must change. Data? The program? The tests? Keep your list, because you will compare it later in this chapter.

The software life cycle

Software engineering is the organized work of developing and maintaining software. The life cycle in this module uses five plain-language stages:

1. Understand the problem and its users.
2. Plan the information and steps needed.
3. Write the program.
4. Test it.
5. Use and maintain it.

Textbooks may divide these activities into different numbers of phases or use different names. The important points are to understand requirements before building, test the program, and maintain it after it is used.

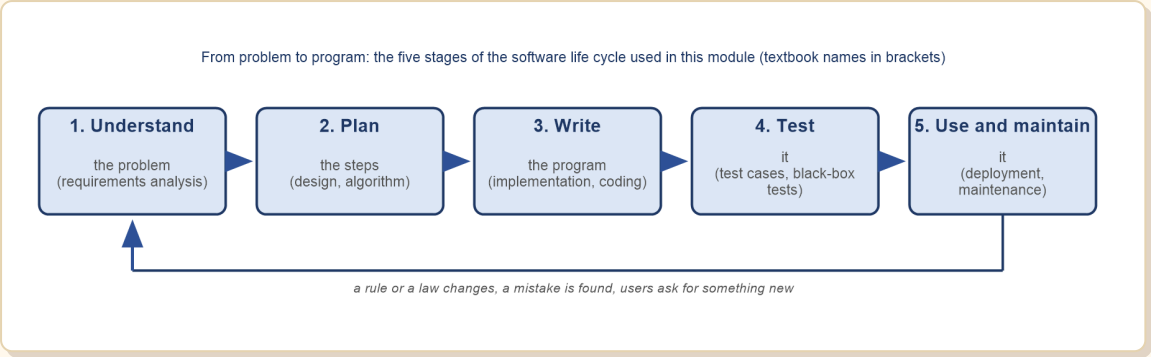


Figure 5.1. The five stages of the software life cycle.

Working at full size: models, modules and quality

The development model, module boundaries, and test plan should fit the problem and its risks.

Model	Useful when	Watch for
Waterfall	Requirements are clear and fixed, and stages need formal sign-off.	A change found late can be expensive.
Incremental	The system can be delivered in planned working parts.	Parts need an overall design so they fit together.
Agile	Requirements may change and users can give feedback in short cycles.	Do not skip design, testing, or useful documentation.

A module should have high **cohesion**: its tasks belong together. Modules should have low **coupling**: one module should depend as little as practical on another's internal details. A clear **interface** lets them exchange the information they need. **Quality assurance** checks how the work is planned and carried out as well as whether the product meets its requirements.

Testing level	Question
Unit	Does one function or module work on its own?
Integration	Do connected modules exchange information correctly?
System	Does the complete system meet its requirements?
Acceptance	Can users confirm that it meets their needs?

Black-box testing checks inputs and outputs against expectations. **White-box testing** also examines internal paths or conditions. **Regression testing** repeats earlier tests after a change to check that existing behaviour still works.

Case study: the library management system

A library system stores information about members, books, and loans. A **database** is an organized store of related data. A **database management system** stores, queries, updates, and manages it. In a **relational database**, data is stored in tables. Each row is a record and each column is a field.

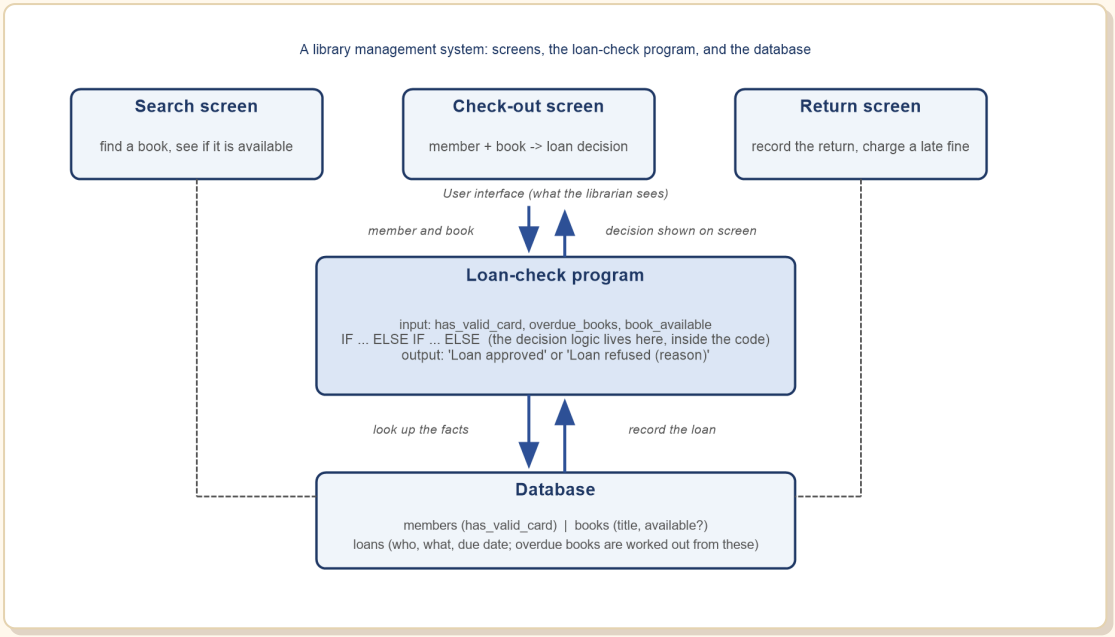


Figure 5.2. The library management system.

Members

member_id	name	has_valid_card
M017	Mei	true
M022	Tom	true
M031	Lin	false

Books

book_id	title	available
B205	*The Little Prince*	false
B310	*Python for Beginners*	false
B412	*A Brief History of Time*	true

Loans

loan_id	member_id	book_id	due_date	returned
L1	M022	B205	2026-10-20	false
L2	M017	B310	2026-11-15	false

A **primary key** identifies each row uniquely. For example, `member_id` identifies a member. A **foreign key** refers to a primary key in another table. In the Loans table, `member_id` and `book_id` connect a loan to its member and book.



Database model	How records relate
Hierarchical	A tree structure; each record has one parent.
Network	Records can have multiple links or parents.
Relational	Data is stored in tables connected through key values.
Newer models	NoSQL systems support other structures for different data and access needs.

A **query** states what information is wanted and the condition it must satisfy; the database searches for matching rows. SQL examples from the library case:

```
-- Find available titles
SELECT title FROM Books WHERE available = TRUE;

-- Count Tom's overdue books on 3 November 2026 (result: 1)
SELECT COUNT(*) FROM Loans
WHERE member_id = 'M022' AND returned = FALSE AND due_date < '2026-11-03';

-- Link members to their unreturned loans
SELECT Members.name FROM Members
JOIN Loans ON Members.member_id = Loans.member_id
WHERE Loans.returned = FALSE;
```

`COUNT(*)` gives the number of matching rows. `JOIN … ON` pairs each loan with its member through the foreign key, so the last query returns Tom and Mei. INSERT adds a row. UPDATE changes existing rows, and WHERE limits which rows are changed.

The loan decision as a program

The loan decision can be written as ordinary program logic:

```
IF the card is not valid
    refuse the loan because the card is invalid
ELSE IF the member has overdue books
    refuse the loan because books are overdue
ELSE IF the requested copy is not available
    refuse the loan because the copy is unavailable
ELSE
    approve the loan and record it
```

The policy is embedded in the program's conditions and their order. The chain stops at the first true condition: Tom hears only about his overdue book, even if the book is also on loan. If the library changes its policy—for example, setting a limit of five books on loan—the program and its tests must change too.

Testing the program

A **test case** includes an input and an expected result. Write the expected result before running the program, then compare it with the actual result. A useful set of tests covers each important branch:

Test	Valid card	Overdue books	Book available	Expected result
T1 Mei	true	0	true	Approve and record the loan
T2 Tom	true	1	true	Refuse: books are overdue
T3 Lin	false	0	true	Refuse: the card is invalid
T4 Mei, copy on loan	true	0	false	Refuse: the copy is unavailable

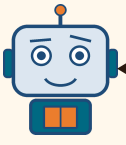
Follow the change: at most five books

Now compare with the list you wrote at the start of this chapter. The one-sentence policy runs through the whole life cycle:

- **Understand:** write down the new requirement: "a loan is refused when the member already has five books on loan."
- **Plan:** the program needs one more input, the number of books on loan, and one more condition.
- **Write:** read the new value, and insert a new condition in the chain (below).
- **Test:** run all four old cases again, each with fewer than five books on loan, to make sure nothing else broke. Add two new ones: a member with four books (approved) and a member with five (refused). They sit on either side of the limit, which is where mistakes usually hide.
- **Use:** the new version replaces the old one at the loan desk.

```
IF the card is not valid
    refuse the loan because the card is invalid
ELSE IF the member has overdue books
    refuse the loan because books are overdue
ELSE IF the member has five or more books on loan      // new condition
    refuse the loan because the limit is reached
ELSE IF the requested copy is not available
    refuse the loan because the copy is unavailable
ELSE
    approve the loan and record it
```

Three things are worth noticing. One sentence of policy needed a programmer, a code change and a full re-test. The order of the conditions is a design decision, because it changes which reason a member hears when two apply. And nobody can see the policy without reading the code, which is the problem that Chapter 9's rules set out to solve.



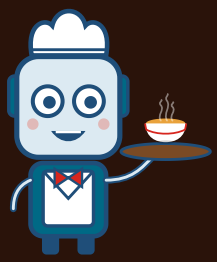
Pip says *"I once wrote the expected result after running the test. It passed every time. I have been asked not to do that."*



ACTIVITY 5.1

Design and test a small program

Goal	Connect a user need to requirements, program logic, database information, and tests.
Preparation	Use the library example and the tables above. Paper is enough.
Task	1. Write a requirement for deciding whether a member can borrow a book. 2. List the required inputs, processing steps, and output. 3. Add one test for each important decision branch. 4. Add a test for any new policy you introduce. 5. Write a simple database query to find available books.
Observe	Write each expected output before tracing or running the program.
Reflect	Which branches did your tests exercise? What could happen if one branch were never tested?
Hint	Start from the three questions the loan desk asks: Is the card valid? Are any books overdue? Is the book available? Each question is a branch, so each needs at least one test.
Feedback	A sample requirement: "The system must tell the librarian whether a member may borrow a chosen book, using the member's card, their overdue books and the book's availability, and must record the loan if it is approved." A useful test set reaches each important output branch. A query such as `SELECT title FROM Books WHERE available = TRUE;` finds titles whose availability field is true. An untested branch can hide a mistake until a real member meets it. When a policy changes, update the requirements, program, database needs, and tests together.



Order up! Check your answers before the next dish.

Try it 5.2 — Test the five-book policy

A loan policy allows at most five books. Name four useful tests, including boundary cases, and state what each should check.

Check Test a valid member with four books (a loan may pass if other conditions hold), a member with five requesting another (the sixth loan must not be approved), an invalid card, and a valid member with an overdue book. Record expected outputs before running the tests, and repeat the old tests after the change.

Try it 5.3 — A risky update

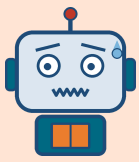
Why is `UPDATE Books SET available = TRUE;` risky if the task is to return only book B205?

Check It has no WHERE condition, so it marks every book as available. Add `WHERE book_id = 'B205'` to target only that row.

Try it 5.4 — Choose a process model

For each project, choose waterfall, incremental or agile, and give one reason. (1) A new phone app for members: the librarians are not sure what they want, and they are happy to try it every two weeks. (2) A program that controls the library's fire doors: a safety law fixes exactly what it must do. (3) A new library system: the library wants the loan check working first, then returns and fines, then reports. (4) The Fines module also prints library cards and sends opening hours. Is that a cohesion or a coupling problem?

Check (1) Agile: requirements are likely to change and users can give feedback. (2) Waterfall: the requirements are clear and fixed, and the system is safety-critical. (3) Incremental: it is delivered in working pieces, though the overall design still needs care. Other answers can be fair if the reason is good. (4) Low cohesion: the jobs inside the module do not belong together.



Common mistake: "It ran without an error, so it works."

Pip's program ran perfectly while issuing a sixth book. A program that runs is not necessarily correct: only comparing actual results with expected results shows whether it does what the requirement says.



Mission complete

Chapter 5: your bill

Takeaway: A program that runs is not necessarily correct. Testing compares actual results with expected results.

Pip's log: Pip's program now checks the loan count, and two new tests sit on either side of the limit: four books approved, five refused.

Before you move on, can you:

☐ Describe the five stages of the software life cycle, and choose a process model.

☐ Read the loan decision as a program and write test cases for it.

☐ Explain tables and keys, and read simple SQL.

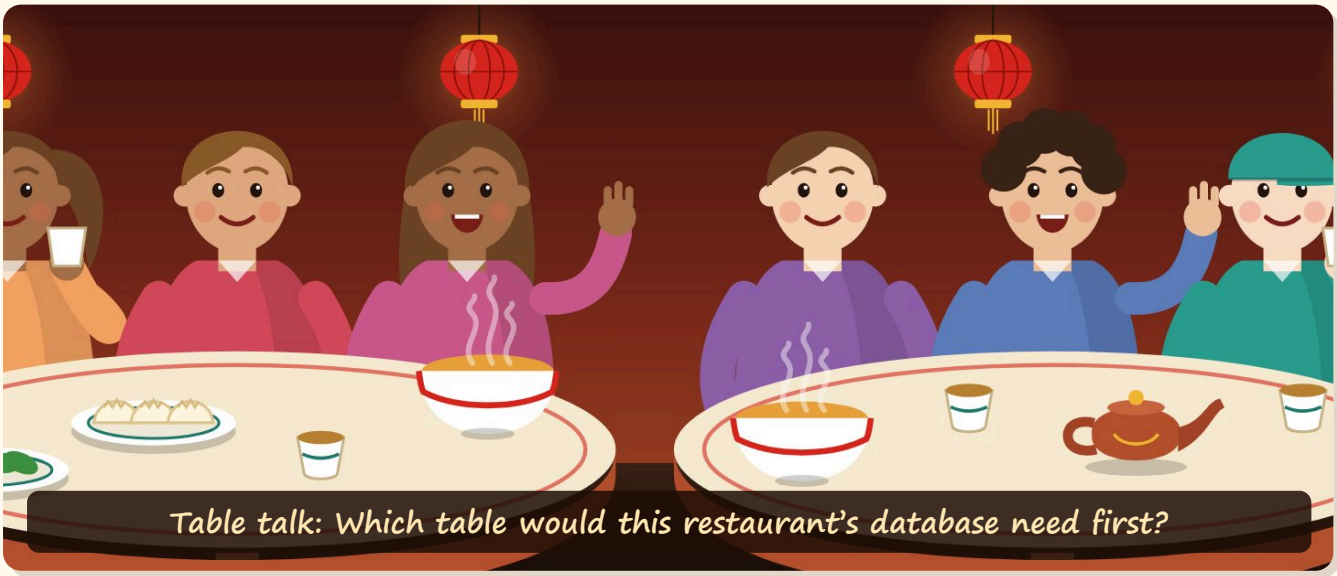
Practise next: Test Bank T09–T10, M09–M10, M33–M34, O03 (after Chapter 12). *Optional:* Rulebound, Stage 5.

Sources: Forouzan (2018), Ch. 8, 10 and 14; Brookshear and Brylow (2019), Ch. 7; Sommerville (2016), Ch. 1 and 2.

Checkpoint A: one loan, five chapters

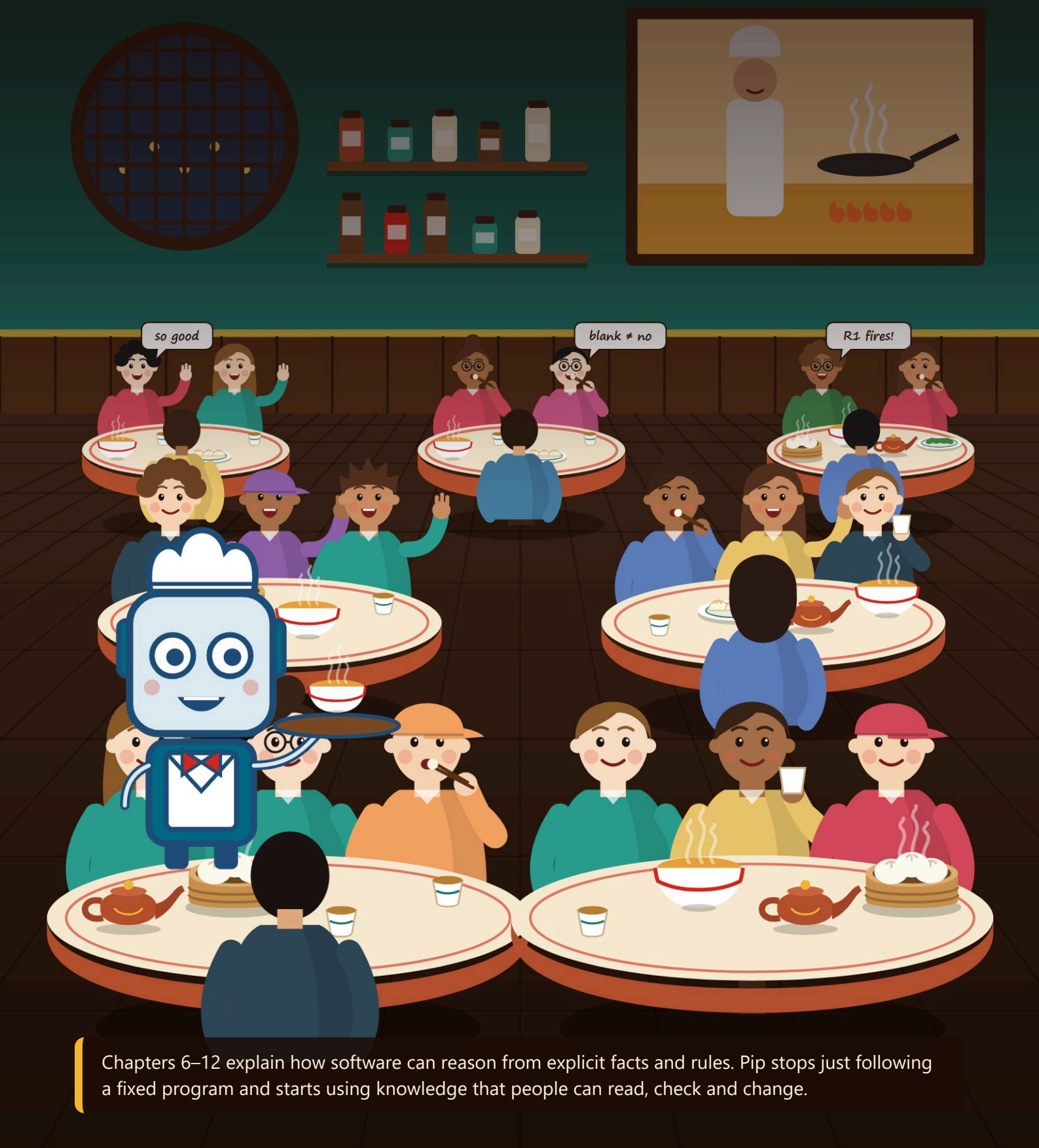
Mei asks to borrow a book. Answer without looking back. (1) What kind of software is the loan program, and what does the operating system do for it? (2) Which control structure decides the reply? (3) Which data structure fits the line of members at the desk? (4) Which table and which key tell you Mei's current loans? (5) What must you decide before you run a test?

Check (1) Application software; the operating system shares out the processor and memory. (2) Selection, an IF–ELSE chain. (3) A queue. (4) The Loans table, through the foreign key `member_id`. (5) The expected result. If one of these stumped you, go back to that chapter before you start Part B.



Part B

Symbolic AI



Chapters 6–12 explain how software can reason from explicit facts and rules. Pip stops just following a fixed program and starts using knowledge that people can read, check and change.

Chapter Six

SYMBOLIC AI

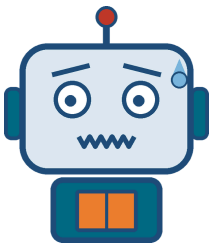
06

Introduction to Symbolic AI



On today's menu: house-rule dumplings

IF the dough is ready AND the filling is ready THEN fold. A rule only fires when every condition is met, and a blank on the order slip is not a "no".



Pip's problem

A patient's record says fever: yes, cough: yes, and the "difficulty breathing" box is blank. Pip announces: "No difficulty breathing, so no pneumonia!" Is Pip allowed to say that?

Your mission

Find out when a rule fires, why a blank is not a "no", and what explicit rules are good and bad at.

You will be able to

- Read an IF–THEN rule, say when it fires, and explain why "may have" and missing facts matter.
- Give the benefits and limitations of symbolic AI, and where it is used.

You need: Chapters 1 and 5.

Watch: Video 04. While watching, ask: what does a rule need before it can fire?



Predict first

Look at Pip's problem again. Can the pneumonia rule fire for this patient? And may Pip conclude "no pneumonia"? Write your answers and your reasons.

Symbolic AI represents knowledge explicitly with symbols, facts, and rules. It is also known as Good Old-Fashioned AI (GOFAI), a name for the classic approach to representing knowledge and reasoning with symbols. For the umbrella case:

```
Facts:  raining
        going_outside
Rule:   IF raining AND going_outside THEN recommend_umbrella
```

The rule **fires** when all its conditions match the available facts. The result is a conclusion supported by those facts and rules.

Rules in action: a medical example

```
IF the patient has a fever
AND the patient has a cough
AND the patient has difficulty breathing
THEN the patient may have pneumonia
```

If all three conditions are true, the rule can fire. The word "may" matters: this rule supports a possible conclusion for a doctor to check; it does not prove a diagnosis.

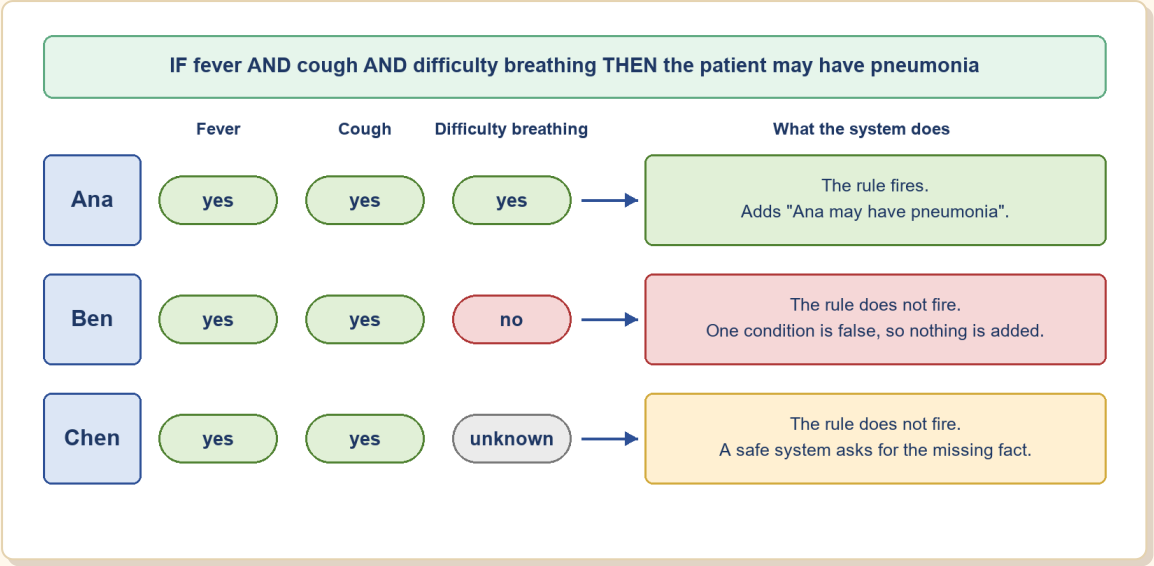


Figure 6.1. The same rule gives three different results: for Ana it fires; for Ben one condition is false; for Chen one condition is unknown.

Strengths and limits

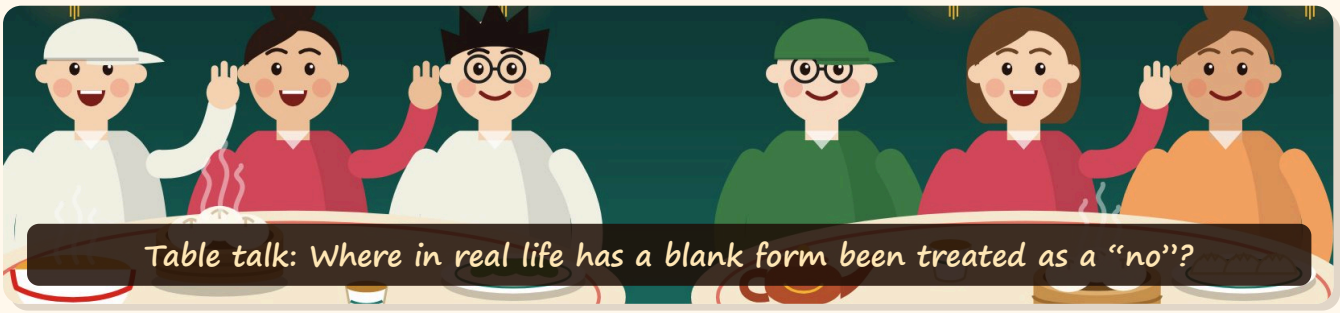
Explicit rules can be read, inspected, tested, and changed. This can make the reasoning easier to explain. Rules work well when important knowledge can be stated clearly. The course slides name three benefits:

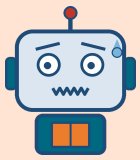
- **Interpretability:** every decision can be traced back to the exact facts and rules that produced it.
- **Knowledge representation:** knowledge is stored in a readable form.
- **Flexibility:** the rules are kept apart from the engine that applies them, so a rule can be added or changed on its own.

Symbolic systems also have limitations:

- **Incomplete knowledge:** important facts or rules may be missing or out of date, so people must update and check them.
- **Scalability:** a very large set of rules can be difficult to manage.
- **Uncertain or ambiguous information:** real cases may not fit clear yes-or-no rules, and facts or rules can conflict.
- **Limited learning:** a system using only hand-written rules does not automatically learn from examples.

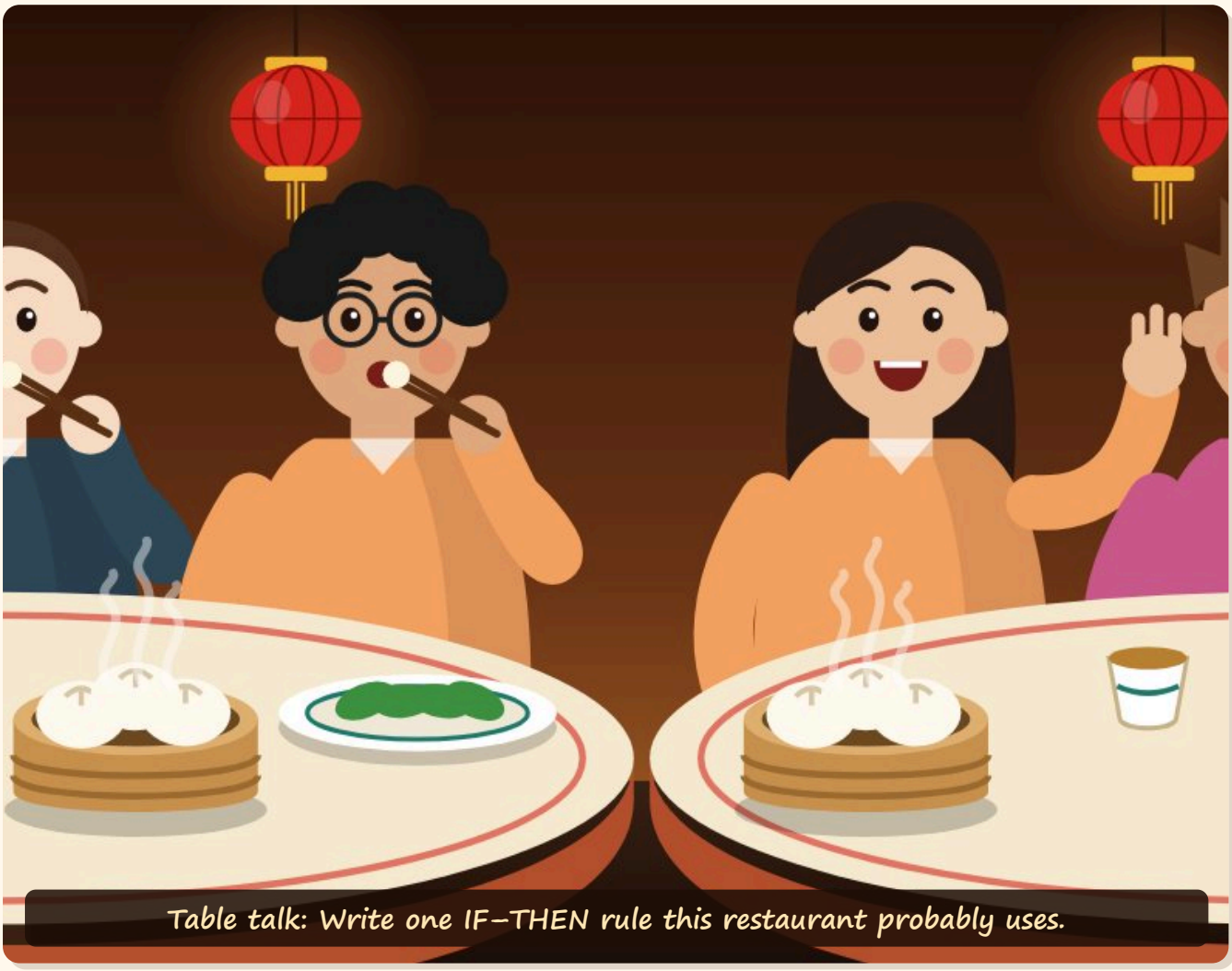
Symbolic AI appears in digital assistants, medical diagnosis systems, autonomous cars, and computer vision systems. In most modern products, symbolic rules work together with machine learning rather than alone.





Common mistake: "If it isn't recorded, it must be false."

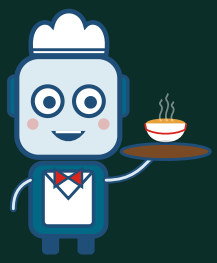
Missing information is not the same as false information. If a medical record says fever = true but the cough field is blank, the system does not know whether the patient has a cough. A rule requiring both fever and cough cannot fire until the missing information is supplied. Some rule engines do treat anything they cannot find as false: this is the *closed-world assumption*, which is safe only when the data is known to be complete.



ACTIVITY 6.1

Will the rule fire?

Goal	Distinguish true, false, and missing information.
Preparation	Use this rule: IF fever = true AND cough = true THEN qualify_screening = true. Then use the pneumonia rule above.
Task	1. Predict, then test: fever is true; cough is true. 2. Fever is true; cough is false. 3. Fever is true; cough is missing. 4. Pneumonia rule: Dina has a fever, a cough and difficulty breathing, and she also has a headache. Does the rule fire? 5. Pneumonia rule: Eli has difficulty breathing and a cough, but his temperature has not been measured. What should a well-designed system do?
Observe	Record whether the rule fires in each case.
Reflect	Why should the system treat "missing" differently from "false"? What was wrong with Pip's announcement?
Feedback	The screening rule fires only in case 1. In case 2, a condition is false. In case 3, the system lacks information; it should ask for it or report that it cannot decide. Dina's rule fires, and the extra fact changes nothing. For Eli, the rule cannot fire yet, and the system should ask for the temperature rather than assume there is no fever. **Back to your prediction:** for Pip's patient, the rule cannot fire yet, because one condition is unknown. That does not mean "no pneumonia". Pip treated a blank as a no, and that is what was wrong with the announcement.



Order up! Check your answers before the next dish.

Pip says *"A blank is not a no. A blank is not a yes. A blank is a blank. I have written this on my hand."*

Mission complete

Chapter 6: your bill

Takeaway: A rule-based system can explain which conditions led to a result, but it can only use the facts and rules it has.

Pip's log: Pip now says: "I cannot decide yet. Please check the patient's breathing."

Before you move on, can you:

☐ Read an IF–THEN rule, say when it fires, and explain why "may have" and missing facts matter.

☐ Give the benefits and limitations of symbolic AI, and where it is used.

Practise next: Test Bank T11–T12, M11–M12, O05 (after Chapter 9). *Optional:* Rulebound, Stage 6.

Sources: chap4 slides 3–7; Russell and Norvig (2021), Ch. 1 and 7; Newell and Simon (1976).



Chapter Seven

LOGIC AND REASONING

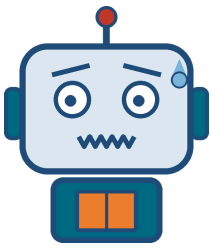
07

Logic and Reasoning



On today's menu: a pot of tea for two

The house blend is always served hot. This tea is hot. So it is the house blend? Pour a cup, and work out what really follows.



Pip's problem

Pip sees a wet floor by the door and tells everyone: "If it is raining, the floor gets wet. The floor is wet. So it is raining!" Then the cleaner walks past with a mop.

Your mission

Learn the rules that say which conclusions really follow, and spot the arguments that only look right.

You will be able to

- Work out AND, OR, NOT and implication, and read a truth table.
- Use modus ponens and modus tollens, and spot the two invalid forms.
- Read and write simple statements with predicates and quantifiers.

You need: Chapter 6. Comfort reading AND and OR conditions.

Watch: Video 04. While watching, pause at each argument and say whether it is valid.



Predict first

Is Pip's argument about the wet floor valid? Write yes or no, and give one other way the floor could get wet.

Logic gives precise ways to represent statements and check whether conclusions follow. A **proposition** is a statement that is true or false. Common operators include:

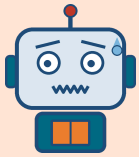
- **NOT P:** reverses P's truth value.
- **P AND Q:** true only when both are true.
- **P OR Q:** true when at least one is true.
- **P → Q:** "If P, then Q." It is false only when P is true and Q is false.
- **P ↔ Q:** "P if and only if Q"; each statement implies the other.

P	Q	P AND Q	P OR Q	NOT P
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Logical OR is *inclusive*: P OR Q is true when both are true.

P	Q	P → Q
True	True	True
True	False	False
False	True	True
False	False	True

Why are the last two rows true? Think of P → Q as a promise: "If it rains, the match is cancelled." The promise is broken only when it rains and the match goes ahead. If it does not rain, the promise has not been broken, whatever happens to the match.



Common mistake: "It did not rain, so the rule must be false."

Pip heard "If it rains, the match is cancelled." On a dry day the match was cancelled anyway, and Pip announced that the rule was broken. It was not. An IF part that is false leaves the promise untouched.

Try it 7.2 — Is it a proposition?

Which of these are propositions as written: "Beijing is in China," "Please sit down," and `y + 1 = 5`?

Check Only the first. A command has no truth value, and `y + 1 = 5` is not true or false until y is given a value.

Two useful equivalences are **De Morgan's laws**: NOT (P AND Q) is equivalent to (NOT P) OR (NOT Q). NOT (P OR Q) is equivalent to (NOT P) AND (NOT Q).

In the library. A member may borrow when the card is valid (c) AND there are no overdue books (n). When may the member *not* borrow? NOT (c AND n), which by De Morgan is (NOT c) OR (NOT n): the card is not valid, or there are overdue books. Either reason alone is enough.

Inference rules

Modus ponens: from P → Q and P, conclude Q. **Modus tollens:** from P → Q and NOT Q, conclude NOT P. Two common mistakes look similar but are not valid:

- **Affirming the consequent:** from P → Q and Q, concluding P. *The ground is wet, so it is raining?* Not necessarily: a sprinkler may have been on.
- **Denying the antecedent:** from P → Q and NOT P, concluding NOT Q. *It is not raining, so the ground is not wet?* Again, not necessarily.



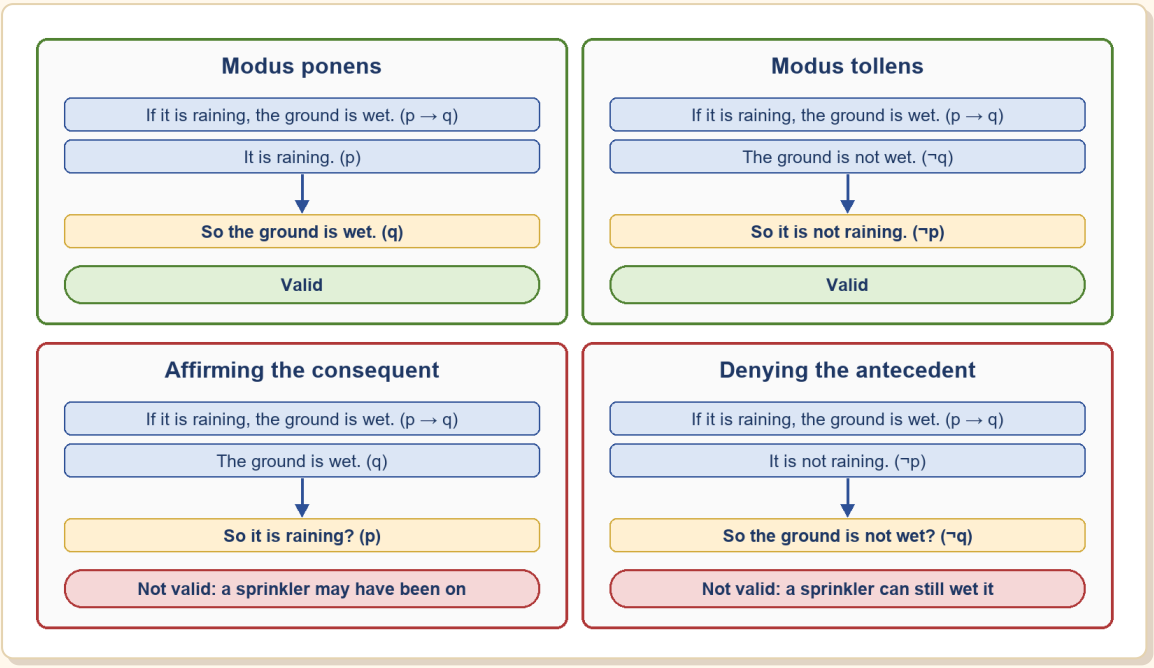


Figure 7.1. Two valid forms of reasoning, and the two common mistakes that look like them. In the figure, ¬ means NOT.

Back to your prediction. Pip's argument has the form of affirming the consequent, so it is not valid. The cleaner's mop also wets the floor.

Predicates and quantifiers

Predicate logic describes properties and relationships involving objects. `ValidCard(Mei)` is a predicate about Mei. A universal quantifier ∀ means "for every"; an existential quantifier ∃ means "there is at least one." Examples (∧ means AND):

```
∀m (ValidCard(m) ∧ NoOverdue(m) → MayBorrow(m))
"Every member with a valid card and no overdue books may borrow."

∃x Available(x)
"There is at least one available item."
```

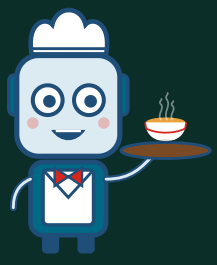
Worked example: Mei. (1) The library's rule: ∀m (ValidCard(m) ∧ NoOverdue(m) → MayBorrow(m)). (2) The facts: ValidCard(Mei) ∧ NoOverdue(Mei). (3) What is true for every member is true for Mei, so ValidCard(Mei) ∧ NoOverdue(Mei) → MayBorrow(Mei). (4) By modus ponens on (2) and (3): **MayBorrow(Mei)**.



ACTIVITY 7.1

Formalize and reason

Goal	Translate a sentence into logic and use an inference rule.
Preparation	Let P mean “the member has a valid card and no overdue books.” Let Q mean “the member may borrow.” Before you start, predict: if Q is false, can you tell which part of P failed?
Task	1. Write “If P, then Q.” 2. If P is true, state what follows. 3. If Q is false, state what follows from the rule. 4. Explain whether the rule tells you exactly which part of P is false.
Observe	Name the inference rule you used.
Reflect	Why does the rule let you reason from “Q is false” but not from “Q is true”?
Hint	Write the rule’s two parts on paper. Modus ponens moves forward from P. Modus tollens moves backward from NOT Q.
Feedback	P → Q and P imply Q by modus ponens. P → Q and NOT Q imply NOT P by modus tollens. The rule does not identify which part of P failed; more information is needed. From Q alone you cannot conclude P: that would be affirming the consequent.



Order up! Check your answers before the next dish.

Try it 7.3 — Valid or not?

Rule: if the book is on loan, it is not on the shelf. (a) The book is on loan. So it is not on the shelf. (b) The book is not on the shelf. So it is on loan. Which argument is valid?

Check (a) is valid: modus ponens. (b) is not valid: affirming the consequent. The book may be on a repair trolley.

Try it 7.4 — Formalize a sentence

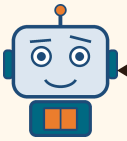
Formalize "Some members have overdue books." Use `Member(x)` and `HasOverdue(x)`. Work in four steps: name the predicates; choose the variable; let the word choose the quantifier ("every" gives ∀, "some" gives ∃); then join the parts (use → with ∀, and ∧ with ∃).

Check ∃x (Member(x) ∧ HasOverdue(x)). With ∃, use ∧, not →: with →, any non-member would make the sentence true, even if no member has an overdue book.

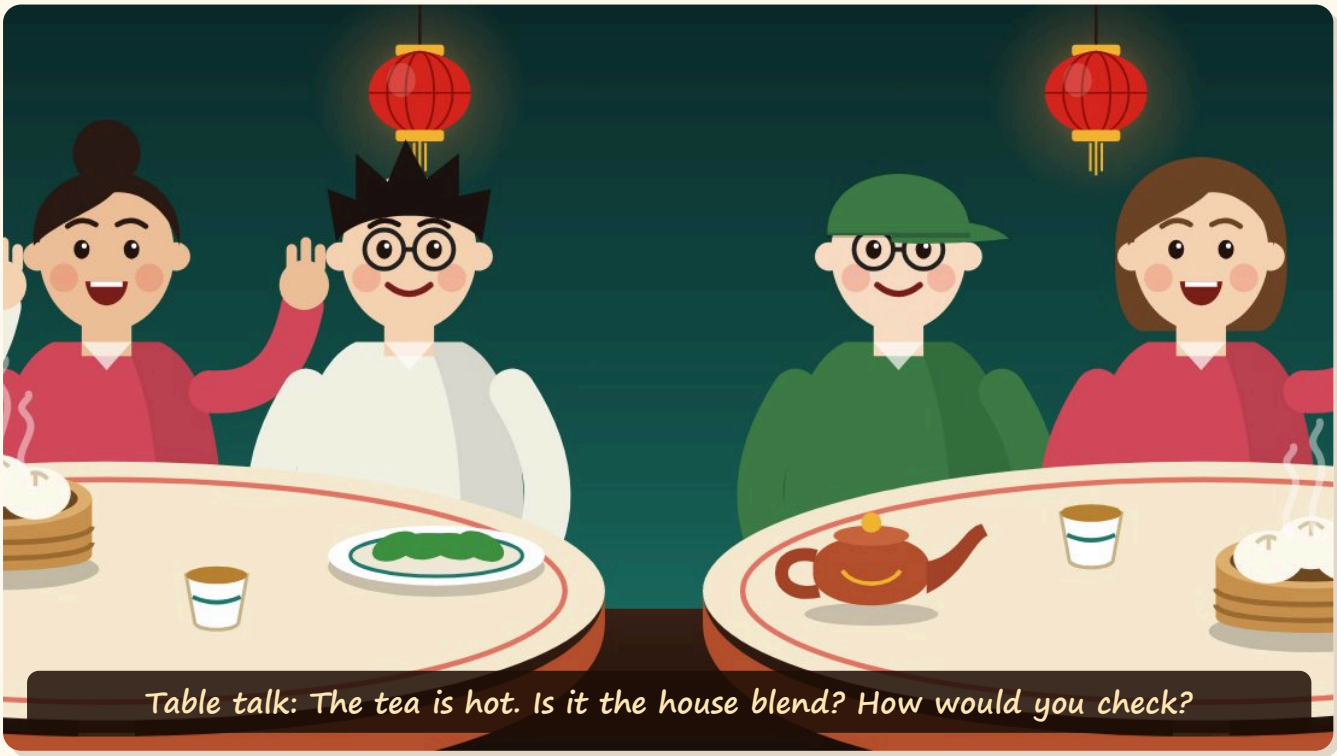
Try it 7.5 — Flip it with De Morgan

Rewrite "It is not true that the shop is open and has bread" using De Morgan's law. Let o mean "the shop is open" and b mean "the shop has bread".

Check NOT (o AND b) becomes (NOT o) OR (NOT b): the shop is not open, or it has no bread. Either one is enough to make the first sentence true.



Pip says *"If I am a robot, then I like books. I like books. So I am a robot. I checked: the conclusion is true, and the argument is still not valid. This is very annoying."*



Mission complete

Chapter 7: your bill

Takeaway: Logic helps make conditions and conclusions precise. It does not supply facts that have not been provided.

Pip's log: Pip has apologized to the cleaner, and now asks: "Is there another way the floor could get wet?"

Before you move on, can you:

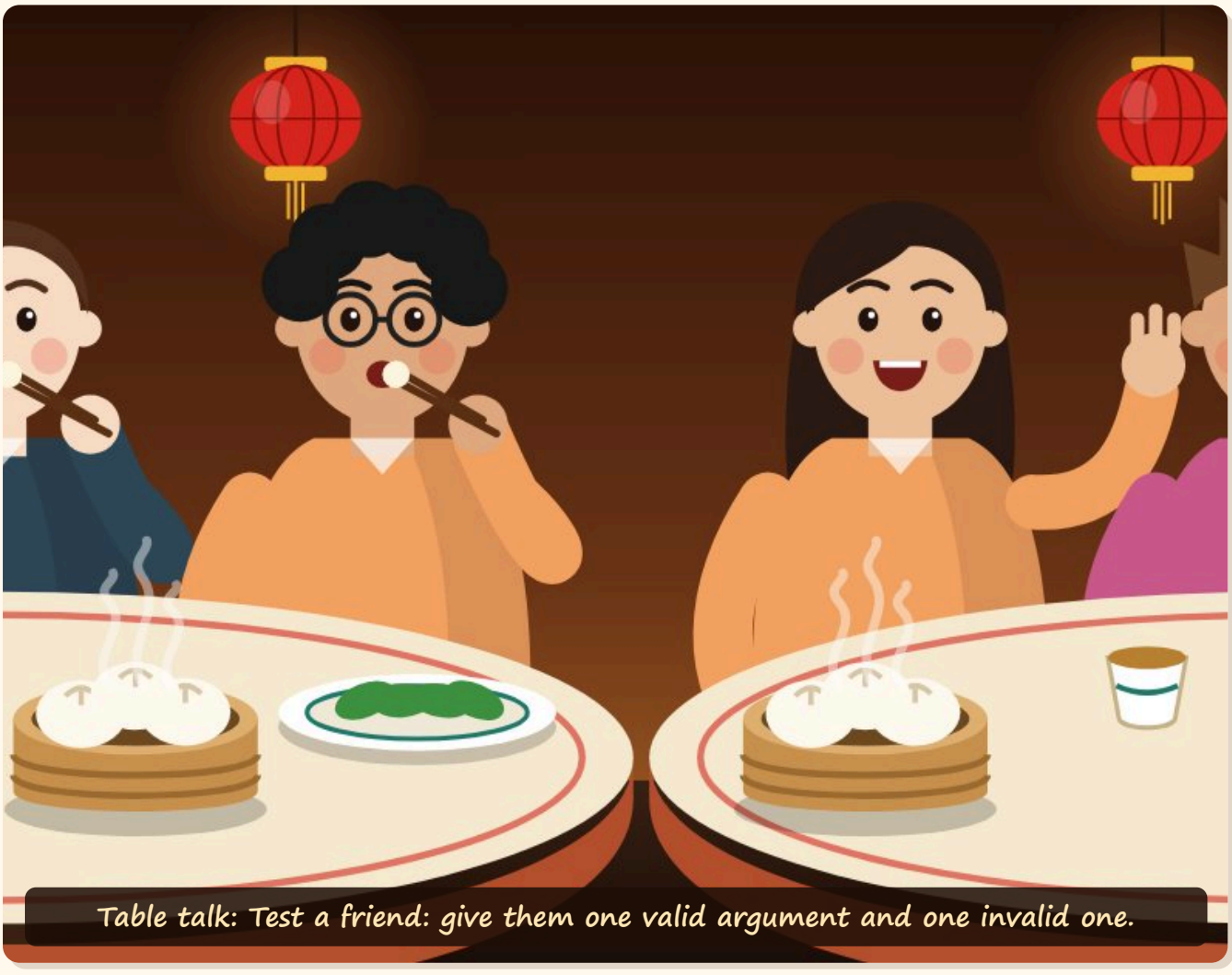
☐ Work out AND, OR, NOT and implication, and read a truth table.

☐ Use modus ponens and modus tollens, and spot the two invalid forms.

☐ Read and write simple statements with predicates and quantifiers.

Practise next: Test Bank T13–T14, M13–M14, M38, O04 (after Chapter 8). *Optional:* Rulebound, Stage 7.

Sources: chap4 slides 8–20; Russell and Norvig (2021), Ch. 7–9; Brookshear and Brylow (2019), Ch. 11.



Chapter Eight

08

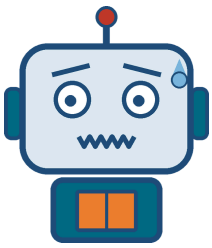
KNOWLEDGE REPRESENTATION

Knowledge Representation



On today's menu: the spice shelf

Every jar has a label and a place. Store your knowledge well, and you can find it in a blink.



Pip's problem

Someone has given Pip the library's knowledge as one enormous paragraph. Pip takes ten minutes to answer "Does *Harry Potter 1* have pages?", and nobody can see which part of the paragraph to change when a policy changes.

Your mission

Try different ways of storing the same knowledge, and choose the one that fits the job.

You will be able to

- Compare logic, semantic networks, frames, scripts, rules and ontologies.
- Use inheritance in a semantic network.

You need: Chapters 4, 6 and 7.

Watch: Video 05. While watching, ask: what does this form make easy to see?



Predict first

Sketch how you would store "*Harry Potter 1* is a book, and books have pages" so that a program could answer "Does *Harry Potter 1* have pages?" Use any drawing or table you like, and keep it to compare with Figure 8.1.

Knowledge representation is how a system stores information so that it can use it. Different forms make different relationships or rules easier to see.

Representation	What it shows	Library example
Logic	Statements and relationships that can be reasoned about	`Available(BookA)`
Semantic network	Concepts as nodes and relationships as labelled links	Mei —is-a→ Member
Frame	Fields describing a typical object or situation	Member: card status, loans, fines
Script	A familiar sequence of events	Select member → check loan → record checkout
Rules	Conditions connected to conclusions	IF card valid AND no overdue books THEN may borrow
Ontology	Concepts, categories, and relationships	A book copy is a copy of a title; "car" and "automobile" are related terms

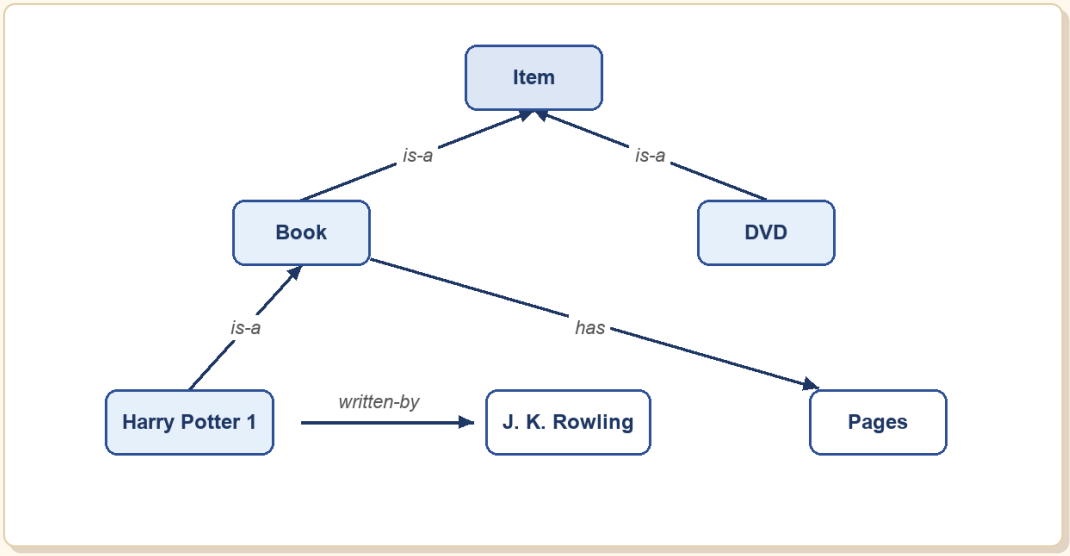
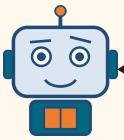


Figure 8.1. A semantic network for part of the library.

Inheritance. Does *Harry Potter 1* have pages? The network does not say so directly. But *Harry Potter 1* is-a Book, and Book has Pages, so by inheritance, yes: a node has the properties of the nodes above it.



Pip says "I checked the network. Harry Potter 1 has pages. I do not. I am not a Book. This took longer than it should have."

Frames and defaults. A frame is like a form to fill in. Slots can have default values, which are used unless something more specific says otherwise:

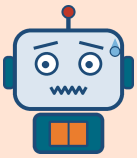
Frame: Book	Value	Frame: Reference book	Value
is-a	Item	is-a	Book
title, author	(fill in)	can be borrowed	**no** (overrides the default)
loan period	14 days (default)	loan period	none (overrides the default)
can be borrowed	yes (default)		

A dictionary in the reference section fills in the Reference-book frame. Book says "can be borrowed: yes", but Reference book says "no", so the dictionary cannot be borrowed. When two frames disagree, the more specific one wins. This is called **overriding**.

Scripts describe a typical series of events. *Borrowing a book:* (1) the member brings a book and a card to the desk; (2) the librarian scans both; (3) the system applies the loan rules; (4) the loan is recorded with a due date; (5) the member leaves with the book. Scripts let a system fill gaps in a story. If someone says "Mei went to the desk and left with the book", a system with this script can infer that her card was checked and a loan was recorded.

An ontology can help a search system connect related terms, but people must build and maintain it. Each representation leaves some things unstated, so the system should not infer more than its stored information supports.

Form	Best for	Example in this chapter
Logic	Precise statements and proofs	`Available(BookA)`
Semantic network	Seeing relationships and inheritance	Figure 8.1
Frames and scripts	Typical objects with defaults; typical event sequences	Book frame; borrowing script
Rules	Decisions and actions	R1, R2
Ontology	A shared vocabulary between people and programs	"car" and "automobile"



Common mistake: "The Book frame says yes, so every book can be borrowed."

That is how Pip nearly lent out the only dictionary. A default holds only until something more specific overrides it. Always check the most specific frame first.

Try it 8.2 — Extend the network

Add to Figure 8.1: "A DVD has a running time" and "*Spirited Away* is a DVD". Write the new links. Does *Spirited Away* have a running time?

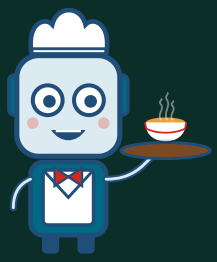
Check DVD —has→ Running time; *Spirited Away* —is-a→ DVD. Yes, by inheritance.



ACTIVITY 8.1

Represent the same case in different ways

Goal	Compare a relationship representation with a decision rule.
Preparation	Use these facts: Mei has a valid card. Mei has no overdue books. Book A is available. Before you start, predict which of the forms would be easiest to change if the library's loan policy changed.
Task	1. Draw a small semantic network for Mei and Book A. 2. Write a rule that would approve a loan if its conditions are met. 3. Write a frame for a member with at least four fields. 4. State one way an ontology could help a library search. 5. Write the borrowing script in your own words, in four or five steps. 6. Predict, then check with the frames: can a dictionary from the reference section be borrowed?
Observe	Mark which information appears in the network and which appears in the rule.
Reflect	Which representation would be easier to update if the library changes its loan policy?
Hint	A frame is a form to fill in, so name the fields first. A script is a sequence, so start at the desk and finish when the member leaves.
Feedback	A sample network: Mei —is-a→ Member; Book A —is-a→ Book; Mei —has→ valid card. A sample frame: is-a: Person; name: (fill in); card valid: (fill in); maximum loans: 5 (default); overdue books: 0 (default). The network makes entities and relationships visible. The rule states conditions for a decision, and is the easiest to update when the policy changes. An ontology can represent related concepts such as a book copy and its title. A system can use more than one representation, but new knowledge still needs to be added and checked. A dictionary cannot be borrowed: the Reference-book frame overrides Book's default.



Order up! Check your answers before the next dish.

Mission complete

Chapter 8: your bill

Takeaway: Choose a representation that makes the needed knowledge and relationships clear.

Pip's log: Pip has swapped its giant paragraph for a network, some frames and a short list of rules, and answers in a blink.

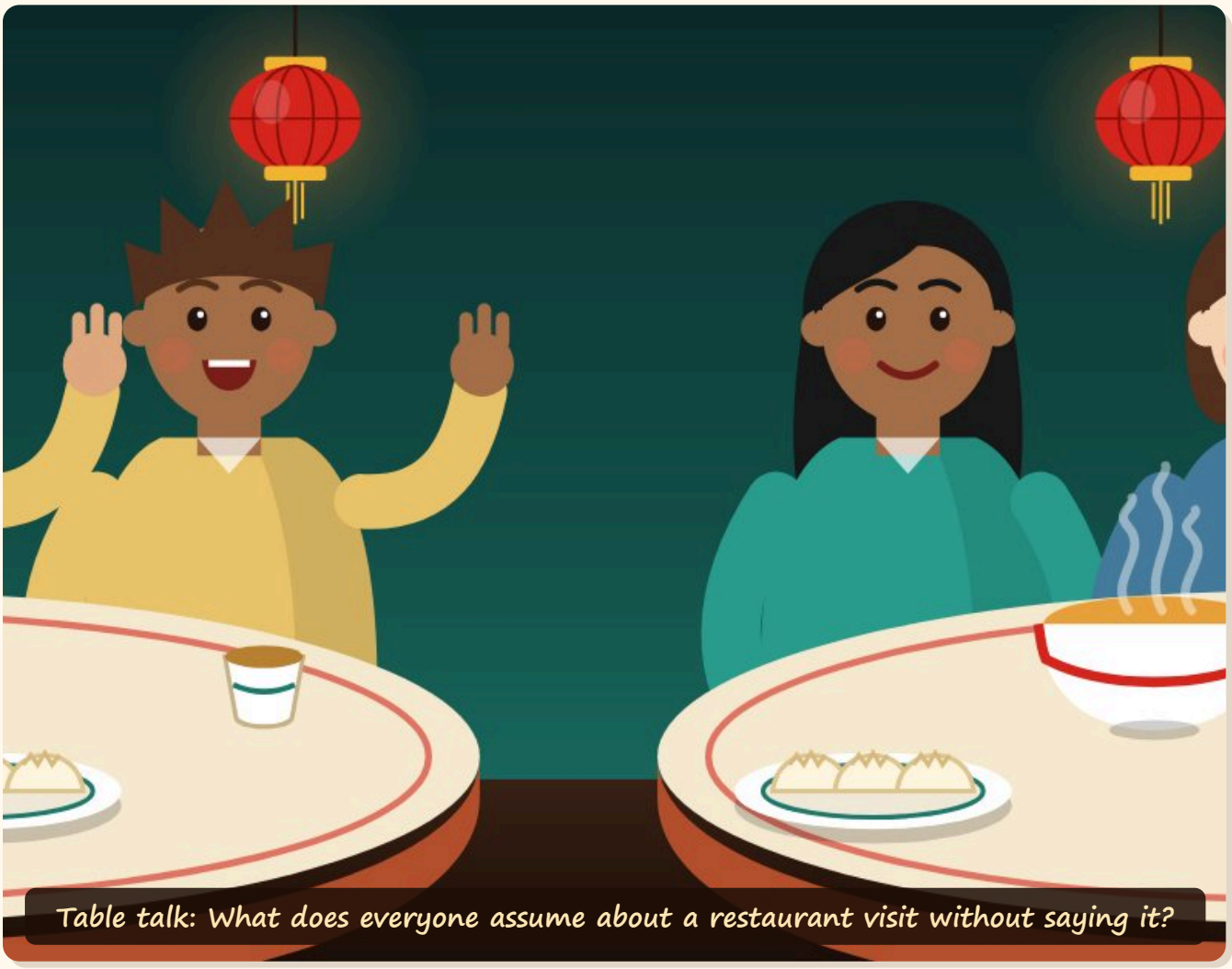
Before you move on, can you:

☐ Compare logic, semantic networks, frames, scripts, rules and ontologies.

☐ Use inheritance in a semantic network.

Practise next: Test Bank T15–T16, M15–M16, M31, O04. *Optional:* Rulebound, Stage 8.

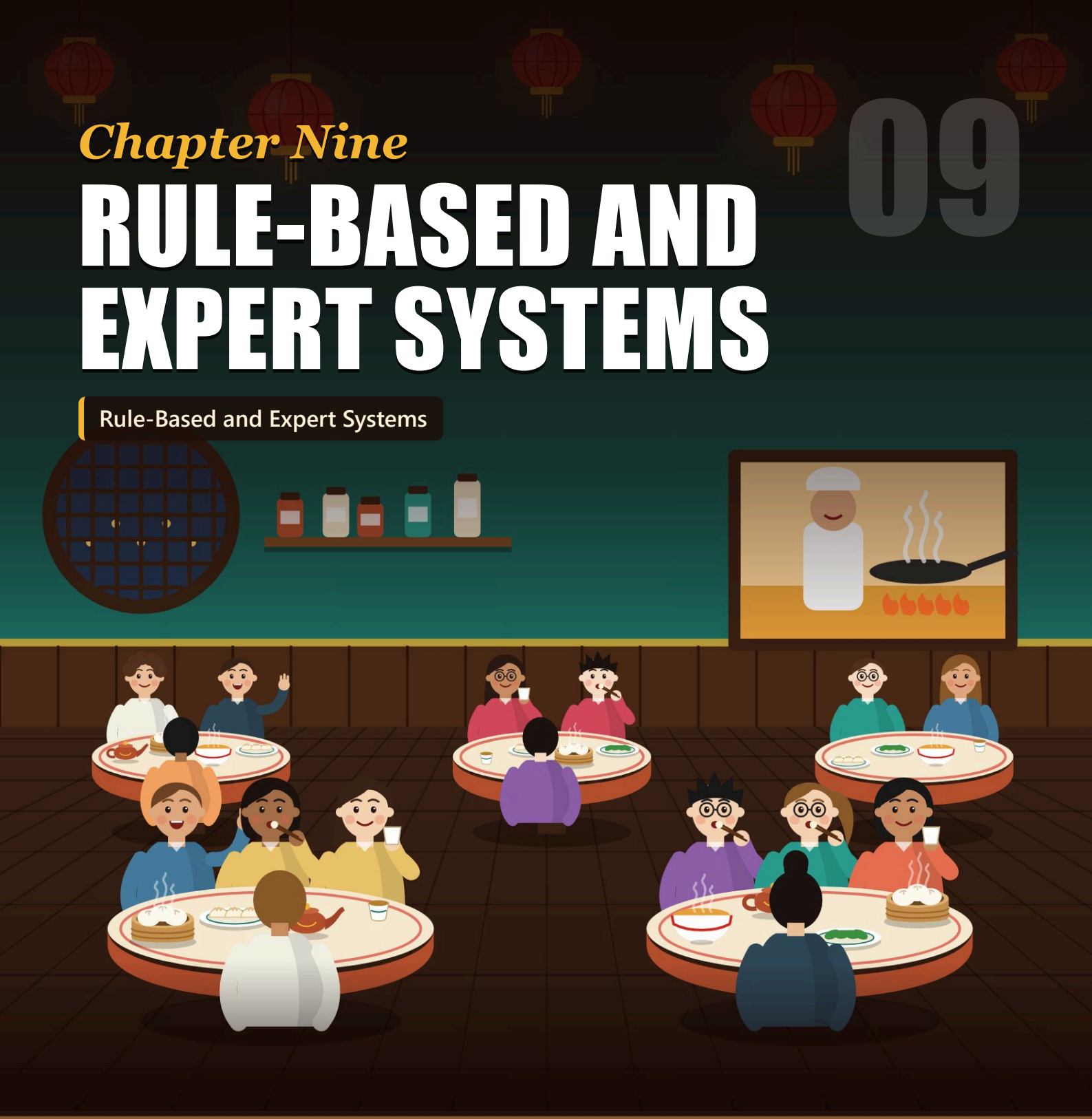
Sources: chap4 slides 21–28; Russell and Norvig (2021), Ch. 10; Minsky (1974); Schank and Abelson (1977); Gruber (1993).



Chapter Nine

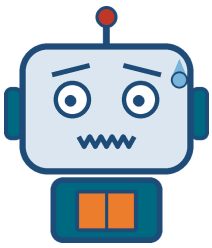
09 RULE-BASED AND EXPERT SYSTEMS

Rule-Based and Expert Systems



On today's menu: the chef's special

A cookbook, a counter and a chef: the picture this chapter uses to show how rules become decisions.



Pip's problem

A member wants a book that is out on loan. R2 cannot fire, so Pip announces: "Loan refused!" But nobody ever gave Pip a rule that says refused. Pip has facts and rules, and no program telling it what to do with them. How can it reach a decision on its own, and what may it say when no rule gives it a conclusion?

Your mission

See how a rule-based system chains facts and rules to reach a conclusion. This is the central idea of the whole module.

You will be able to

- Say what the knowledge base, working memory, inference engine and user interface each do.
- Trace forward and backward chaining on the library loan.
- Explain how MYCIN combined certainty factors, and why rule engines keep rules outside the code.

You need: Chapters 3, 5, 7 and 8.

Watch: Video 05. While watching, pause before each rule fires and predict which one is next.



Predict first

A member has a valid card and no overdue books, and the book is available. R1 says who may borrow. R2 says whether the loan is approved, and needs R1's conclusion. Which rule fires first? Check your answer in the worked example.

A rule-based system commonly contains:

Component	Role
Knowledge base	Stores rules and lasting domain knowledge.
Working memory	Holds facts about the current case and new conclusions.
Inference engine	Matches facts to rules and decides which rule to fire.
User interface	Lets a person provide information and receive results or explanations.



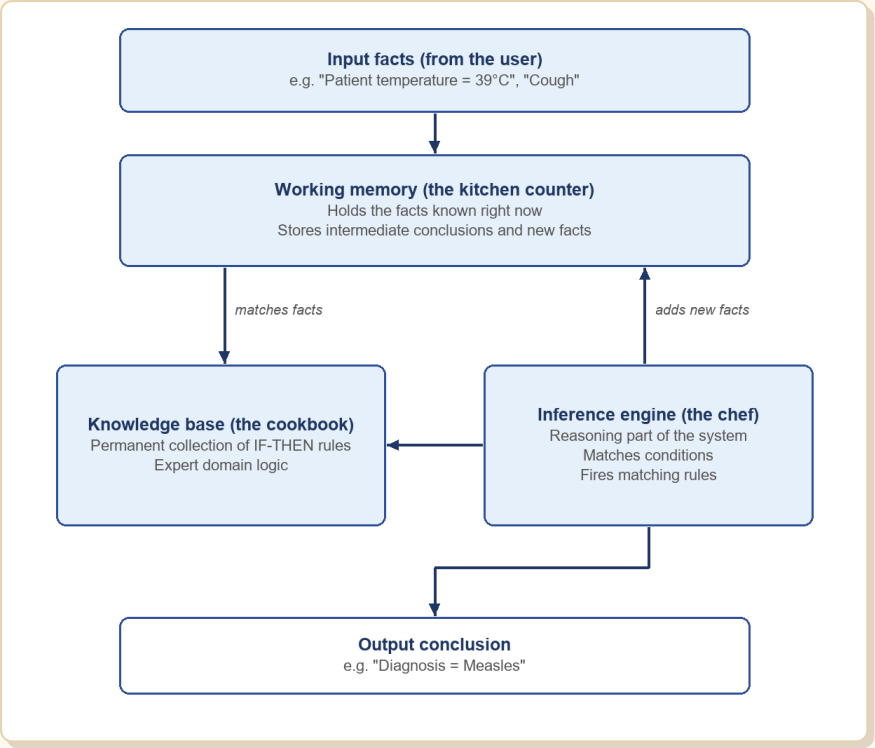


Figure 9.1. The kitchen picture, shown on a medical case: the cookbook holds the rules, the counter holds the facts, and the chef does the matching.

The kitchen is only an analogy: an inference engine has no skill or judgement of its own. It only matches facts against rules and fires those that match.

A rule **fires** when its conditions match. A new conclusion can become a fact that enables another rule to fire. **Forward chaining** starts with known facts and applies matching rules, adding conclusions as new facts, until no rule can add anything new. **Backward chaining** starts from a goal —such as "Is the loan approved?"—and works backward to find the facts needed to support it. Forward chaining is Chapter 7's modus ponens used again and again: when a rule's IF part is known to be true, its THEN part is added as a new fact.

What if several rules match at the same moment? They form the **conflict set**, and the engine must pick one to fire first. A simple way is to take the first match in the list, or to give some rules priority.

Worked example: the library loan

F1	The member has a valid library card.
F2	The member has no overdue books.
F3	The requested book is available.
R1	IF the member has a valid card AND has no overdue books, THEN the member may borrow.
R2	IF the member may borrow AND the requested book is available, THEN the loan is approved.

Step	Facts known	Rule check	Result
1	F1, F2, F3	R1 matches: the card is valid and there are no overdue books. R2 cannot fire yet.	Add F4: member may borrow
2	F1 to F4	R2 now matches: the member may borrow and the book is available.	Add F5: loan approved
3	F1 to F5	No rule can add a new conclusion.	Stop

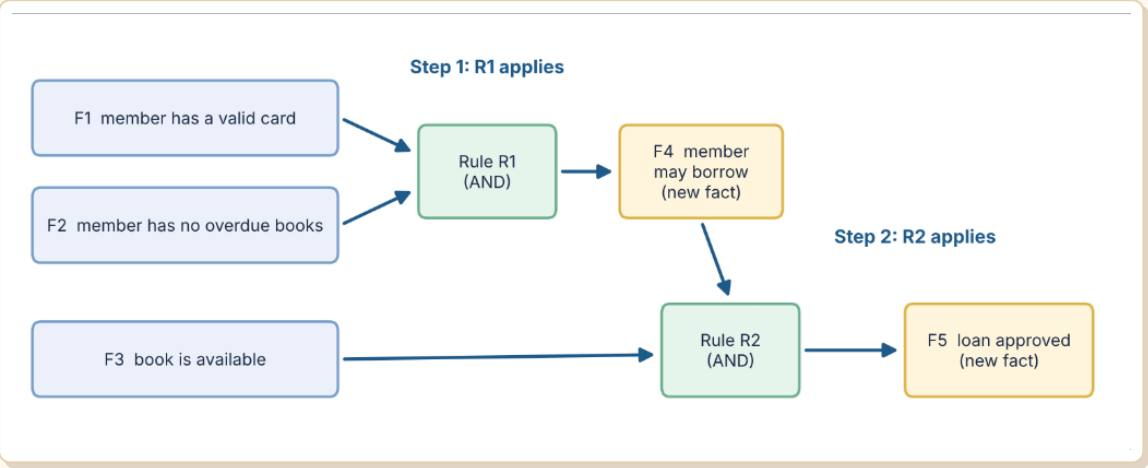
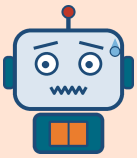


Figure 9.2. Forward chaining for the library loan: R1 adds F4, then R2 uses F4 to add F5.

Backward chaining on the same loan. Start from the goal "loan approved".

Step	Current goal	What the engine does	Result
1	Loan approved	R2 can conclude this. It needs "member may borrow" AND "book available".	Two new goals
2	Member may borrow	R1 can conclude this. It needs "valid card" AND "no overdue books".	Two new goals
3	Valid card	No rule concludes this, so check the facts or ask the user.	F1 is true: met
4	No overdue books	Check the facts or ask the user.	F2 is true: met, so R1 proves "member may borrow"
5	Book available	Check the facts or ask the user.	F3 is true: met, so R2 proves "loan approved"

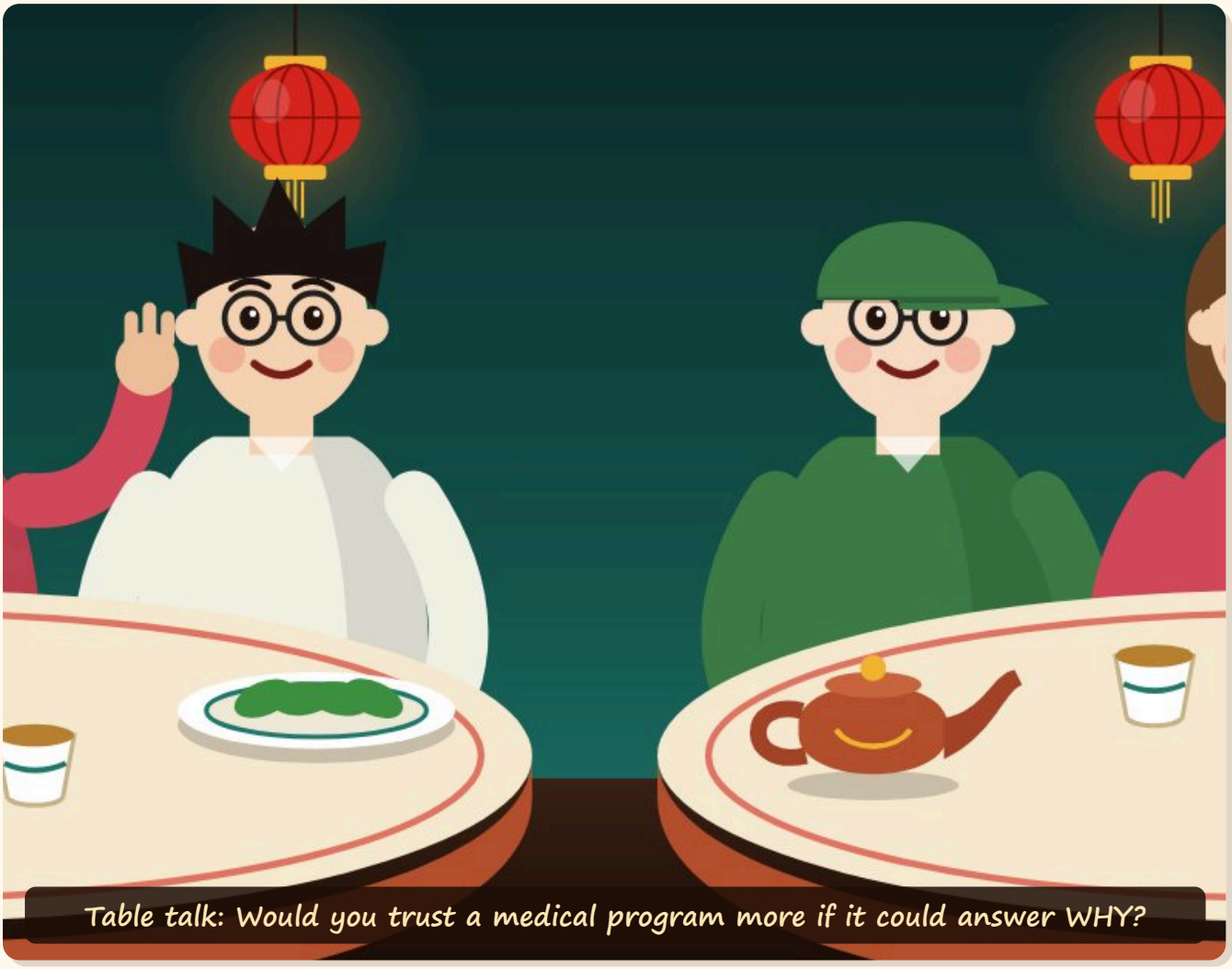
Both methods reach the same conclusion. Backward chaining looks only for what the goal needs: if the member's card were not valid, it would never need to ask about overdue books. Rule order may change how many checks a system makes. For rules that only add facts, changing the order should not change the final conclusions.



Common mistake: "R2 did not fire, so the loan is refused."

If the book is unavailable, R2 does not fire. The system cannot approve the loan from these rules. It also cannot conclude "refused" unless a rule supports that conclusion. A rule-based system concludes only what its rules support.

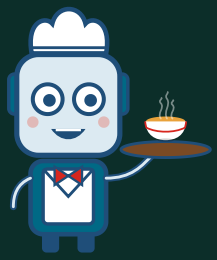
Pip says *"Nobody told me to say refused, so now I say nothing. The member found this very rude. Activity 9.1 shows a fix for overdue books."*



ACTIVITY 9.1

Be the inference engine

Goal	Trace forward chaining by hand, and see what happens when a rule cannot fire.
Preparation	Write R1 and R2 on one card each. Write each starting fact on a small slip, and put the slips on the table: that table is your working memory. No computer is needed.
Task	1. Case A: put down F1, F2 and F3. Predict which rule fires first. Then fire rules one at a time, adding a new slip for each conclusion. 2. Case B: a member has a valid card and the book is available, but the member has an overdue book. Predict, then trace. 3. Case C: F1 and F2 are true, but the book is on loan. Trace again. 4. Write a new rule so that the system can say "refused" in Case B.
Observe	Record which rules fire, and which slips are on the table at the end of each case.
Reflect	Compare the slips with what you would say as a person. Why is "no conclusion" not the same as "refused"?
Hint	Check every rule against the slips on the table. A new slip can let another rule fire, so go round again until nothing changes.
Feedback	Case A: R1 adds "may borrow", then R2 adds "loan approved". Case B: R1 does not fire, because "no overdue books" is false, and R2 needs "may borrow", which was never added. No rule fires, and there is no conclusion. Case C: R1 fires, but R2 cannot, so the system can neither approve nor refuse. One possible rule: **R4:** IF the member has overdue books, THEN the loan is refused. A similar rule would cover Case C: IF the requested book is not available, THEN the loan is refused.



Order up! Check your answers before the next dish.

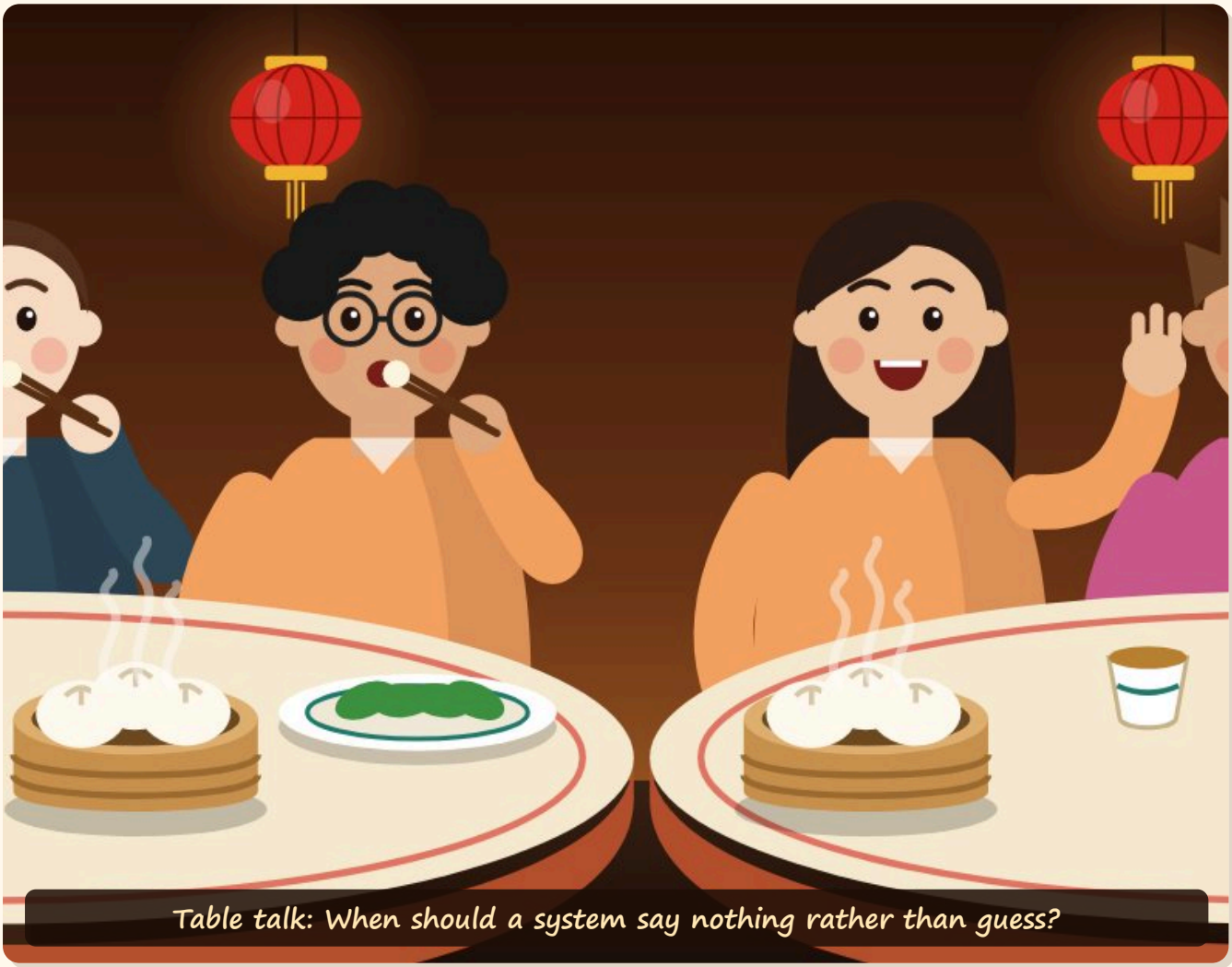
Want to see rules fire in a fight? Play Stage 9 of Rulebound, the optional game described before Part A (Figures 0.2 and 0.3). The rival's IF-THEN rules appear on screen as it fights. Pick one rule and explain when it would fire.

Mini Prolog exploration

Prolog is a programming language often used to represent facts and rules. Here is the library loan in Prolog. Each line ends with a full stop, `:-` means "if", a comma means "and", and a word starting with a capital letter, such as `Person`, is a variable.

```
valid_card(mei).
no_overdue_books(mei).
book_available(book_a).

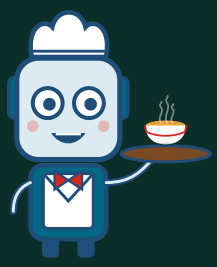
may_borrow(Person) :- valid_card(Person), no_overdue_books(Person).
approve_loan(Person, Book) :- may_borrow(Person), book_available(Book).
```



ACTIVITY 9.2

Predict, query, and change a rule

Goal	Connect facts and rules to a computer query.
Preparation	Use the Prolog example above. In SWISH (see the Resource below), choose to create a new program: the program goes in the program editor, and each query is run separately. `?-` is the prompt; type what follows it. Put each new fact beside the matching facts. If you cannot use the site, trace by hand: the feedback gives the expected output.
Task	1. Predict whether `?- approve_loan(mei, book_a).` succeeds. Then run it. 2. Predict and run `?- may_borrow(Who).` 3. Change the fact to `book_available(book_c).` Predict the result of the first query again and run it. 4. Change it back, and add `book_available(book_b).` Test a query for that book. 5. Add `valid_card(tom).` Predict and run `?- may_borrow(tom).` 6. Change a rule: the library adds a loan limit. Add `under_limit(Person)` to the `may_borrow` rule, then add the facts `under_limit(mei).` and `no_overdue_books(tom).` Predict, then run `?- may_borrow(mei).` and `?- may_borrow(tom).`
Observe	Record which facts and rules support each result.
Reflect	Does a failed query prove that a real-world fact is false, or only that the current knowledge base does not prove it?
Hint	If SWISH reports a syntax error, check that every line ends with a full stop and that names of facts and members start with a lowercase letter.
Feedback	The first query succeeds (`true`), because the facts support both rules. `Who = mei`. With the fact changed, the query fails (`false`), because the rule cannot establish that book_a is available. With the new fact, `approve_loan(mei, book_b)` succeeds. `may_borrow(tom)` fails, because no fact says that Tom has no overdue books. After the rule change, `may_borrow(mei)` succeeds, and `may_borrow(tom)` fails, because no fact says that Tom is under the limit. In an incomplete real-world system, this failure should not automatically be treated as proof that the book is unavailable, or that Tom has overdue books.



Order up! Check your answers before the next dish.

Resource: SWISH

What is it? SWISH, from SWI-Prolog: swish.swi-prolog.org

Why use it? It lets you run the rules above and see exactly what follows from them.

Which part? Create a new program from the start page; the Help menu explains where to enter a program and how to run a query.

What to do Follow Activity 9.2.

Watch out Do not assume the online environment supports every feature available in a local installation. A `false` answer means "cannot be proved from this program".

If you cannot open it Trace the rules by hand and use the expected output in the Activity's feedback.

Expert systems in practice: MYCIN

MYCIN was a medical expert system with about 450 expert-written IF–THEN rules in LISP. It used backward chaining to support medical recommendations, asking the doctor only the questions it needed. A doctor could ask WHY to see the rule it was trying to use, and HOW to see which rules and facts led to a conclusion. It was an influential research system, but it was never used routinely in patient care: typing in a consultation took time, it had no links to other hospital computers, and collecting and maintaining expert knowledge was difficult.

MYCIN used **certainty factors** from −1.0 (definitely false) to +1.0 (definitely true); 0 means no evidence either way. A certainty factor is a rule-system measure of support, not a guarantee or an ordinary probability. For two positive factors supporting the same conclusion:

```
combined CF = CF1 + CF2 × (1 − CF1)
CF1 = 0.8, CF2 = 0.5
combined CF = 0.8 + 0.5 × (1 − 0.8) = 0.9
```

What the formula means, step by step. (1) Find the doubt left after the first rule: 1 − 0.8 = 0.2. (2) Let the second rule remove its share of that doubt: 0.5 × 0.2 = 0.1. (3) Add this to the first factor: 0.8 + 0.1 = 0.9. The belief grows, but it can never pass 1.0. Swapping the rules gives the same answer.

Try it 9.3 — Combine two certainty factors

Two positive rules support the same conclusion with CFs 0.6 and 0.5. Use the formula above to combine them.

Check 0.6 + 0.5 × (1 − 0.6) = 0.6 + 0.2 = 0.8. The answer is a combined certainty factor under the stated rule, not a guarantee that the conclusion is true.

Modern **business rule engines**, such as **Drools**, keep organizational rules in a separate rule system. This can make policy easier to inspect and change without rewriting application code, but each change still needs human review, testing and deployment.

Mission complete

Chapter 9: your bill

Takeaway: Rule-based systems make reasoning inspectable, but they depend on the quality and completeness of their facts and rules.

Pip's log: Pip can now approve Mei's loan, and explain why: F1 and F2 fired R1, then F4 and F3 fired R2. When no rule gives an answer, Pip now says: "I have no rule that lets me decide this."

Before you move on, can you:

☐ Say what the knowledge base, working memory, inference engine and user interface each do.

☐ Trace forward and backward chaining on the library loan.

☐ Explain how MYCIN combined certainty factors, and why rule engines keep rules outside the code.

Practise next: Test Bank T17–T18, M17–M18, M25, M35, O05. *Optional:* Rulebound, Stage 9.

Sources: Russell and Norvig (2021), §1.3.4 and Ch. 7; Davis et al. (1977); Buchanan and Shortliffe (1984); Apache KIE.

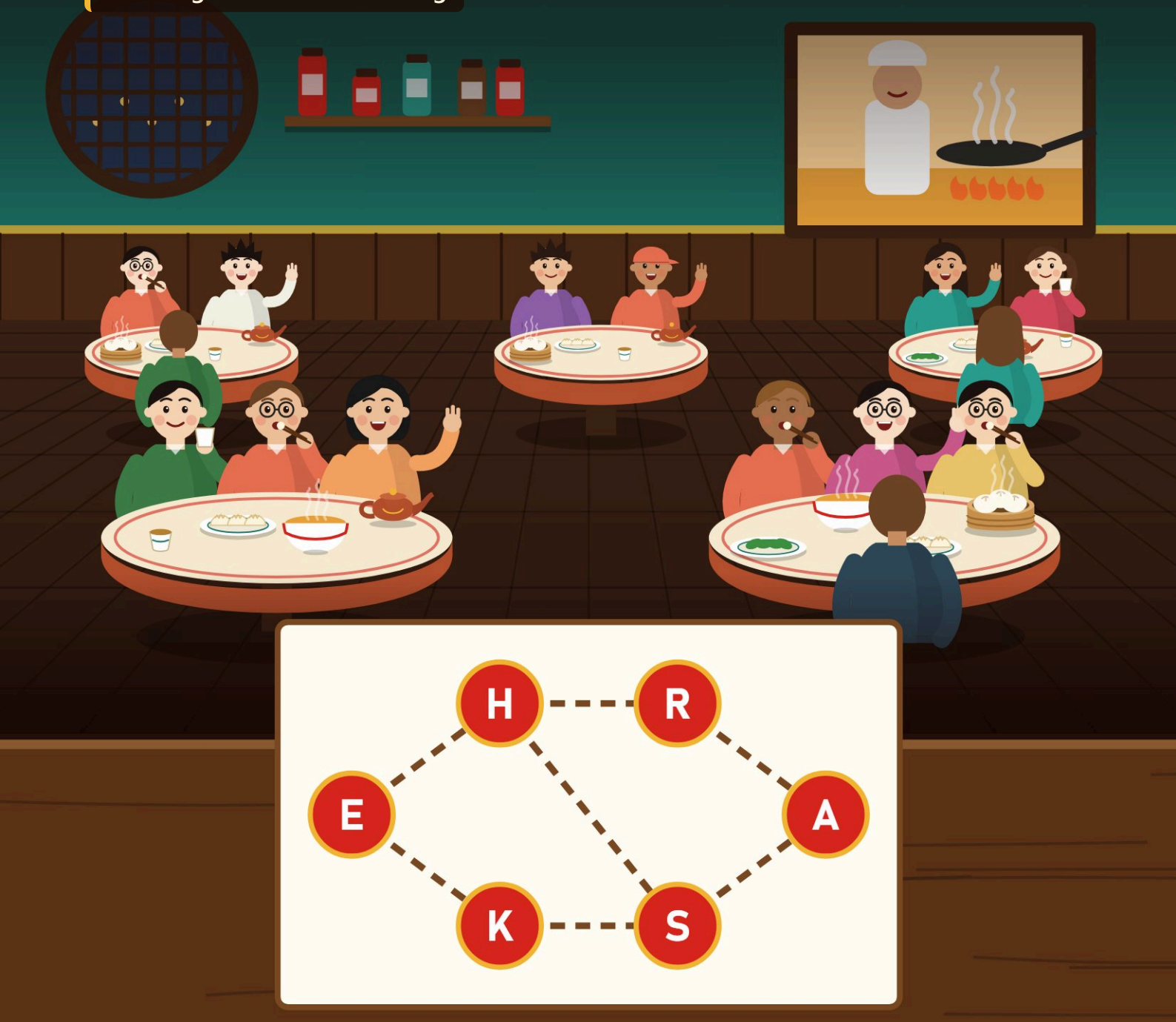


Chapter Ten

10

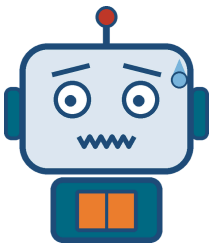
PLANNING AND PROBLEM SOLVING

Planning and Problem Solving



On today's menu: tonight's delivery route

Fewest stops, or shortest walk? Before anyone sets off, someone has to decide what "best" means.



Pip's problem

Pip must carry a book from the Entrance to the Archive. Three routes take exactly three steps, but they differ in distance. Pip counts steps, picks the first one it finds, and walks much further than it needs to.

Your mission

Learn to describe a problem as states and actions, then choose a search method that fits what "best" means.

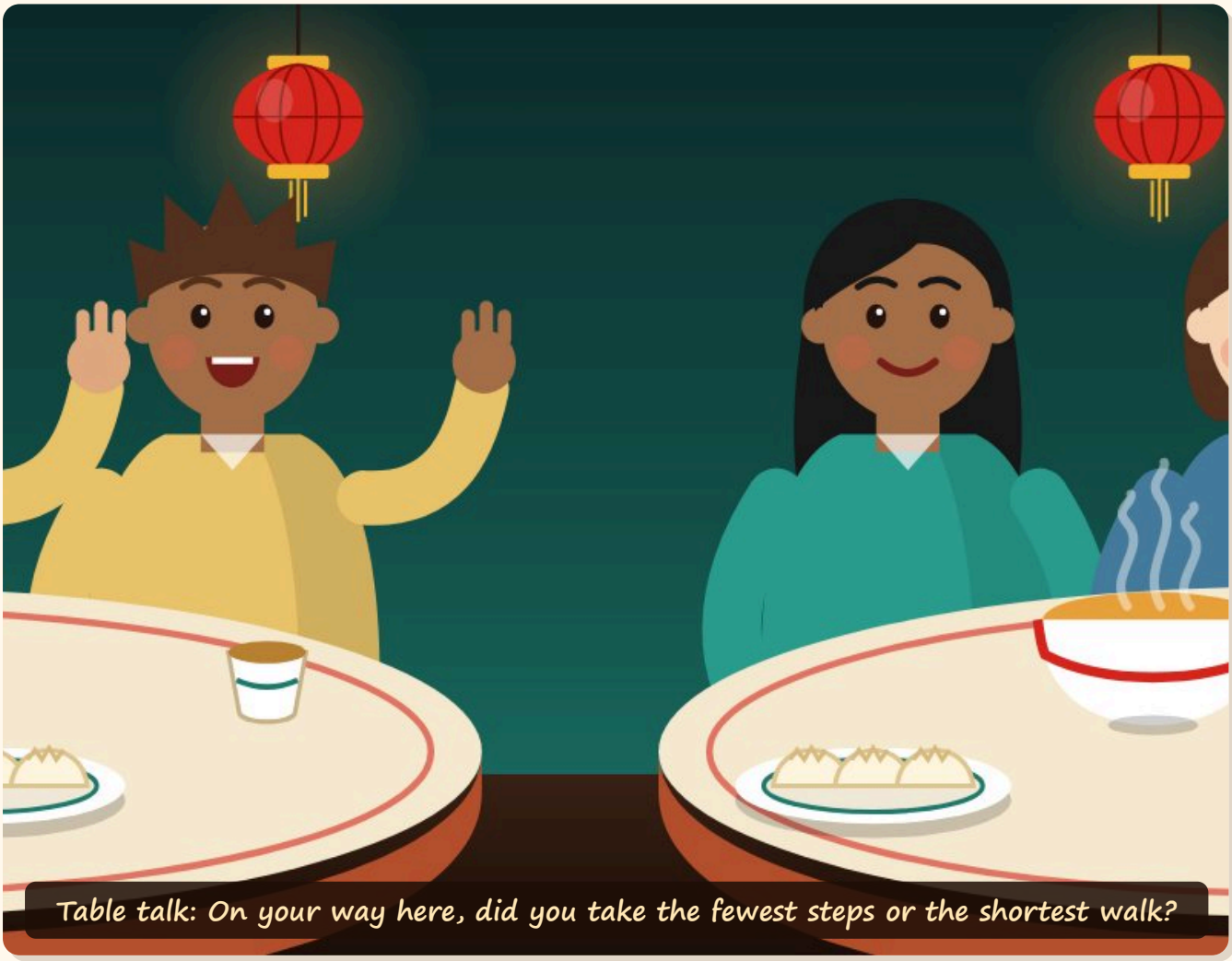
You will be able to

- Describe a problem as a state space, with actions and a goal.
- Trace BFS, DFS, uniform-cost, greedy and A* search on one map.
- Work out minimax and pruning, and explain hill climbing and simulated annealing.

You need: Chapter 4 (queues, stacks, graphs).

Watch: Video 06. While watching, ask: what does this method count as "best"?

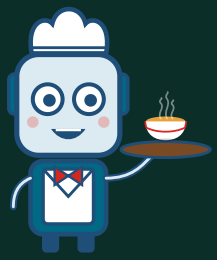
Before any theory, try a very old puzzle. You will use its ideas for the rest of the chapter.



ACTIVITY 10.1

Get everyone across the river

Goal	Discover states, legal actions, constraints and a goal by solving a problem yourself.
Preparation	Four small paper cards: Farmer, Wolf, Goat, Cabbage. Draw a river across a page. A farmer must take all three across. The boat holds the farmer and at most one other. Left alone without the farmer, the wolf eats the goat, and the goat eats the cabbage.
Task	1. Predict: which first move looks tempting? Which is safe? 2. Move the cards to find a plan that gets everyone across. 3. Write each situation as a state, for example "Left: Wolf, Cabbage. Right: Farmer, Goat." 4. Write down one move that fails, and which rule it breaks.
Observe	Record your plan as a list of crossings, and count them.
Reflect	Which part of the puzzle is the state, which is an action, and which is the goal? Which tempting moves fail, and why?
Hint	What must be true of what you leave behind on each bank? Nothing says that the boat must return empty.
Feedback	A valid plan takes seven crossings: take the goat; return alone; take the wolf; bring the goat back; take the cabbage; return alone; take the goat. (You can swap the wolf and the cabbage.) Taking the wolf or the cabbage first fails, because it leaves the goat with the cabbage, or the wolf with the goat. Each arrangement of the cards is a state, each crossing is an action, and "everyone on the far bank" is the goal state. The surprising step is bringing the goat back: search sometimes has to step away from the goal.



Order up! Check your answers before the next dish.

Describing a search problem

A search problem can be represented with:

- **State:** the current situation.
- **Action:** a permitted move to another situation.
- **Goal state:** the situation we want to reach.
- **State space:** the possible states and transitions.
- **Path cost:** the total cost of the actions taken.

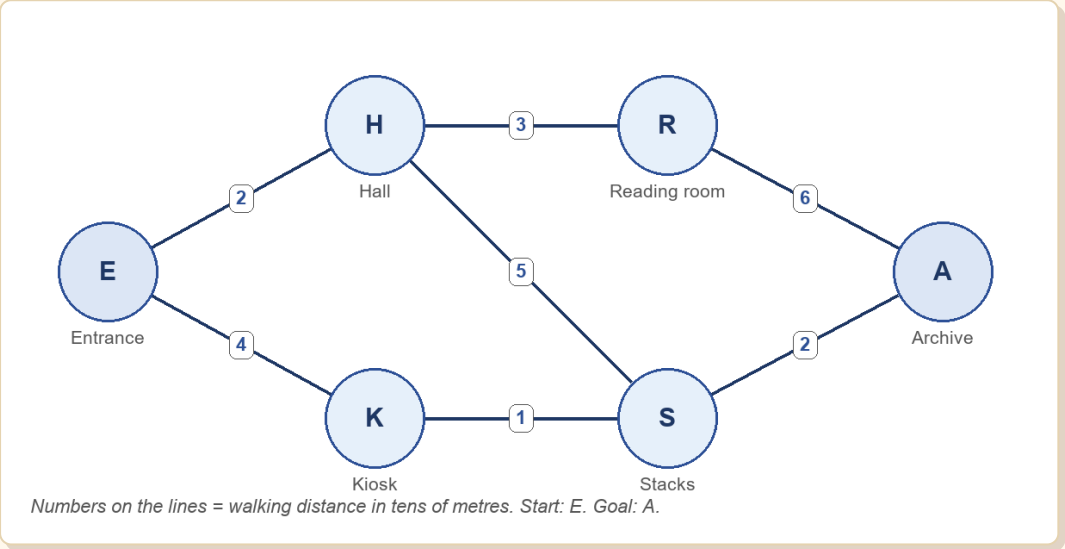


Figure 10.1. The library map. Numbers show walking distance. Start: E. Goal: A.

Route	Calculation	Total
E → H → R → A	2 + 3 + 6	11
E → H → S → A	2 + 5 + 2	9
E → K → S → A	4 + 1 + 2	7 (cheapest)
E → K → S → H → R → A (longer)	4 + 1 + 5 + 3 + 6	19

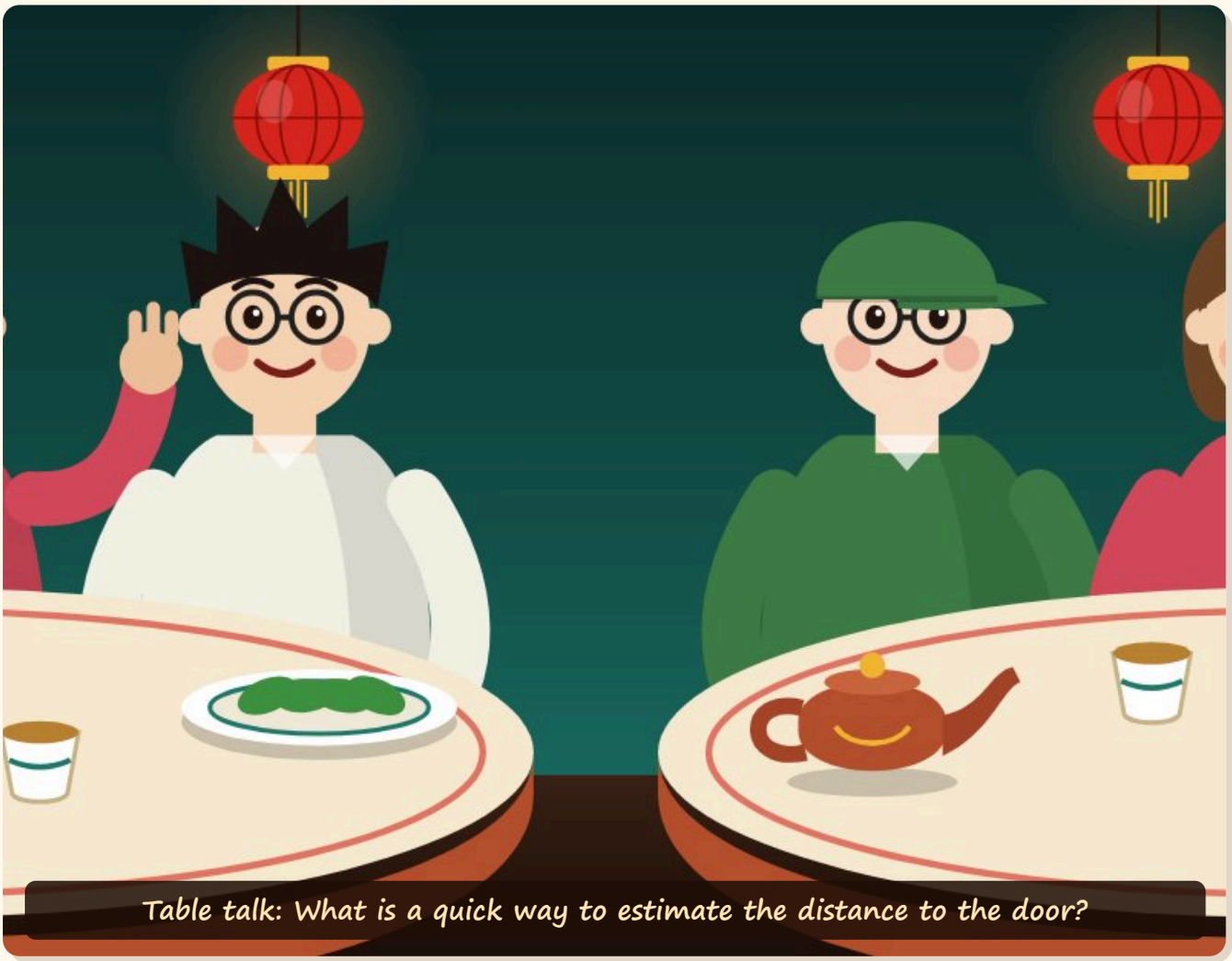
How search methods choose what to explore next



Method	How the next location is chosen	What it promises
Breadth-first search (BFS)	One layer at a time, using a queue.	Always the fewest edges, whatever the costs. No promise of the cheapest route.
Depth-first search (DFS)	Follows one path deeply, commonly using a stack.	May use less memory, but does not guarantee the shortest or cheapest route.
Uniform-cost search (UCS)	The path with the lowest cost so far, g.	With positive edge costs, the cheapest path.
Greedy best-first search	The heuristic estimate h only.	Can be fast, but does not guarantee the cheapest route.
A*	f = g + h, where g is cost so far and h is estimated remaining cost.	With an admissible heuristic, which does not overestimate, a cheapest path.

A **heuristic** estimates the remaining cost to a goal. This one is **admissible** because no estimate is greater than the true cheapest remaining cost:

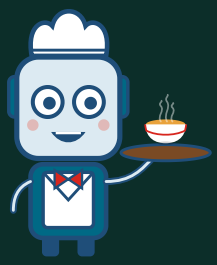
Room	E	H	K	R	S	A
h (estimate to A)	6	5	3	4	2	0
True cheapest cost to A	7	7	3	6	2	0



ACTIVITY 10.2

Plan a route

Goal	Compare route-finding methods and the role of a heuristic.
Preparation	Use Figure 10.1 and the heuristic table. Start at E and search for A. Add neighbours in alphabetical order.
Task	1. Predict which route BFS finds. 2. Compare that route with the cheapest route. 3. Explain which route UCS finds. 4. For a state with g = 5 and h = 2, calculate f for A*. 5. Explain why greedy search and A* may explore different routes.
Observe	Record the route, cost, and search method for each result.
Reflect	Which method finds the fewest steps? Which finds the lowest cost? How does a heuristic affect A*?
Hint	For UCS, keep a list of rooms with their costs so far, and always take the cheapest. If you find a cheaper way to a room, cross out the old cost.
Feedback	BFS finds E → H → R → A, a three-step route with cost 11. All three short routes use three edges, so BFS returns the first one it reaches: it compares depth, not cost. UCS finds E → K → S → A, the cheapest route at cost 7; along the way it reaches S at cost 7 through H, then finds a cheaper way at cost 5 through K. For A*, f = g + h = 5 + 2 = 7. Greedy search uses h alone; A* uses both g and h. On this map, both find the cost-7 route, greedy exploring four rooms, A* five, and UCS six.



Order up! Check your answers before the next dish.

Check your traces

Compare your work with these traces of the same map. The frontier is the list of rooms found but not yet explored. Ties break alphabetically, and the search ends only when the goal is taken from the frontier.

Uniform-cost search: always take the lowest cost so far, g.

Step	Take (cost g)	New or improved costs	Frontier after
1	E (0)	H = 2, K = 4	H 2, K 4
2	H (2)	R = 5, S = 7	K 4, R 5, S 7
3	K (4)	S = 4 + 1 = **5** (better than 7)	R 5, S 5
4	R (5)	A = 5 + 6 = 11	S 5, A 11
5	S (5)	A = 5 + 2 = **7** (better than 11)	A 7
6	A (7): goal		

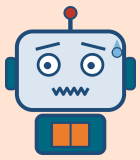
Greedy best-first search: always take the lowest estimate, h.

Step	Take (h)	Add	Frontier after
1	E (6)	H (5), K (3)	K 3, H 5
2	K (3)	S (2)	S 2, H 5
3	S (2)	A (0)	A 0, H 5
4	A: goal		

A*: always take the lowest f = g + h.

Step	Take (f = g + h)	New or improved	Frontier after
1	E (0 + 6 = 6)	H: 2 + 5 = 7; K: 4 + 3 = 7	H 7, K 7
2	H (7)	R: 5 + 4 = 9; S: 7 + 2 = 9	K 7, R 9, S 9
3	K (7)	S: 5 + 2 = **7** (better)	S 7, R 9
4	S (7)	A: 7 + 0 = 7	A 7, R 9
5	A (7): goal		

Method	Lowest value of	Route found	Cost	Rooms explored	Guarantee
UCS	g	E-K-S-A	7	6	Cheapest route
Greedy	h	E-K-S-A	7	4	No promise of the cheapest route
A*	f = g + h	E-K-S-A	7	5	Cheapest route, with an admissible heuristic



Common mistake: "BFS found it, so it's the best route!"

Pip's first walk to the Archive went E → H → R → A. BFS promised the fewest steps, and it delivered: three steps, cost 11. The cheapest route also takes three steps and costs only 7. Before you trust a search, ask what it counts as "best".

Try it 10.3 — When an estimate misleads

Change one estimate: h(H) = 1 instead of 5. Is the heuristic still admissible? Which route does greedy search find now, and at what cost? What does A* find?

Check It is still admissible, because 1 does not exceed H's true cost of 7. Greedy search goes E → H → S → A, cost 9, because H now looks closest. A* still finds E → K → S → A, cost 7, because it also counts the distance already walked.

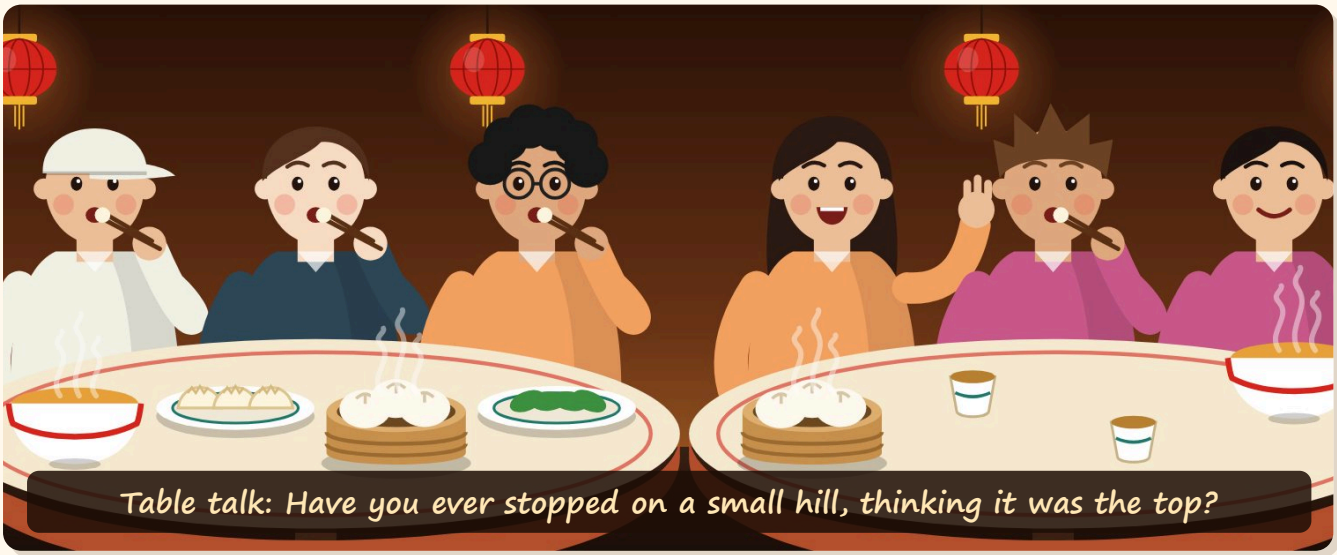
Try it 10.4 — Trace DFS and A*

(a) Trace depth-first search from E with a stack, pushing neighbours so that the alphabetically first one is on top. Which route does it find, and at what cost? (b) With the original estimates, work out f for H and for K after leaving E. Which is explored first if ties go alphabetically? When K is explored, what is f for S?

Check (a) Take E, push K and H. Take H, push S and R. Take R, push A. Take A: the route is E → H → R → A, cost 11. (b) H: 2 + 5 = 7. K: 4 + 3 = 7. It is a tie, so H goes first. Through K, S has f = 5 + 2 = 7, better than the 9 it had through H, and A* goes on to find the cost-7 route.

Games: minimax and alpha–beta pruning

Planning also includes problems where an opponent responds to your choices. In a game tree, MAX chooses the highest score and MIN chooses the lowest score. Left → MIN chooses between 3 and 5, so Left has value 3. Right → MIN chooses between 2 and 9, so Right has value 2. MAX chooses Left because 3 is higher than 2. Choosing Right in the hope of 9 would be a mistake: a sensible opponent would answer with 2.



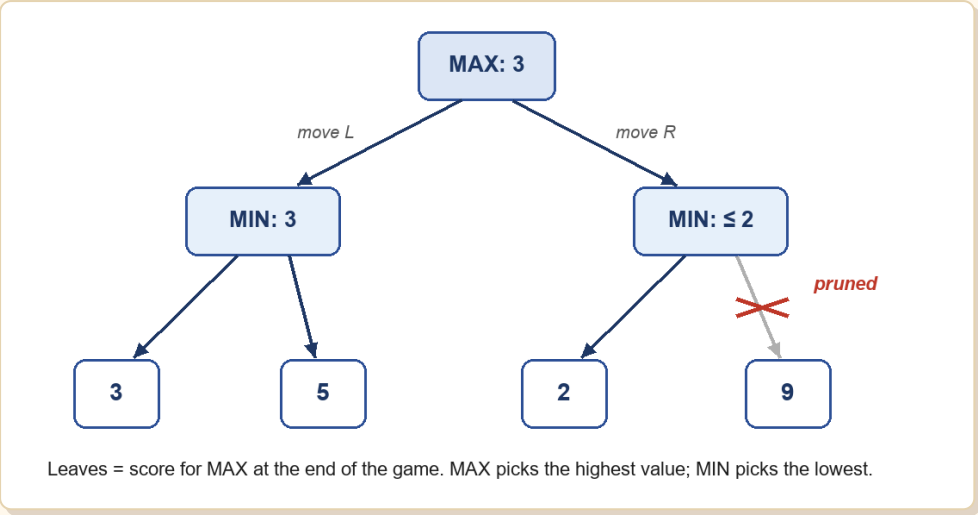


Figure 10.2. A two-move game tree with minimax values. The crossed branch is skipped by alpha–beta pruning.

Alpha–beta pruning skips branches that cannot change the final choice. After MAX has a choice worth 3, if MIN's branch on the right finds a value of 2, the remaining leaf 9 cannot make that branch better for MAX, so it can be skipped.

The name comes from two numbers it remembers. **Alpha** is the best score MAX is already sure of. **Beta** is the lowest score MIN can already force in the branch being explored. As soon as beta is less than or equal to alpha, the rest of that branch can be skipped. In Figure 10.2: after L, alpha = 3. Exploring R, the first leaf is 2, so beta = 2. Beta is less than alpha, so the leaf 9 does not need to be looked at.

Try it 10.5 — Play the game tree

After L, MIN can choose 6 or 4. After R, MIN can choose 7 or 1, in that order. Which move should MAX play? Can pruning skip the leaf 1?

Check L: MIN holds L to 4 and R to 1, and MAX picks the higher. Pruning cannot skip the leaf 1: after the leaf 7, R could still be worth more than 4, so MAX must look at the next leaf. Pruning saves work only when the numbers allow it.

Local search

Other problems aim to improve a starting arrangement, such as a timetable or a shelf layout, and do not need a route at all. **Hill climbing** repeatedly moves to a neighbouring solution that looks better. It can stop at a **local maximum**, where no neighbour is better even though a better solution exists elsewhere. Imagine a landscape with these quality scores:

Position	1	2	3	4	5	6	7	8
Quality	2	4	6	5	3	7	9	8

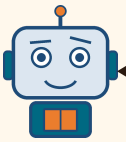
Starting at position 1, hill climbing goes 1 → 2 → 3 and stops at quality 6 because both neighbours of position 3 are lower. The best solution is position 7 with quality 9. **Simulated annealing** sometimes permits a worse move early in the search, which may help it escape a smaller hill. As the search progresses, it becomes less likely to accept worse moves.

In this landscape, it might go 3 (6) → 4 (5) → 5 (3) while the "temperature" is high, then climb 6 (7) → 7 (9), and stay on the higher hill as it cools. It is not guaranteed to find the very best solution, but it usually does much better than plain hill climbing.

Try it 10.6 — Where you start matters

Using the landscape above, run hill climbing from position 4, and then from position 5. Always move to the better neighbour. Where does each one stop?

Check From position 4 (quality 5), the better neighbour is position 3 (quality 6), and it stops there, on the small hill. From position 5 (quality 3), it goes to 6 (quality 7) and then 7 (quality 9), the best solution. Hill climbing's answer depends on where it starts.

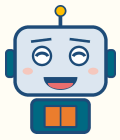


Pip says "I started by the door and got stuck on a small hill. The view was fine. I told everyone it was the best in the building."



Mission complete

Chapter 10: your bill



Takeaway: Model the state, actions, and goal first, then choose a search strategy that fits what "best" means.

Pip's log: Pip now asks "fewest steps or shortest walk?" before setting off, and takes E → K → S → A.

Before you move on, can you:

☐ Describe a problem as a state space, with actions and a goal.

☐ Trace BFS, DFS, uniform-cost, greedy and A* search on one map.

☐ Work out minimax and pruning, and explain hill climbing and simulated annealing.

Practise next: Test Bank T19–T20, M19–M20, M26, M29–M30, O02. *Optional:* Rulebound, Stage 10.

Sources: chap4 slides 33–40; Russell and Norvig (2021), Ch. 3, 4, 6 and 11.



Chapter Eleven

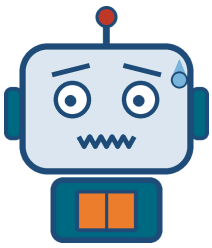
11 SYMBOLIC LANGUAGE PROCESSING

Symbolic Natural Language Processing



On today's menu: skewers and table talk

"Good, the soup is cold again." Rules can read every word of that sentence. Can they read the sigh?



Pip's problem

A visitor sighs: "Good, the printer is broken again." Pip's word list spots *good* and beams: "I am so glad you are happy!" Can written rules really handle human language?

Your mission

See how rules take a sentence apart and find its meaning, and find out where those rules break.

You will be able to

- Follow a sentence through a language pipeline, and draw its parse tree.
- Give an example and a limit for sentiment analysis, question answering and information retrieval.

You need: Chapters 7 and 8.

Watch: Video 07. While watching, ask: what do the rules get right, and what do they miss?



Predict first

Pip adds 1 for each positive word. What score should "The books were not good" get? What about "Good, the printer is broken again"? Write down what a person would say. Try it 11.4 lets you check.

Natural language processing (NLP) is the processing of human language by computers. A symbolic approach uses explicit rules and structured representations. A simple symbolic pipeline can include:

1. **Tokenization:** separate a sentence into words or tokens.
2. **Part-of-speech tagging:** label words such as nouns, verbs, and determiners.
3. **Parsing:** apply grammar rules to find the sentence's structure.
4. **Semantic representation:** represent some of the sentence's meaning.
5. **Inference or retrieval:** use represented knowledge to answer a question or find related information.

Follow one sentence through the pipeline. Syntax is about structure, and semantics is about meaning.



Step	"The cat chased the mouse"
Tokenization	the / cat / chased / the / mouse
Part-of-speech tagging	the (determiner), cat (noun), chased (verb), the (determiner), mouse (noun)
Parsing	A parse tree: see Figure 11.1 below
Semantic representation	`Chased(cat, mouse)`. "The mouse was chased by the cat" has different words in a different order, but the same meaning, so it gets the same representation.
Inference or retrieval	If an ontology says a cat is-a animal and a mouse is-a animal, the system can answer "Did an animal chase another animal?" with yes.

A **grammar** describes permitted structures. A **parse tree** shows how words combine into phrases. Example grammar:

```
Sentence    → Noun Phrase + Verb Phrase
Noun Phrase → Determiner + Noun
Verb Phrase → Verb + Noun Phrase
Determiner  → the
Noun        → cat | mouse
Verb        → chased
```

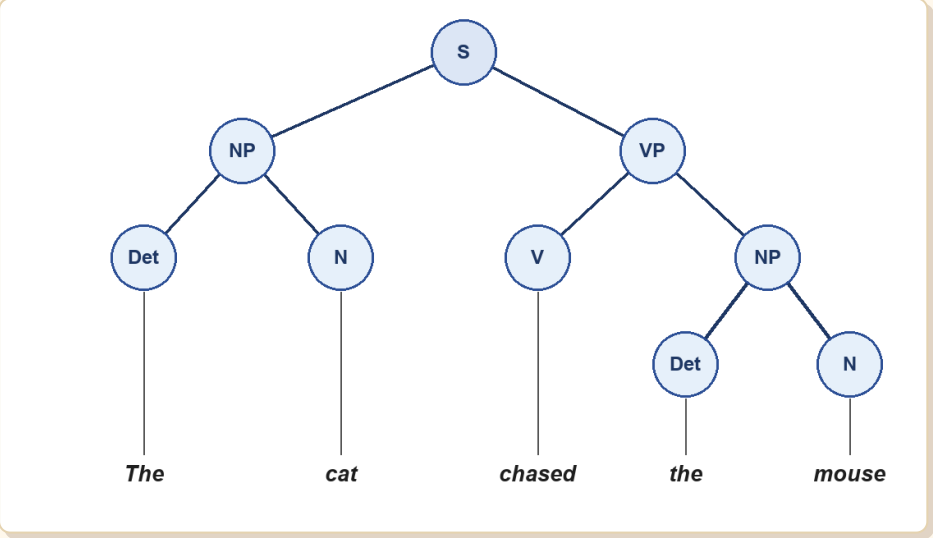


Figure 11.1. The parse tree for "The cat chased the mouse". S = sentence, NP = noun phrase, VP = verb phrase, Det = determiner, N = noun, V = verb.

A small grammar may reject sentences that people can still understand because its rules cover only a limited pattern. A semantic network might represent cat —chased→ mouse. An ontology can connect cat and mouse to the category animal, allowing a system to answer a question about animals if the relevant relationships are represented.

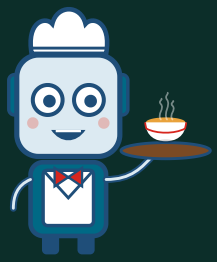




ACTIVITY 11.1

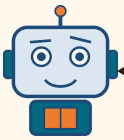
Build a parse tree

Goal	Apply grammar rules and identify where the grammar is limited.
Preparation	Use the grammar above and the sentence “The dog chased the cat.” Ignore capital letters. Before you start, predict which word the grammar will fail on.
Task	1. Label each word as a determiner, noun, or verb. 2. Add the word rule the grammar is missing. 3. Group the words into two noun phrases and one verb phrase. 4. Draw the parse tree. 5. Try “The cat chased the mouse.”
Observe	Check that every group in your tree follows one of the grammar rules.
Reflect	What sentence might this grammar reject? Does rejection prove that the sentence is meaningless?
Feedback	The grammar needs one new word rule: Noun → dog. The tree has the same shape as Figure 11.1, with *dog* in place of *cat* and *cat* in place of *mouse*. Both sample sentences then fit. A sentence with a different word order may be rejected by this small grammar; that shows the grammar’s limits, not necessarily that a human cannot understand the sentence.



Order up! Check your answers before the next dish.





Pip says *"I parsed 'The mouse chased the cat.' It fits the grammar perfectly. The cat has complaints."*

What rules can do with language

Application	Example of symbolic processing	Limit to remember
Sentiment analysis	Word lists score good as positive and bad as negative; not or never directly before a sentiment word reverses it. "The service is not helpful" scores as negative.	A simple word list may miss sarcasm, context, or a word it does not know.
Question answering	Turn "Which books by Rowling are available?" into a query over stored facts, and return only matching known facts.	The system can answer only from the knowledge it has been given.
Information retrieval	Use an ontology to connect car and automobile; use stemming to normalize word forms.	An incomplete ontology misses terms. Removing a grammar word can also break parsing.

Search tools may remove common words, a process called **stop-word removal**. They may also use **stemming** or **lemmatization** to normalize terms. Stemming applies rough rules; lemmatization aims for a dictionary form. These methods can help document retrieval, but a parser needs words such as "the" when its grammar expects a determiner.

Try it 11.3 — What can go wrong?

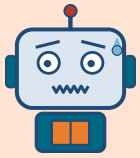
A grammar expects "The cat chased the mouse", but a search-cleaning step removes every occurrence of *the*. What can go wrong? Name one other limitation of a symbolic NLP application.

Check The parser may fail because its grammar requires a determiner. Other valid limitations include sarcasm confusing sentiment rules, a knowledge base lacking the requested fact, or an ontology missing a synonym.

Try it 11.4 — Score it like Pip

Positive words: good, excellent, happy, helpful. Negative words: bad, poor, disappointed, slow. "Not" or "never" directly before a sentiment word reverses it. Add up the score: above 0 is positive, below 0 is negative, 0 is neutral. Score these: (a) "The staff were helpful and the room was excellent." (b) "The books were not good." (c) "The printer is good but the Wi-Fi is slow." (d) "Good, the printer is broken again." Which score is wrong, and why?

Check (a) +2, positive. (b) −1, negative. (c) 0, neutral. (d) +1, positive, because *good* is on the list and *broken* is not. A person hears sarcasm and a complaint, so (d) is the wrong one. This is exactly the mistake Pip made at the start of the chapter.



Common mistake: "It parsed, so the program understood it."

Pip parsed "The cat chased the mouse" and told a visitor that it knew all about cats. A parse tree shows structure, not understanding. A system knows only the meanings, facts and relationships that someone has represented.

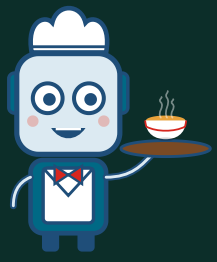




ACTIVITY 11.2

Be ELIZA, then meet her

Goal	See how pattern-matching rules can seem to understand, and investigate where the conversation breaks down.
Preparation	ELIZA is a famous program written by Joseph Weizenbaum in the 1960s. It finds patterns in what you type and turns your words into a reply. First be ELIZA on paper, using these three rules, in order. Rule 1: if the sentence contains "I am X", reply "Why do you say you are X?". Rule 2: if it contains "my mother", reply "Tell me more about your family." Rule 3: otherwise, reply "Please go on." Ignore capital letters. Paper is enough. The online part is optional (see the Resource below).
Task	1. With a partner, one person says a sentence and the other applies the rules exactly. Try: "I am tired."; "I am worried about my exam."; "My mother likes cats."; "The printer is broken."; "Are you a robot?" 2. Predict each reply before applying the rules. 3. Find a sentence that makes the rules give a silly reply. 4. Optional: try the same sentences on an online ELIZA, and note the site you used.
Observe	Record each input, the rule used, and the reply. Mark any reply that seemed to understand.
Reflect	Did any reply show understanding, or did it only reuse your words? What does ELIZA demonstrate, and what does it not demonstrate?
Feedback	The replies are: "Why do you say you are tired?"; "Why do you say you are worried about my exam?"; "Tell me more about your family."; "Please go on."; "Please go on." The exam reply shows the problem: the rule copies *my* without changing it, so real ELIZA programs also swap words such as *my* and *your*. A silly reply is easy to find: "I am hungry and I am late" gives "Why do you say you are hungry and I am late?" ELIZA shows how far simple pattern rules can go, and how easily people read understanding into them. It has no knowledge of exams, families or printers. Online versions differ, so your replies there may not match these.



Order up! Check your answers before the next dish.

Resource: Try ELIZA, on the ELIZA historical project site

What is it? Try ELIZA, on the ELIZA historical project site, from ELIZAGEN, curated by Jeff Shrager: elizagen.org/try-eliza.html

Why use it? It lets you talk to a version of the program from Activity 11.2.

Which part? The "Try ELIZA" page.

What to do Type the five sentences from the Activity, then try to make it break.

Watch out This is one reconstruction: other versions can reply differently. It does not show what modern language models do.

If you cannot open it The paper version in Activity 11.2 does the same job.

Resource: Natural Language Processing: Crash Course Computer Science #36

What is it? Natural Language Processing: Crash Course Computer Science #36, from CrashCourse: youtube.com/watch?v=fOvTtapxa9c

Why use it? It shows parse trees and rules on real examples, and mentions ELIZA.

Which part? The whole episode, about 12 minutes. The first half matches this chapter best.

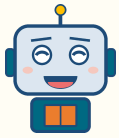
What to do Pause at the first parse tree, and label its noun phrase and verb phrase yourself.

Watch out Later parts go beyond this chapter, into speech and statistical methods. These are not symbolic NLP.

If you cannot open it Team Video 07 and the worked example above cover what you need.

Mission complete

Chapter 11: your bill



Takeaway: A symbolic system's answer depends on the words, grammar, facts, and relationships that have been represented.

Pip's log: Pip now answers "Good, the printer is broken again" with: "I may have misread that. Do you need help with the printer?"

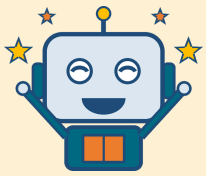
Before you move on, can you:

☐ Follow a sentence through a language pipeline, and draw its parse tree.

☐ Give an example and a limit for sentiment analysis, question answering and information retrieval.

Practise next: Test Bank T21–T22, M21–M22, M27, O06 (after Chapter 12). *Optional:* Rulebound, Stage 11.

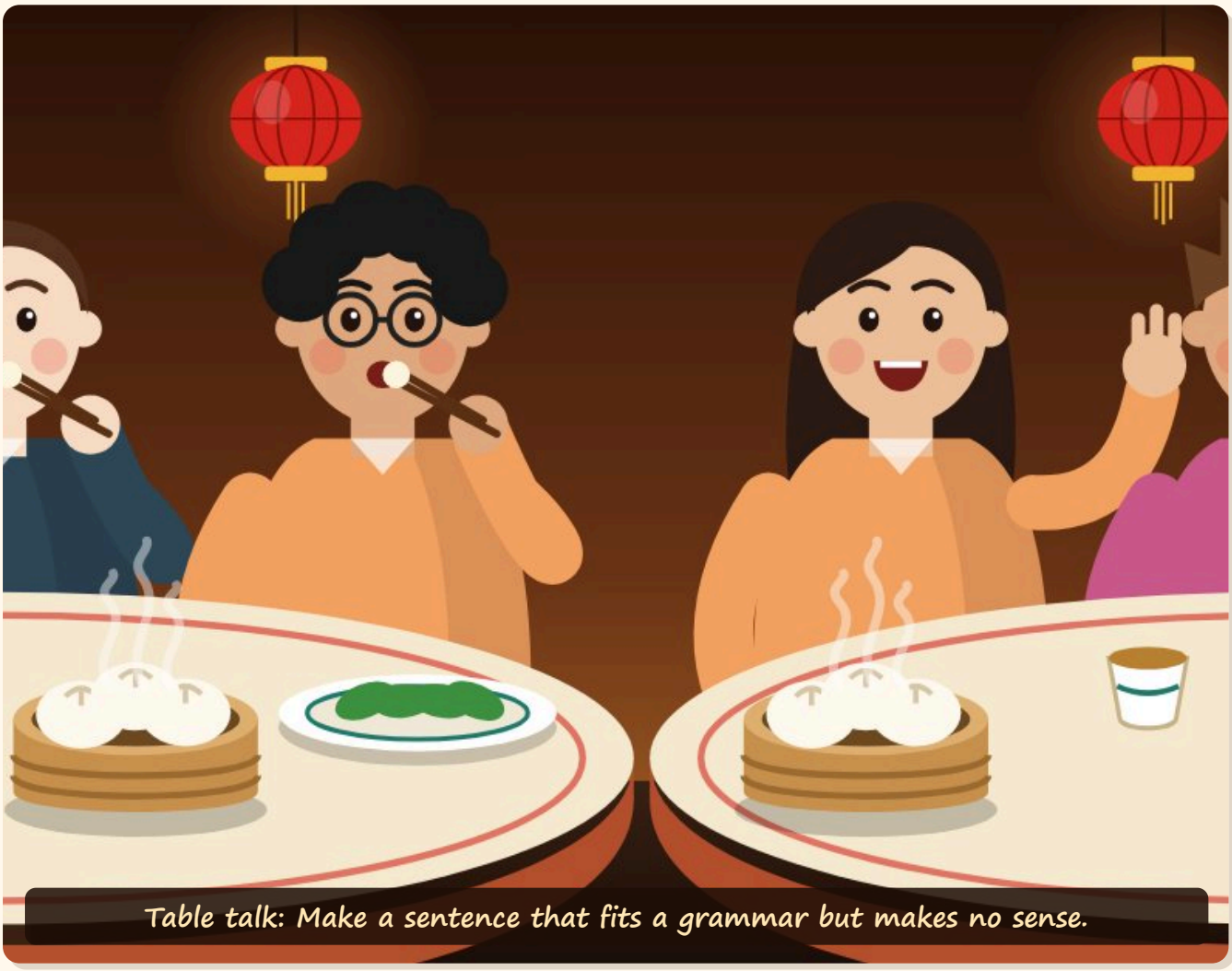
Sources: chap4 slides 41–49; Jurafsky and Martin; Russell and Norvig (2021), Ch. 24; Weizenbaum (1966).



Checkpoint B: from rules to reasoning

You have finished Chapters 6 to 11. (1) Fever is true and the cough box is blank: can a rule that needs both fire? (2) Which inference rule is forward chaining using, again and again? (3) Which form would you use to say that a reference book cannot be borrowed, as an exception? (4) Which search would you trust for the cheapest route, and which for the fewest steps? (5) Why can a parser fail after stop-word removal?

Check (1) No: the cough value is unknown, so the rule cannot fire yet. (2) Modus ponens. (3) A frame that overrides a default. (4) Uniform-cost search or A* for the cheapest; BFS for the fewest steps. (5) The grammar may need a word such as "the" that was removed.



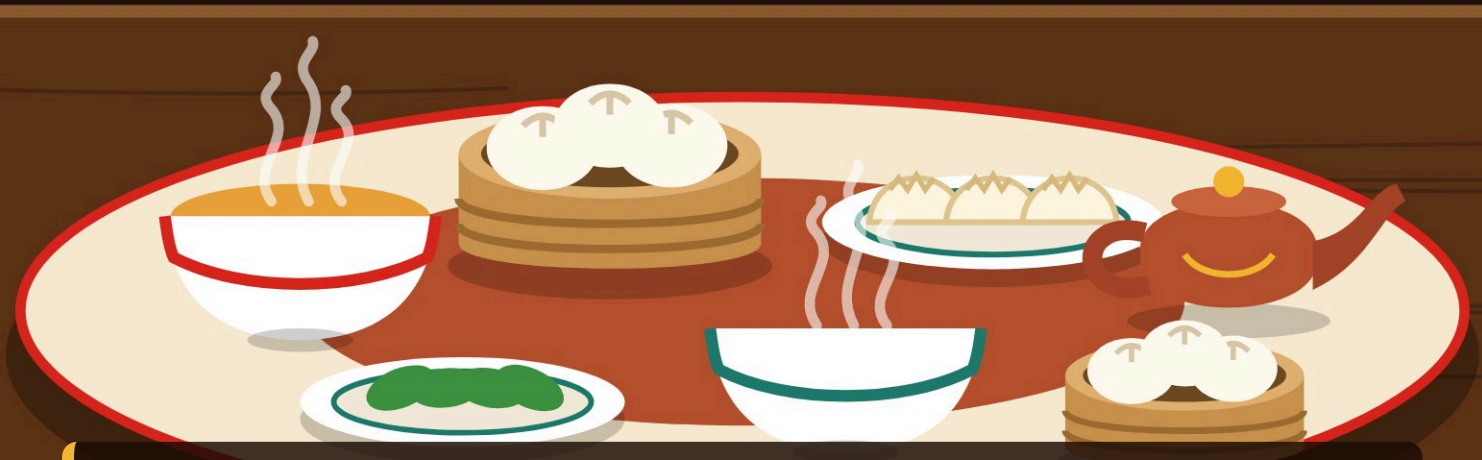
Chapter Twelve

12 SOFTWARE AND SYMBOLIC AI TOGETHER

Software and Symbolic AI Together

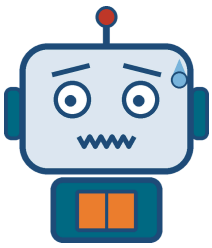






On today's menu: the full banquet

Everything comes to the table at once, with one last question: who do you trust to serve it?



Pip's problem

The library builds two versions of Pip to control the reading-room fan and lamp: Rule-Pip, with written rules, and Model-Pip, trained on example sentences. A visitor types "make me a sandwich." Rule-Pip does nothing. Model-Pip switches on the fan. Which one would you trust?

Your mission

Compare fixed program logic, explicit rules and a trained model, and find out what each one should be trusted with.

You will be able to

- Explain how program logic and a rule-based component work together in a hybrid system.
- Compare a rule-based assistant with a machine-learning assistant, using real results.
- Say how a system should respond to missing, conflicting, outdated or low-confidence information.

You need: Chapters 5 and 9.

Watch: Video 07. While watching, ask: who makes the final decision, the program, the rules or the model?



Predict first

Which version of Pip would you trust with the reading-room fan? Write your answer and one reason now. You will come back to it at the end of this chapter.

Ordinary software and symbolic AI can work together. A program can collect input, store information, and display a result. A symbolic component can represent a policy as explicit rules and infer a conclusion. A system that divides the work this way is called a **hybrid system**: the program logic moves the data, and the rules make the decision.



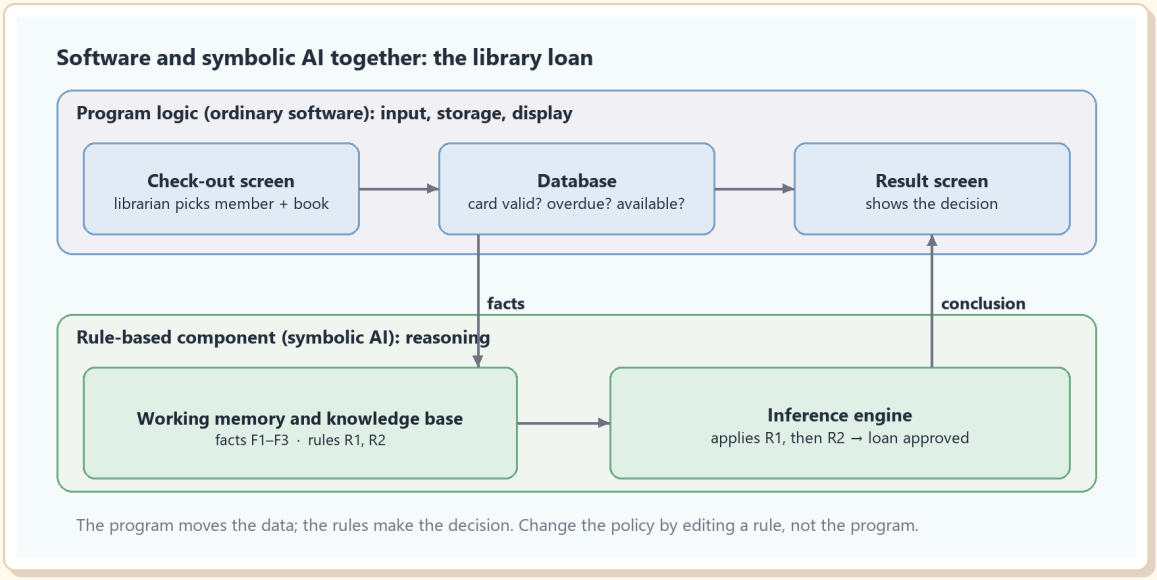


Figure 12.1. Software and symbolic AI together: the library loan.

One decision, two ways

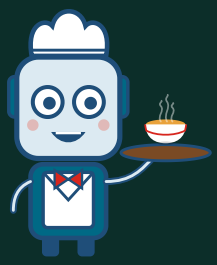
The library decision can be written as a program or as rules. In a program, the policy is encoded in the code's conditions and their order. A final branch can produce an output for cases not covered by earlier branches. In a rule-based system, the policy is explicit and can be easier to inspect or change. If no rule matches, there may be no conclusion. Both approaches need accurate requirements and tests. Neither should treat missing information as proof of the opposite.



ACTIVITY 12.1

Change the library policy

Goal	Compare how a program and rules represent a changed policy.
Preparation	Use the library program from Chapter 5 and rules R1 and R2 from Chapter 9. New policy: a member with overdue books may borrow if they have paid the fine. Before you start, predict: will the change be smaller in the program or in the rules?
Task	1. Describe what must change in the program and its inputs. 2. Write a rule for the changed policy, and call it R5. 3. Create tests for paid and unpaid fines. 4. Trace Ken with R1, R2 and R5: he has a valid card, one overdue book and a paid fine, and the book he wants is available. Which rules fire? 5. Now add R4 from Activity 9.1. What goes wrong for Ken?
Observe	Compare how clearly the new policy appears in the program and in the rule.
Reflect	Which representation is easier for a librarian to inspect? What tests are still needed?
Hint	Write the new policy in one sentence first. Then ask which condition is new, and which old test could now give a different answer.
Feedback	The program needs a `fine_paid` input and an updated condition. A rule could be: IF the member has a valid card AND has overdue books AND has paid the fine THEN the member may borrow. Test the old and new branches. For Ken, R1 does not fire, R5 adds "may borrow", and R2 approves the loan. With R4 as well, the system also adds "refused", so the rules conflict for a member who has paid: see the table below. **Back to your prediction:** the changes are similar in size, and the difference is how easily a librarian can read them. Both versions still need the old and new tests.



Order up! Check your answers before the next dish.

When information is imperfect

Information problem	Why it matters	Good system response
Missing	A blank field does not prove the opposite fact.	Ask for the fact or report that the rule cannot decide.
Conflicting	Rules or facts can support opposite conclusions.	Rewrite rules so they do not clash, or set and document a clear priority or specificity policy.
Outdated	A correct engine can still apply an obsolete rule or stale record.	A responsible person updates the knowledge, tests the change, and runs regression checks.
Low-confidence model output	A predicted label may be wrong even when the model returns one.	Ask for clarification and apply explicit safety rules before taking an action.

Conflict example. Suppose one rule approves a loan when the member may borrow and the book is available, while another refuses it whenever the member has reached the five-book limit. If both conditions can be true at once, the system may add both "approved" and "refused". The team can add the limit condition to the approval rule so the rules cannot clash, or explicitly give refusal priority. The choice should be documented and tested with a member exactly at the limit.

Hands-on 12.2 — Rule-based assistant vs. machine-learning assistant

This activity compares exact rules with a text classifier, using the fan-and-lamp assistant from the professor's PPT 2024, chap4, slide 50. A **text classifier** is a small machine-learning model that reads a sentence and puts it into one of the groups (*labels*) you chose. **Confidence** is the model's own score for the label it chose: high confidence does not prove the label is right, and the model picks a label even when its confidence is very low.

Goal	See what a rule-based program understands and what a trained model understands, and why.
Preparation	A computer with a browser. Scratch and Machine Learning for Kids (see the Resources below). About 50 minutes. In Machine Learning for Kids, choose **Get started**, then **Try it now**: no account is needed, but the site first asks you to pass a "Verify you are human" check, and this mode gives you one project, for a few hours, on one computer. Confirm access before class.
If you cannot use the sites	You can still do this activity. Fill in the predict sheet, read Figures 12.2 and 12.5 as the scripts, and use Figures 12.4 and 12.7 and the observation table as your results. All of the reflection questions can be answered from these.

Predict first. Before you build anything, copy this predict sheet. For the rule-based assistant, write the action or "nothing". For the machine-learning assistant, write the label you expect. Fill in the last two columns as you test.

Command typed	My prediction: rule-based	My prediction: machine-learning	What happened: rule-based	What happened: machine-learning
Fan on				
fan on				
Fan on please				
turn on fan				
I'm really hot				
It's too dark to read				
turn the fan off please				
make me a sandwich				

Part A — Rule-based assistant (15 minutes). Blocks are in the coloured groups on the left of the Scratch editor: **Events** (`when green flag clicked`, `broadcast`, `when I receive`), **Control** (`forever`, `if … then`, `wait`), **Sensing** (`ask … and wait`, `answer`), **Operators** (`=`, `or`) and **Looks** (`switch costume to`, `say … for … seconds`). To create a message, open the menu on a `broadcast` block and choose **New message**. Click a sprite before you drag blocks onto it.

1. Make two sprites, a fan and a lamp, each with an *on* and an *off* costume. A simple drawing is enough.
2. Inside a **forever** loop on the fan sprite, ask for a command with **ask … and wait**.
3. Add exact rules for "Fan on," "Fan on please," "Fan off," "Lamp on," and "Lamp off."
4. Broadcast a message when a rule matches, and give each sprite a script to respond to its message.
5. Test the eight commands. Record what happens.



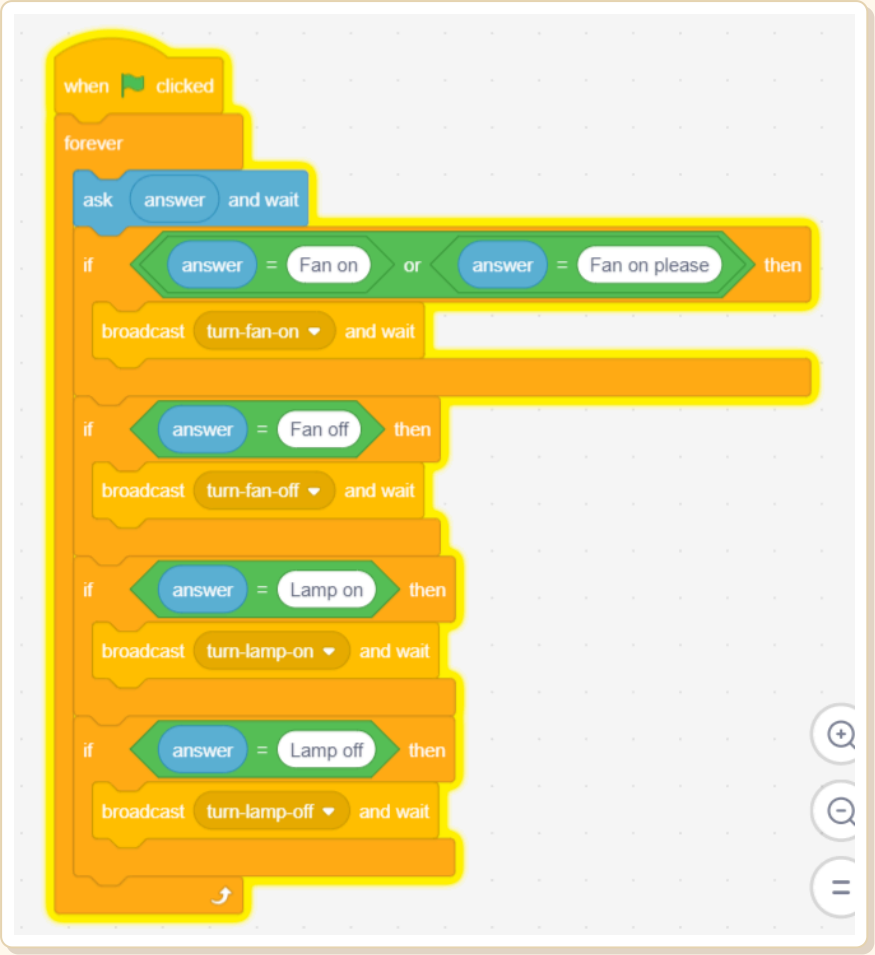
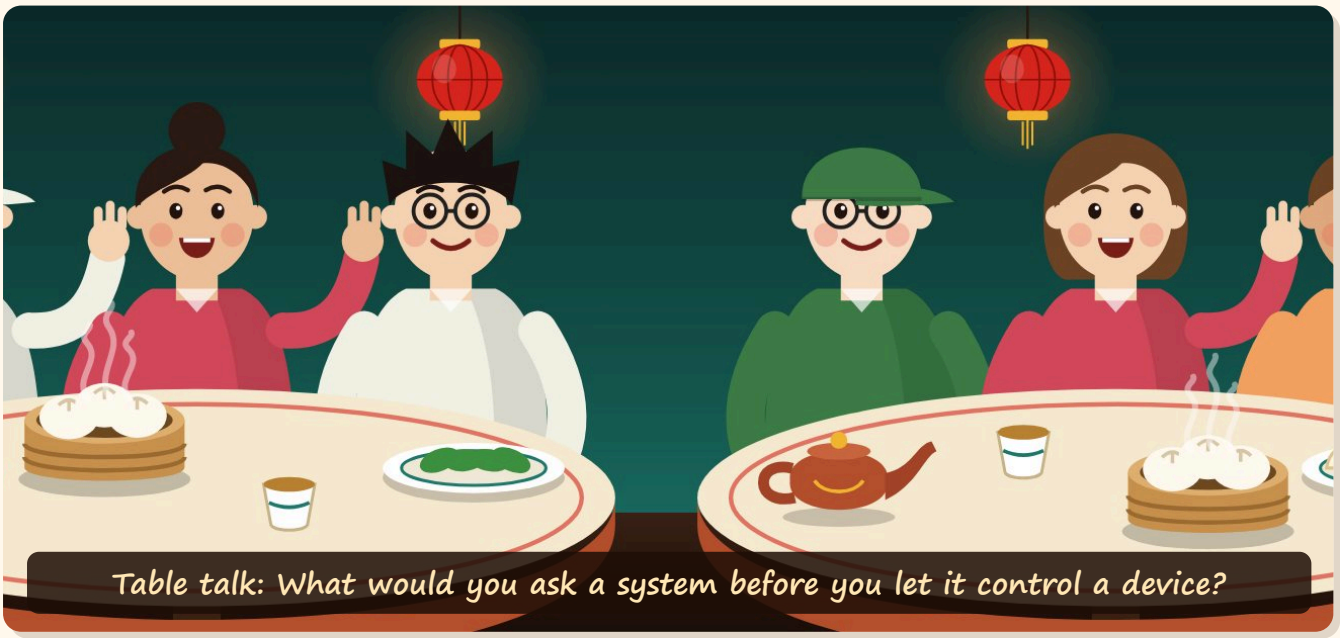


Figure 12.2. The rule-based assistant. Source: the professor's PPT 2024, chap4, slide 50. On the slide, the ask block contains the answer block, so the first question is blank and later questions repeat your last answer; you can type "Enter your command" there instead, as in Figure 12.5.

Part B — Machine-learning assistant (25 minutes).

1. Create a text-recognition project with four labels: `fan_on`, `fan_off`, `lamp_on`, and `lamp_off`.

2. Add at least six varied example sentences to each label, such as "turn on the fan" or "I need some air" for `fan_on`.



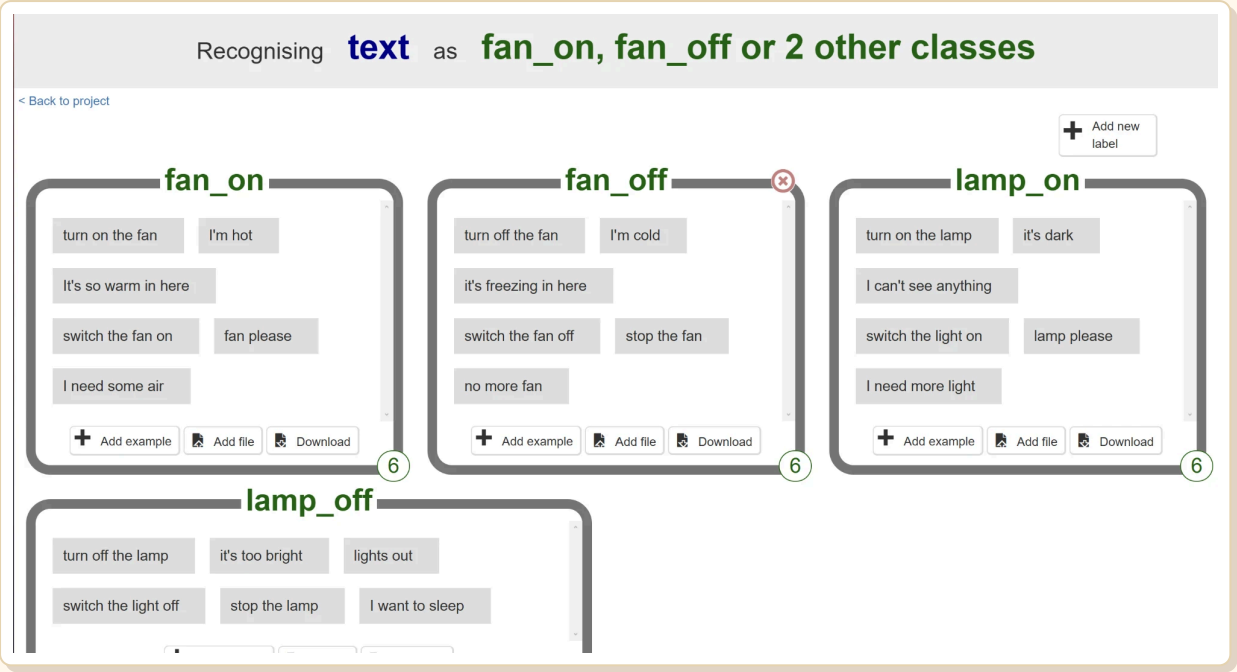


Figure 12.3. The team's training examples: six for each label. Screenshot from Machine Learning for Kids, 4 November 2026.

3. Train the model and test the eight commands on the website, which you did not type as examples. Record each label and its confidence.

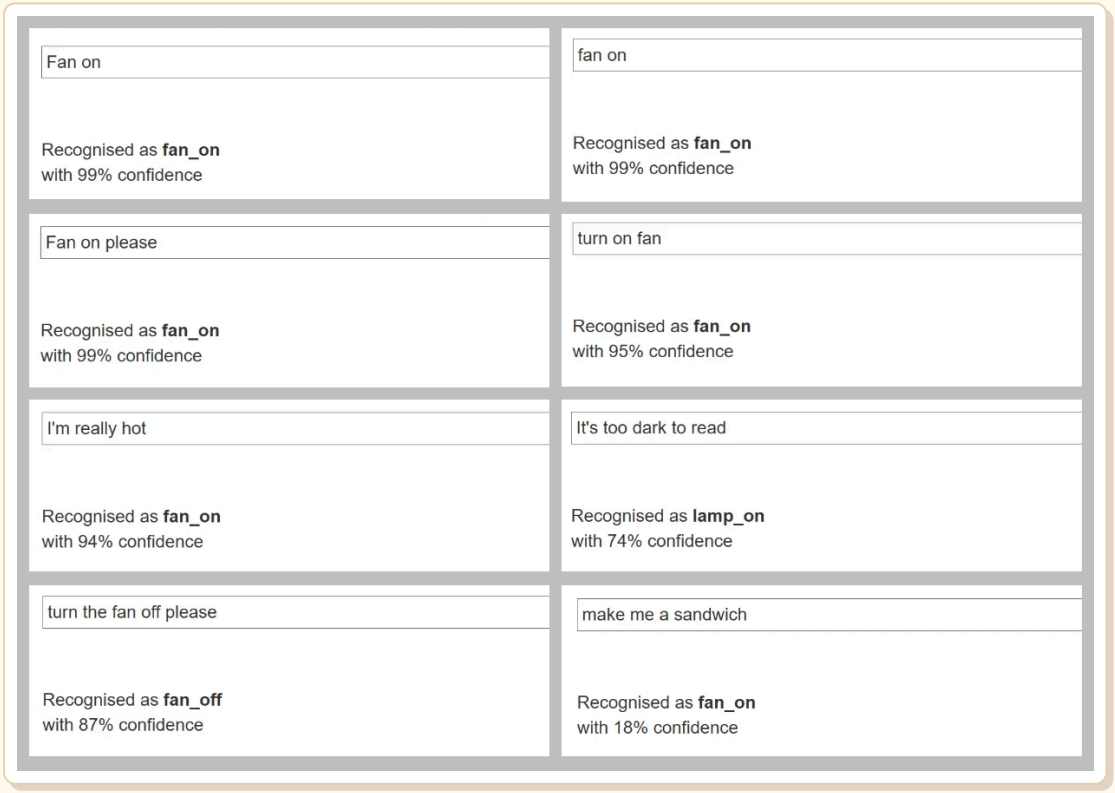


Figure 12.4. The trained model's answers to the eight test commands. Screenshots from Machine Learning for Kids, 4 November 2026.

4. Open the project in Scratch from Machine Learning for Kids. Connect the predicted label to Scratch messages. Figure 12.5 is the slide's script, and Figure 12.6 is the team's working version: without its wait, the next question replaces the reply straight away.

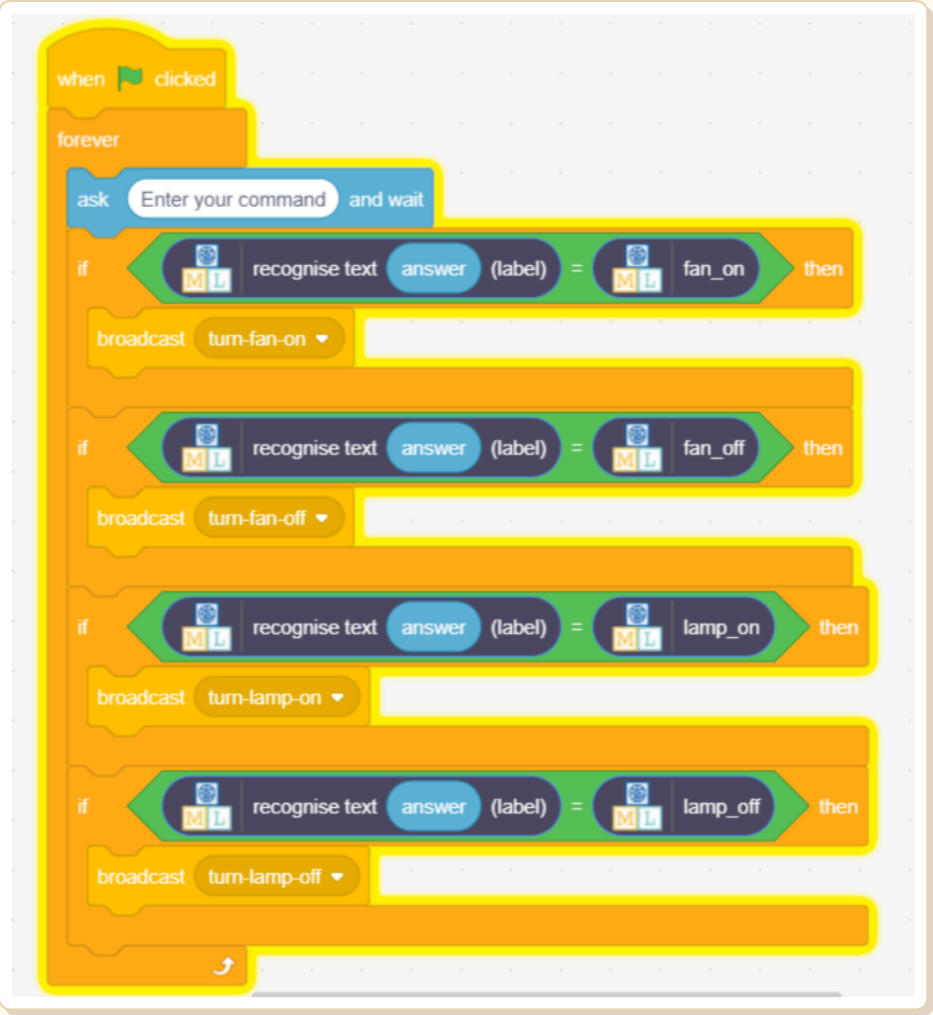
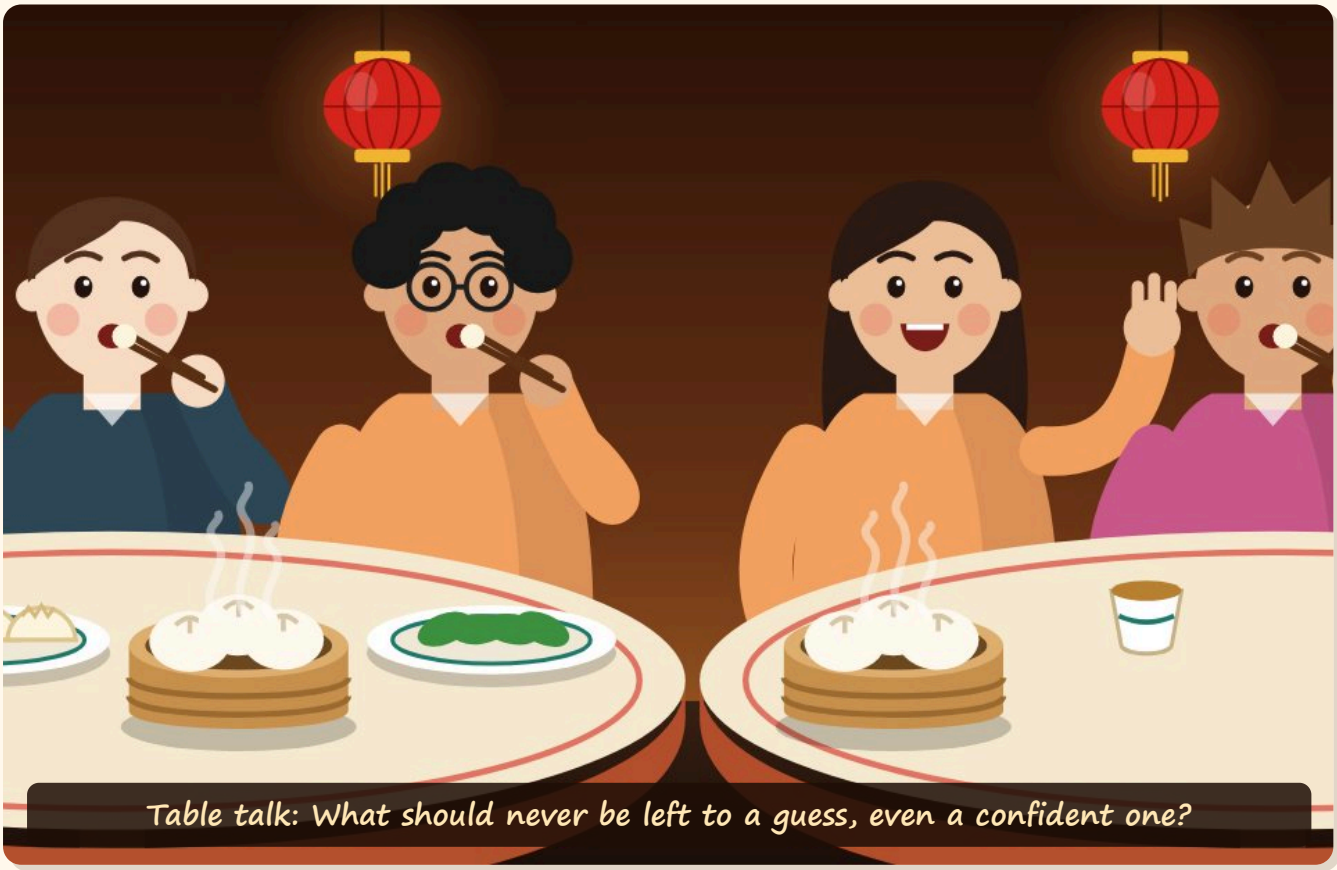


Figure 12.5. The machine-learning assistant. Source: the professor's PPT 2024, chap4, slide 50.



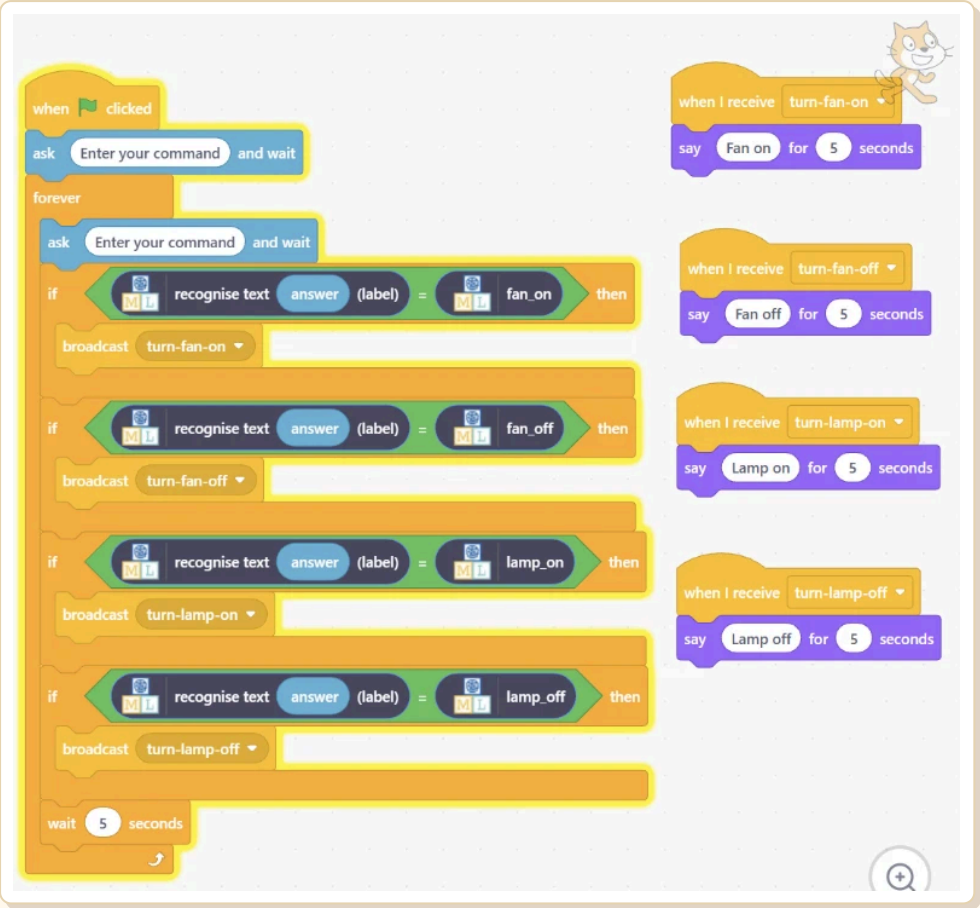


Figure 12.6. The team's working script. It adapts the slide's script: one extra ask before the loop (press Enter the first time), wait 5 seconds at the end of the loop, and four short scripts that show the reply.

5. Click the green flag. Because of the extra ask, the question appears twice at the start: press Enter the first time, then type your command the second time.

6. Test the same commands in Scratch. Scratch shows only the reply, not the confidence.



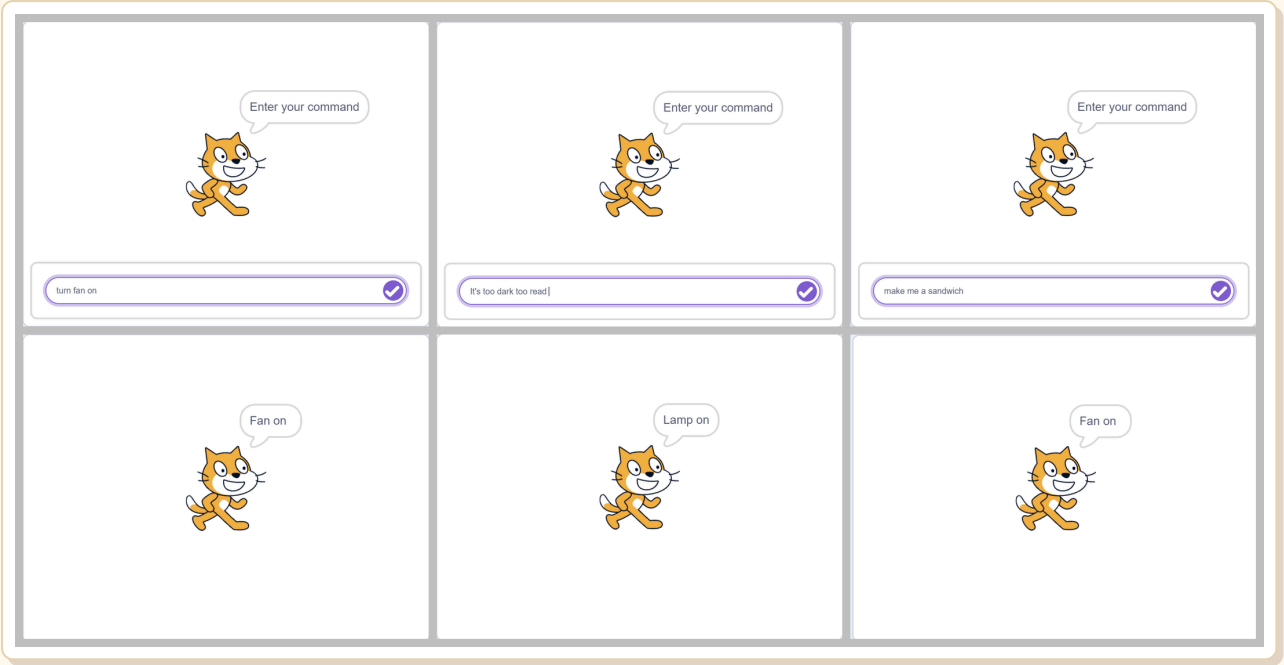


Figure 12.7. Three commands run through the script in Figure 12.6: "turn fan on" gets "Fan on", "It's too dark too read" (a typing slip) gets "Lamp on", and "make me a sandwich" gets "Fan on". The wording differs a little from the table below. Screenshots, 4 November 2026.

Observation table. Compare your predict sheet with the team's results. They come from one model, so your percentages may differ, and sometimes a label may too. Compare the pattern, not the exact numbers.

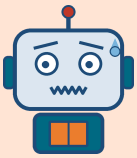
Command typed	Rule-based assistant (tested in Scratch)	Machine-learning assistant (tested on the website)
Fan on	Fan turns on	`fan_on`, 99%
fan on	Fan turns on: Scratch's = block ignores capital letters	`fan_on`, 99%
Fan on please	Fan turns on: there is a rule for exactly this sentence	`fan_on`, 99%
turn on fan	Nothing: not the exact words of any rule	`fan_on`, 95%
I'm really hot	Nothing: no rule matches	`fan_on`, 94%
It's too dark to read	Nothing	`lamp_on`, 74%
turn the fan off please	Nothing	`fan_off`, 87%
make me a sandwich	Nothing	`fan_on`, 18%: it still picks a label, with very low confidence

Part C — Add a safety check (10 minutes, on paper). The script in Figure 12.6 acts on the label alone. Design a rule that stands between the model and the fan.

1. Write the rule: IF confidence is below your threshold, THEN ask the person to repeat the command.
2. Predict: using the team's percentages in the observation table, which commands would be asked again at a threshold of 50%?
3. Try 80% and 96%. What goes wrong when the threshold is too low, and when it is too high?

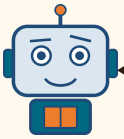
Check Part C

At 50%, only the sandwich (18%) is asked again. At 80%, "It's too dark to read" (74%) is asked again too, although its label was right. At 96%, only the three "fan on" commands (99%) act, and five commands are asked again, four of them with sensible labels. Too low lets wrong labels act, and too high asks about right ones. A threshold also cannot tell whether a label makes sense, so explicit safety rules are still needed.



Common mistake: "The model gave an answer, so it must know."

Model-Pip picks a label for everything, including a sandwich. A label is a guess from examples. Confidence tells you how strong the guess is, not whether it is right, so an unrelated request can still get a label.



Pip says *"Model-Pip heard 'make me a sandwich' and switched on the fan. Rule-Pip heard it and did nothing. For the record, nobody got a sandwich."*

Reflect

- Which assistant handles different wording better?
- Which assistant makes its decision easier to inspect?
- What should happen when the model is uncertain?
- What safeguard should prevent an uncertain prediction from controlling a device?

Feedback

Explicit rules are easy to trace but need rules for the wordings they accept. A trained classifier can generalize to new wording but may assign an unrelated request to a known label. The team's recorded test labelled the sandwich request `fan_on` at 18% confidence, and the script checked only the label, so the cat said "Fan on". Treat this as the team's recorded result, not a guaranteed result for every run. A safer design can ask the user to repeat a low-confidence command and apply explicit safety rules before taking action.



Resource: Scratch video tutorials and starter projects

What is it? Scratch video tutorials and starter projects, from Scratch Foundation: scratch.mit.edu/help/videos

Why use it? New to Scratch? These get you ready to build the fan-and-lamp assistant.

Which part? Start with "Getting Started with Scratch", then choose one short introductory tutorial. Also open a starter project.

What to do Inspect a starter project's scripts, predict what a changed block will do, and then test your prediction.

Watch out The tutorials cover much more than you need here.

If you cannot open it Figures 12.2 and 12.5 show the blocks you need.

Resource: Machine Learning for Kids

What is it? Machine Learning for Kids, from D. Lane: machinelearningforkids.co.uk

Why use it? It lets you train a real text classifier and use it inside Scratch.

Which part? Use the text-classification project in Hands-on 12.2. Project worksheets are also available on the site.

What to do Follow Part B above, then try improving a wrong label by changing your training examples.

Watch out Confirm access and any account requirements before learners begin. A trained label is a guess from examples, not understanding.

If you cannot open it Use Figures 12.3 to 12.7 and the observation table.

Try it 12.3 — What should the program do?

A text model predicts `fan_on` with low confidence for a request unrelated to the fan. What should the program do before controlling the fan?

Check Do not act on the label alone. Ask the person to clarify, check the request against explicit permission and safety rules, and record the prediction, confidence, and decision for later review.

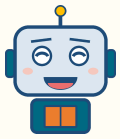
Optional extension — Neuro-symbolic AI (not examined)

Some systems combine neural models with symbolic rules. For example, a model might interpret an image or sentence while explicit rules check safety conditions: the model helps with perception, and the rules decide what is allowed. This is an extension topic, not part of the examined core content, and it is different from what this module calls a hybrid system.



Mission complete

Chapter 12: your bill



Takeaway: Program logic moves the data, and rules make decisions that people can inspect. A trained model can read varied wording, but it can still guess, so a label should not directly control a device when confidence is low or the request is unrelated.

Pip's log: You have helped Pip from its first day to here: from "is a calculator AI?" to knowing what a system may conclude, and when it must ask. Well done, both of you.

Before you move on, can you:

☐ Explain how program logic and a rule-based component work together in a hybrid system.

☐ Compare a rule-based assistant with a machine-learning assistant, using real results.

☐ Say how a system should respond to missing, conflicting, outdated or low-confidence information.

Practise next: Test Bank T23–T24, M23–M24, M28, M36, O03, O06. *Optional:* Rulebound, Stage 12.

Sources: chap4 slide 50; Russell and Norvig (2021), Ch. 7; Lane, Machine Learning for Kids; Scratch Foundation.

Pip's last day as a trainee: bringing the module together

Chapter 6 said what symbolic AI is: knowledge written down as symbols and rules that people can read, check and change. Chapter 10 used that idea for planning: states and actions are written down, so every step of a route can be traced. Chapter 11 used it for language: grammars and word lists handle sentences by rule.

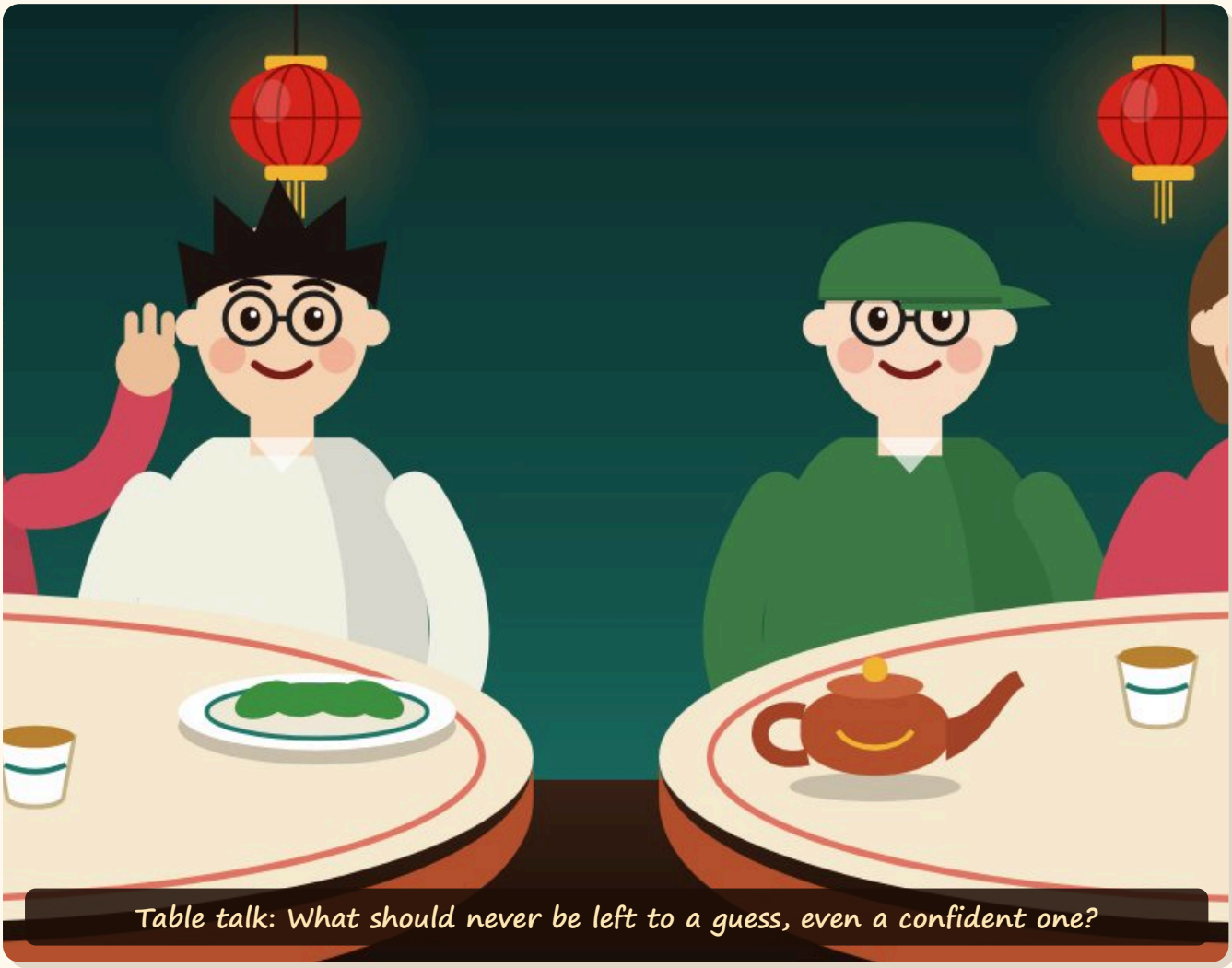
Your rule-based assistant shows the same limit as Chapter 11. "turn on fan" fails for the same reason that a grammar rejects a sentence it has no rule for: the rules are never complete. The trained model copes with new wording, but it cannot show a readable reason, and it picked a label even for a sandwich.

Neither approach is enough for everything. **Program logic moves the data, rules give decisions you can explain and check, and a trained model can help with messy input.** Knowing which job suits which approach is the main lesson of this module.

Back to your prediction. You wrote down which version of Pip you would trust. Would you change your answer now? A good answer depends on the job: Rule-Pip for anything that must be explained or must be safe, and Model-Pip for understanding messy wording, with rules checking it before it acts.

The final trial

The Head Librarian has one more question: can Pip be trusted with the loan desk? Take the Test Bank, **Pip's Library Trials**, as one sitting: 24 true-or-false statements, 38 multiple-choice questions and 6 case files. Its Score Sheet sends you back to any chapter you need.



QUICK REFERENCE GLOSSARY

Term	Meaning
A* search	Search that explores the state with the lowest f = g + h (cost so far plus estimated cost to go); finds the cheapest route when the heuristic never overestimates.
Admissible heuristic	A heuristic that never estimates more than the true remaining cost.
Affirming the consequent	An invalid argument: from P → Q and Q, concluding P.
Algorithm	A clear, ordered procedure for solving a problem.
Alpha–beta pruning	A way of doing minimax that skips branches which cannot change the final choice.
Application software	Software designed to help a user perform tasks.
Array	An ordered collection accessed by position.
Artificial intelligence (AI)	A broad field. In this module, an AI system infers from inputs how to produce outputs such as predictions, recommendations or decisions.
Backward chaining	Starting from a goal and working back to the facts needed to support it.
Base case	The smallest version of a problem in a recursive function, answered directly without another call.
Binary search	Searching a sorted list by checking the middle item and throwing away the half that cannot contain the target.
Breadth-first search (BFS)	Explores a graph layer by layer using a queue.
Certainty factor	A number from −1.0 to +1.0 showing how strongly a rule system supports a conclusion; not a probability.
Closed-world assumption	Treating any fact that cannot be found as false. Safe only when the data is known to be complete.
Cohesion	How well the tasks inside a module belong together.
Compiler	A program that translates a whole high-level program into machine language before it runs.
Confidence	A machine-learning model's own score, shown as a percentage, for how sure it is about the label it chose. It does not prove the label is right.
Conflict set	The rules that match at the same moment, from which the engine must pick one to fire first.
Coupling	How much one module depends on another's internal details.
Data structure	A way of organizing information so a program can store, find and update it.
Database	An organized collection of related data.
Denying the antecedent	An invalid argument: from P → Q and NOT P, concluding NOT Q.
Depth-first search (DFS)	Search that follows one route as deep as possible before trying another, using a stack.

Term	Meaning
Expert system	A program that applies specialist knowledge, stored as rules, in a narrow area such as medicine.
Fact	A stated piece of information about a case.
Foreign key	A column that refers to the primary key of another table.
Forward chaining	Starting from known facts and applying matching rules, adding conclusions as new facts.
Frame	A structure describing a typical object or situation, with slots for its properties and default values.
Frontier	In search, the states that have been found but not yet explored.
Graph	A representation of items and their connections.
Greedy best-first search	Search that always explores the state that looks closest to the goal by the heuristic alone.
Heuristic	An estimate of remaining cost or distance to a goal.
Hill climbing	Local search that always moves to the best neighbouring solution and stops when no neighbour is better.
Hybrid system	A system in which program logic handles input, storage and display while a rule-based component does the reasoning. Not the same as neuro-symbolic AI.
Inference engine	The part of a rule-based system that checks and applies rules.
Inheritance	In a semantic network or frames, a node automatically has the properties of the nodes above it.
Interpreter	A program that reads and carries out a high-level program one statement at a time.
Knowledge base	A store of rules and lasting domain knowledge.
Knowledge representation	How information is stored so a system can use it.
Lemmatization	Reducing a word to its dictionary form, such as “was” → “be”.
Linked list	A chain of nodes, each holding a value and a link to the next node.
Local maximum	A solution better than all its neighbours but not the best overall.
Machine learning	An AI approach in which a system learns statistical patterns from data.
Minimax	Choosing a move in a game by assuming the opponent always makes their best reply.
Modus ponens	From P → Q and P, conclude Q.
Modus tollens	From P → Q and NOT Q, conclude NOT P.
Neuro-symbolic AI	AI that combines neural networks (learning from data) with symbolic rules and logic. Optional in this module.
Ontology	A structured description of concepts and their relationships.
Operating system	System software that manages computer resources.
Overriding	A more specific frame replacing a default from a more general one.
Parse tree	A tree that shows how a sentence is built from grammar rules; also called a syntax tree.

Term	Meaning
Parsing	Finding how a sentence can be built from the rules of a grammar.
Predicate	A statement about an object or a relationship among objects.
Primary key	A field that uniquely identifies a row in a database table.
Process	A running instance of a program.
Proposition	A sentence that is either true or false, but not both.
Quantifier	∀ means "for every"; ∃ means "there is at least one".
Queue	A collection served first in, first out.
Recursion	Solving a problem with a function that calls itself on a smaller version of the problem.
Regression testing	Repeating earlier tests after a change, to check that existing behaviour still works.
Rule	An IF–THEN statement connecting conditions with a conclusion or action.
Script	A sequence of frames describing a typical series of events, such as borrowing a book.
SELECT query	An SQL instruction that asks a relational database for rows matching stated conditions.
Semantic network	A graph of concepts (nodes) joined by labelled relationships (links), such as is-a.
Simulated annealing	Local search that sometimes accepts worse moves, less and less often, to escape local maxima.
Software	Programs and procedures, with related data and documentation, that tell a computer system how to perform tasks.
Software life cycle	The stages of developing and maintaining software: understand, plan, write, test, and use and maintain.
Stack	A collection served last in, first out.
State space	The possible states and transitions in a search problem.
Stemming	Chopping word endings by rule, such as "studies" → "studi".
Stop-word removal	Removing common words such as "the" before searching documents.
Symbolic AI	An AI approach that represents knowledge explicitly with symbols, facts, and rules.
System software	Software that runs the computer and supports applications, above all the operating system.
Test case	An input and its expected result, used to check a program.
Time slice	The short turn on the processor that a process gets before the next one runs.
Tokenization	Splitting text into separate words or symbols.
Uniform-cost search (UCS)	Search that always explores the state with the lowest total cost so far; finds the cheapest route.
Working memory	Facts about the current case held while a rule-based system reasons.

RESOURCES, REFERENCES AND CREDITS

Resource index

Each resource is introduced next to the activity where you use it. [TEAM: watch each recommended video and add its most useful timestamps to its Resource box, test each link signed out, run Activities 9.2, 11.2 and Hands-on 12.2 as written, open Rulebound from the extracted ZIP on a second computer and on a phone, listen to its music, confirm whether the current Machine Learning for Kids Scratch blocks report confidence, and record here the date and the versions tested]

Resource	Used in	Link
Operating Systems (Crash Course #18)	Chapter 2	youtube.com
Intro to Algorithms (Crash Course #13)	Chapter 3	youtube.com
Data Structures (Crash Course #14)	Chapter 4	youtube.com
Rulebound (the team's optional game)	All chapters	`Supporting_Files/Game_Rulebound/`
SWISH (SWI-Prolog)	Activity 9.2	swish.swi-prolog.org
Try ELIZA (ELIZAGEN)	Activity 11.2	elizagen.org
Natural Language Processing (Crash Course #36)	Chapter 11	youtube.com
Scratch tutorials and starter projects	Hands-on 12.2	scratch.mit.edu
Machine Learning for Kids	Hands-on 12.2	machinelearningforkids.co.uk

YouTube may not open on some networks in mainland China. Every video above has an alternative beside it, and the guide contains everything needed for the Test Bank.

References

These are the sources behind the guide. Each chapter's closing box says which ones it uses.

- Brookshear, J. G., and Brylow, D. (2019). *Computer Science: An Overview* (13th ed.). Pearson.
- Buchanan, B. G., and Shortliffe, E. H. (Eds.). (1984). *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Addison-Wesley. Free chapters: people.dbmi.columbia.edu
- Davis, R., Buchanan, B., and Shortliffe, E. (1977). Production rules as a representation for a knowledge-based consultation program. *Artificial Intelligence*, 8(1), 15–45.
- Forouzan, B. A. (2018). *Foundations of Computer Science* (4th ed.). Cengage Learning EMEA.
- Gruber, T. R. (1993). A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2), 199–220.
- IEEE. (1990). *IEEE Standard Glossary of Software Engineering Terminology* (IEEE Std 610.12-1990).

- Jurafsky, D., and Martin, J. H. *Speech and Language Processing* (3rd ed., online draft). web.stanford.edu/~jurafsky/slp3
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.
- Lane, D. *Machine Learning for Kids*. machinelearningforkids.co.uk
- McCarthy, J., Minsky, M. L., Rochester, N., and Shannon, C. E. (1955). *A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence*.
- Minsky, M. (1974). *A Framework for Representing Knowledge* (MIT AI Laboratory Memo 306). Massachusetts Institute of Technology.
- Newell, A., and Simon, H. A. (1976). Computer science as empirical inquiry: Symbols and search. *Communications of the ACM*, 19(3), 113–126.
- OECD. (2019, amended 2024). *Recommendation of the Council on Artificial Intelligence* (OECD/LEGAL/0449). legalinstruments.oecd.org
- Russell, S. J., and Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed., Global Edition). Pearson.
- Schank, R. C., and Abelson, R. P. (1977). *Scripts, Plans, Goals and Understanding*. Lawrence Erlbaum.
- Scratch Foundation. *Scratch*. scratch.mit.edu
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
- SWI-Prolog. *SWISH*. swish.swi-prolog.org
- The Apache Software Foundation (Apache KIE). *Drools Documentation: Introduction*. kie.apache.org
- The professor's PPT 2024 course slides: chap3 Software and Algorithms, and chap4 Symbolic AI.
- Weizenbaum, J. (1966). ELIZA: a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1), 36–45.

Credits

- Figures and diagrams are the team's own, from Module Contents V3, except Figures 12.2 and 12.5 (the professor's PPT 2024, chap4, slide 50) and the screens of Scratch and Machine Learning for Kids in Figures 12.3, 12.4, 12.6 and 12.7.
- The river-crossing puzzle in Activity 10.1 is a traditional puzzle. Pip is an original character for this guide.
- Rulebound (Figures 0.2 and 0.3) is the team's own game, built with AI assistance, including its code, pixel art, music and questions. It uses no third-party code, art, music or fonts. Its characters and battlegrounds are original.
- Significant AI assistance is recorded in the team's project and personal logs.



Team contributions

Member	Role	Chapters led
Gunawan Nicolas Benedictus	Coordinator	1, 2, 6
Ritchard Alexander	Content Lead; Study Guide lead	3, 4
Eizel Mthana Conqueror	Visual Design Lead; Study Guide layout	5, 10
Frederic Chandra	Quality Lead	7, 12
Louis Alexander Dalim	Repository and Submission Lead	8, 9
Frederick Bryan The	Video Lead	11

[TEAM: confirm this table against who actually wrote each section]

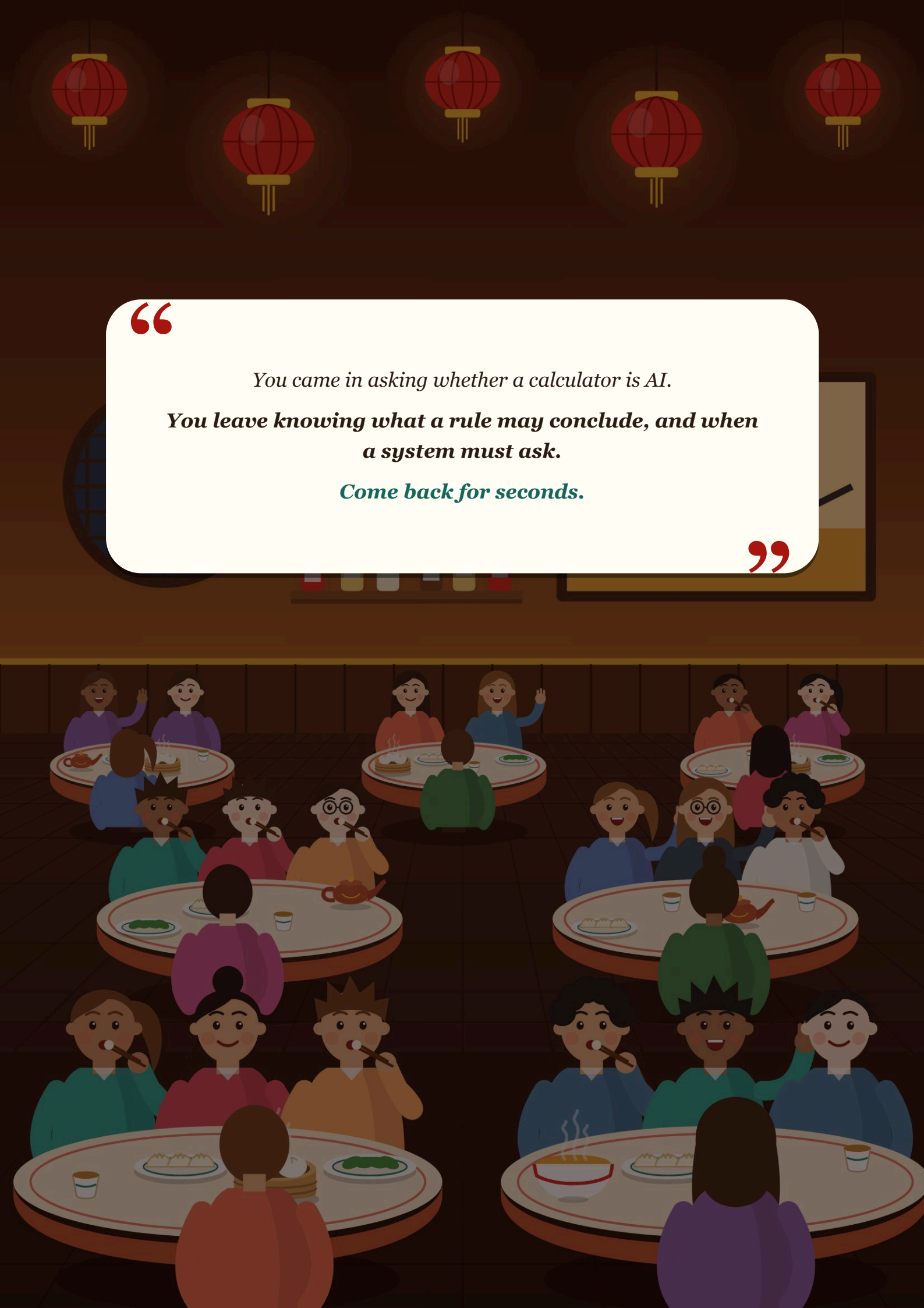
About this edition

This edition keeps the explanations, activities, answers and numbering of Study Guide V1.3, in a new layout. Its restaurant setting, chapter openers and front pages are new. The kitchen and the dishes are playful comparisons, not technical explanations: wherever they appear, the technical explanation in the chapter is the one to trust.

The book's format (full-page openers, one idea at a time, an activity card after each topic, and a closing page) is modelled on the Study Guide Book of the 2025 team BOTUMANJI, which we used as a format reference, as the Shared Assignment Rules allow. No text, questions or pictures were taken from it.

The restaurant pictures, the logo and Pip's waiter outfit were drawn for this project with AI assistance, and use no third-party art.




You came in asking whether a calculator is AI.
You leave knowing what a rule may conclude, and when a system must ask.
Come back for seconds.

Team 风味小吃 (Fengwei Xiaochi)

Software and Symbolic AI · Study Guide V1.4 · 2026

